

CSE480: Web Security

4. SSL/TLS & HTTPS

Seongil Wi

Web Threat Models

- **Network attacker:** resides somewhere in the communication link between client and server
 - Passive: eavesdropping
 - Active: modification of messages, replay...
- **Remote attacker:** can connect to remote system via the network
 - Mostly targets the server
- **Web attacker:** controls attacker.com
 - Can obtain SSL/TLS certificates for attacker.com
 - Users can visit attacker.com



Today's Topic



- **Network attacker:** resides somewhere in the communication link between client and server
 - Passive: eavesdropping
 - Active: modification of messages, replay...



- **Remote attacker:** can connect to remote system via the network
 - Mostly targets the server



- **Web attacker:** controls attacker.com
 - Can obtain SSL/TLS certificates for attacker.com
 - Users can visit attacker.com



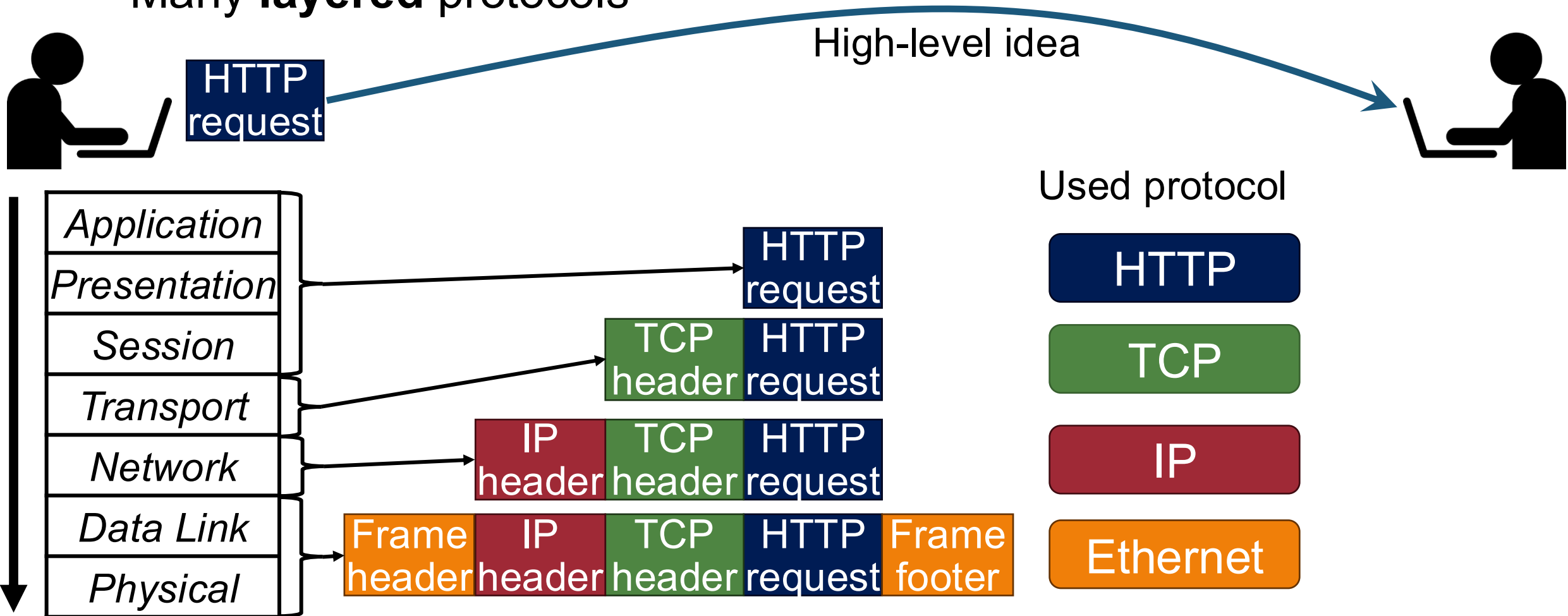
Protocol



- A system of digital **rules** for data exchange between computers

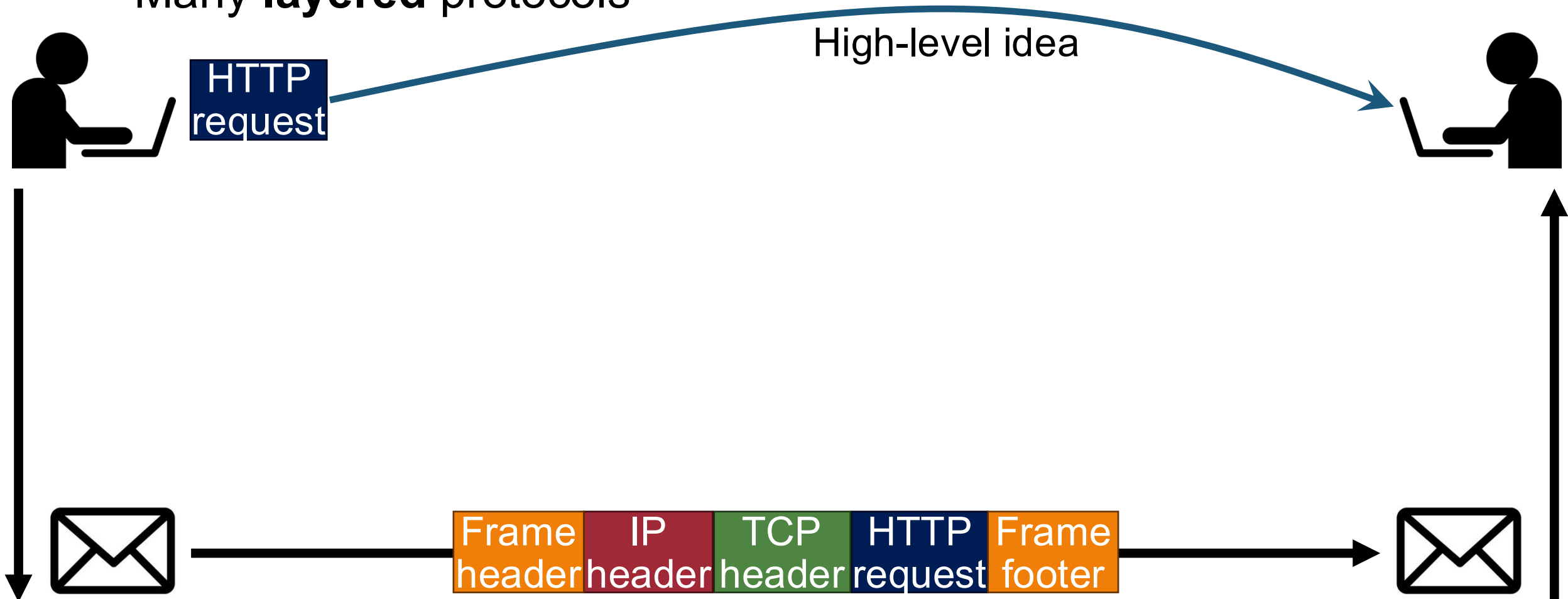
Protocol

- A system of digital **rules** for data exchange between computers
- Many **layered** protocols



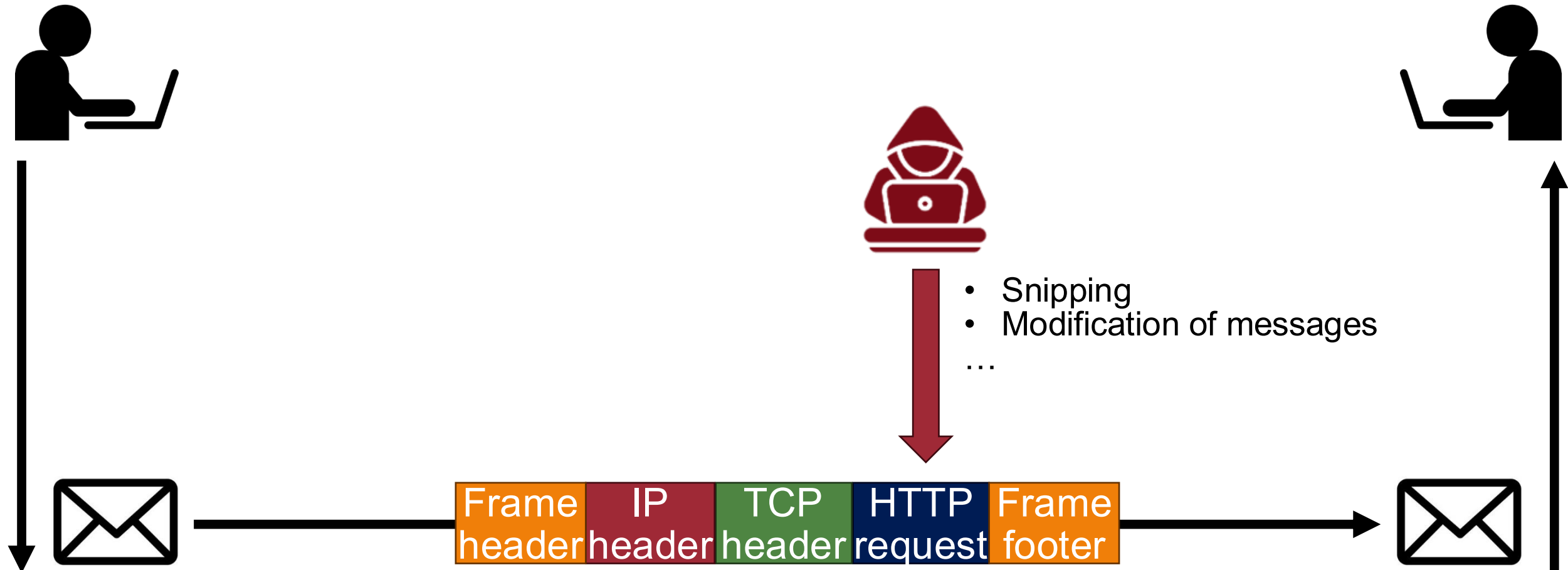
Protocol

- A system of digital **rules** for data exchange between computers
- Many **layered** protocols



Network Attackers

- A system of digital **rules** for data exchange between computers
- Many **layered** protocols



How is such an Attack Possible?

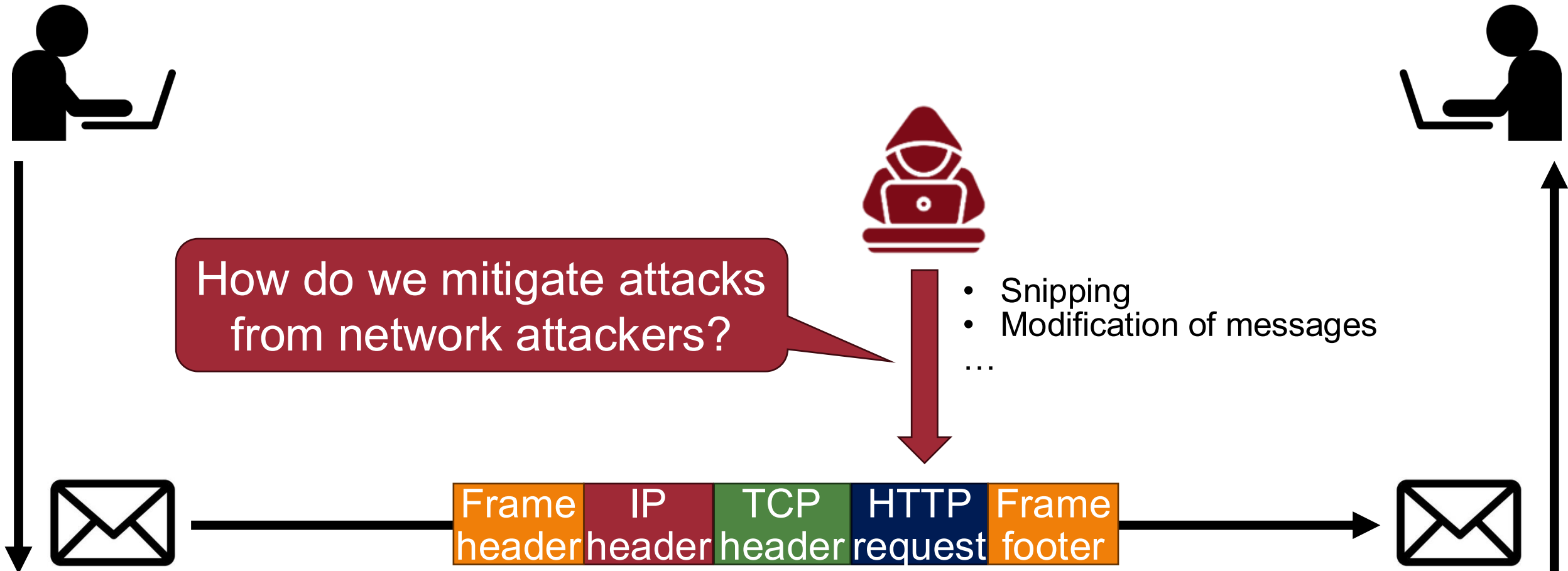


- Such attacks can be carried out using techniques such as:
 - ARP spoofing
 - DNS spoofing
 - BGP hijacking

- This topic is out of scope for this course!

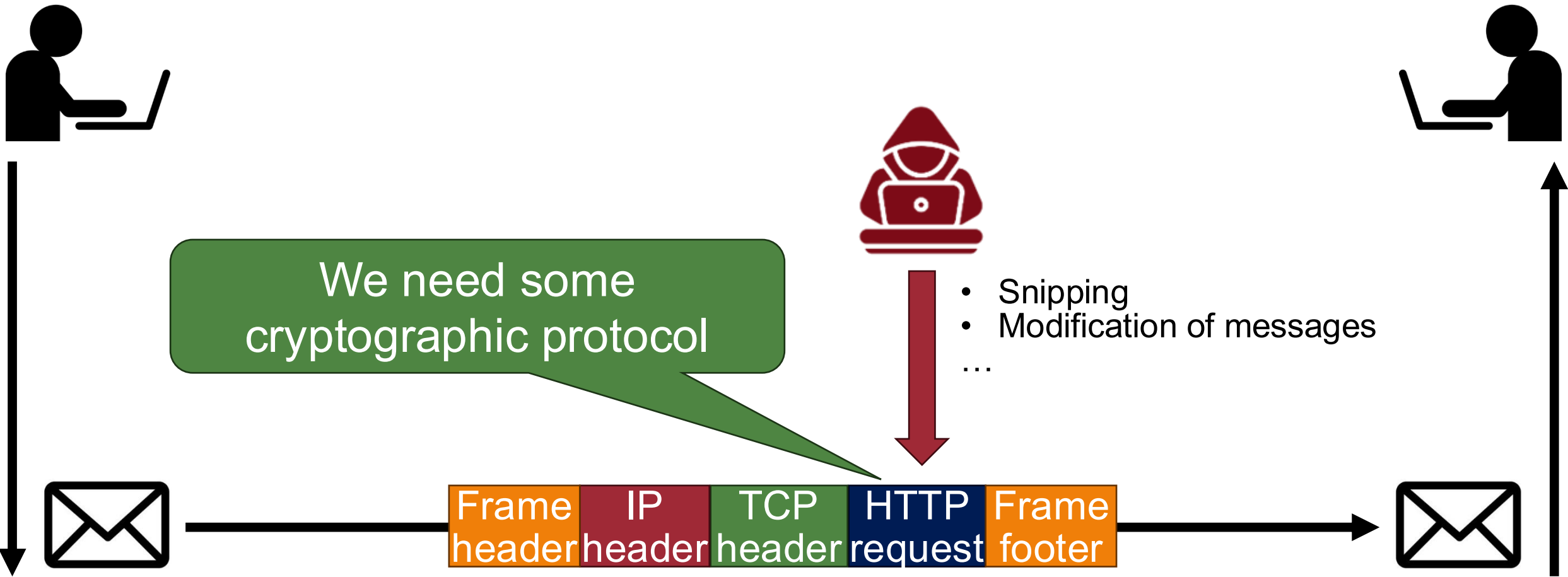
Motivative Question

9



Motivation: Cryptographical Protocol

10



SSL/TLS 

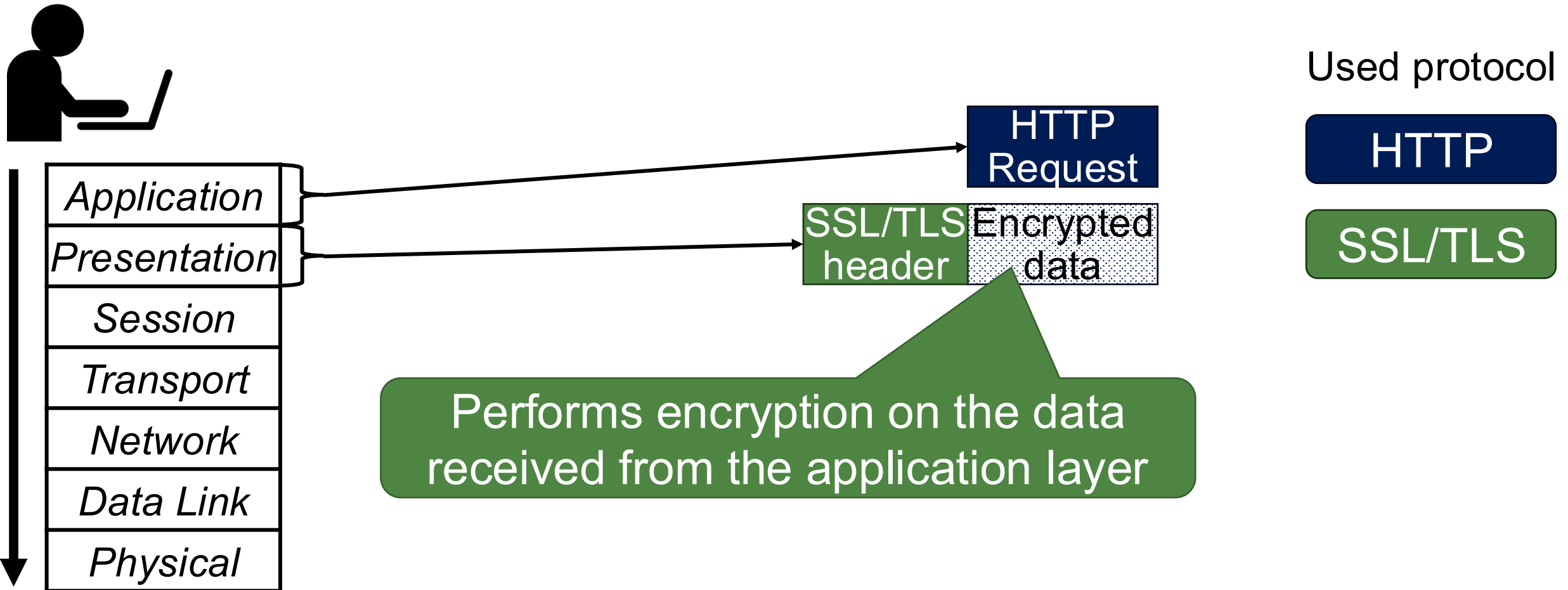
What is SSL/TLS?



- **Secure Sockets Layer (SSL) and Transport Layer Security (TLS) protocols**
 - Same protocol design, different crypto algorithms
 - (Reserved) port number: 443
- Security goals: achieving...
 - Confidentiality
 - Integrity
 - Authentication
- ***De facto* standard for Internet security**

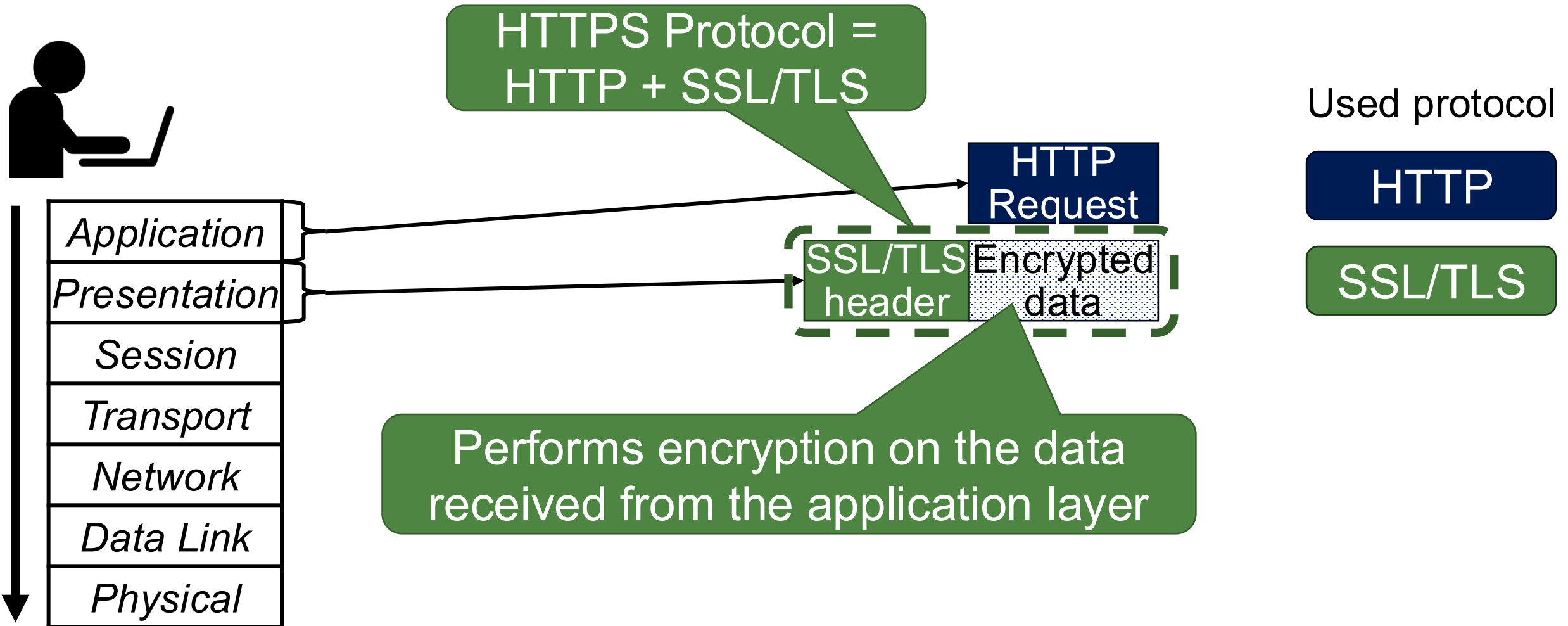
SSL/TLS Basic Idea

- Adding a protocol layer for secure communication!



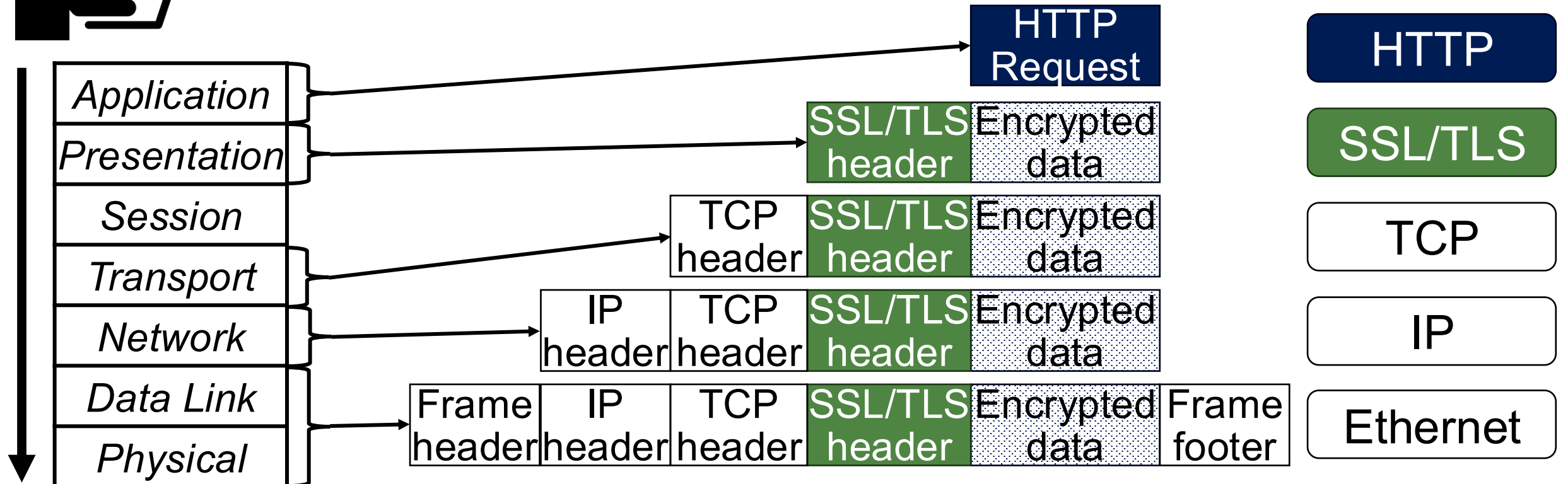
SSL/TLS Basic Idea

- Adding a protocol layer for secure communication!



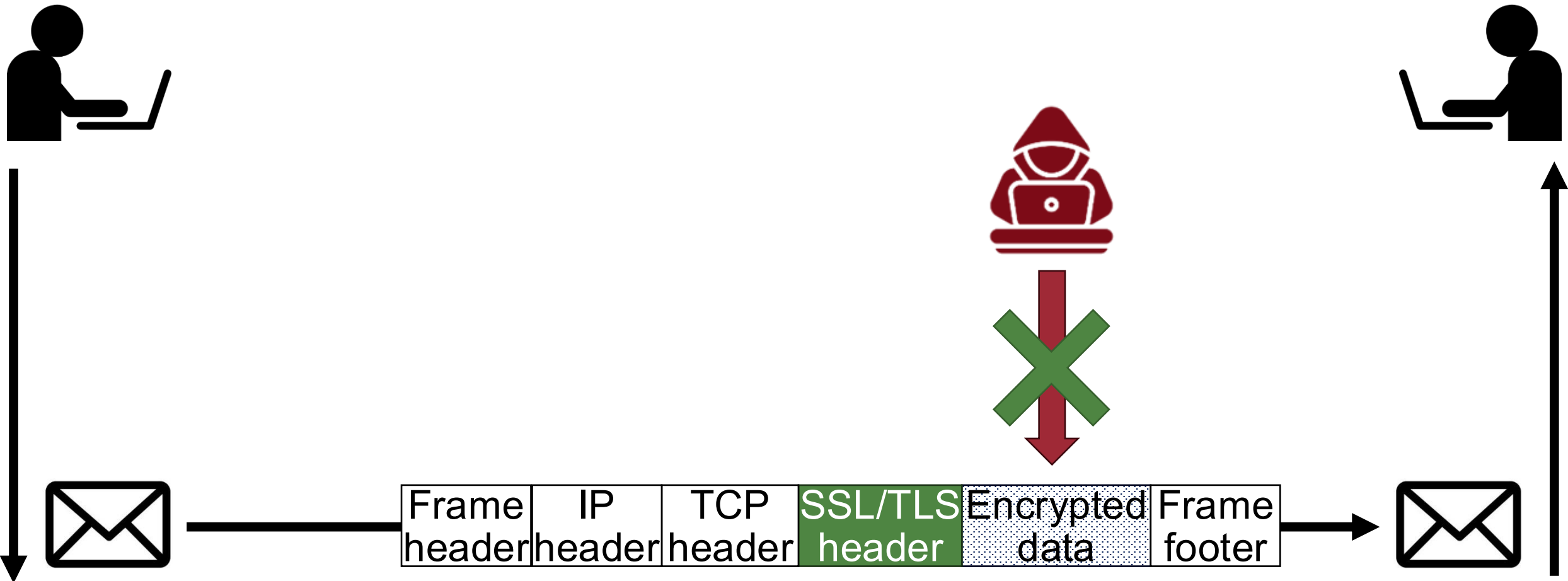
SSL/TLS Basic Idea

- Adding a protocol layer for secure communication!



SSL/TLS Basic Idea

- Adding a protocol layer for secure communication!



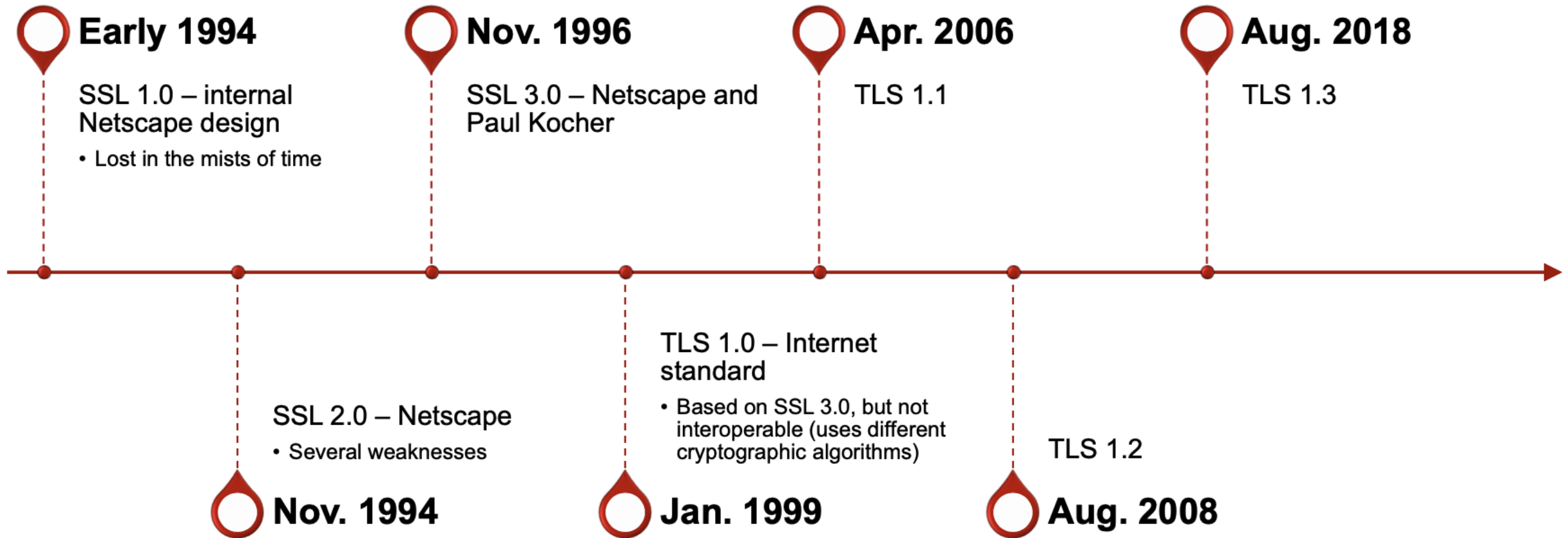
Use Cases



- Email
- Vice over IP (VoIP)
- Payment systems (transactions)
- **HTTPS**
 - The most publicly visible use case!
 - Deployed in every web browser

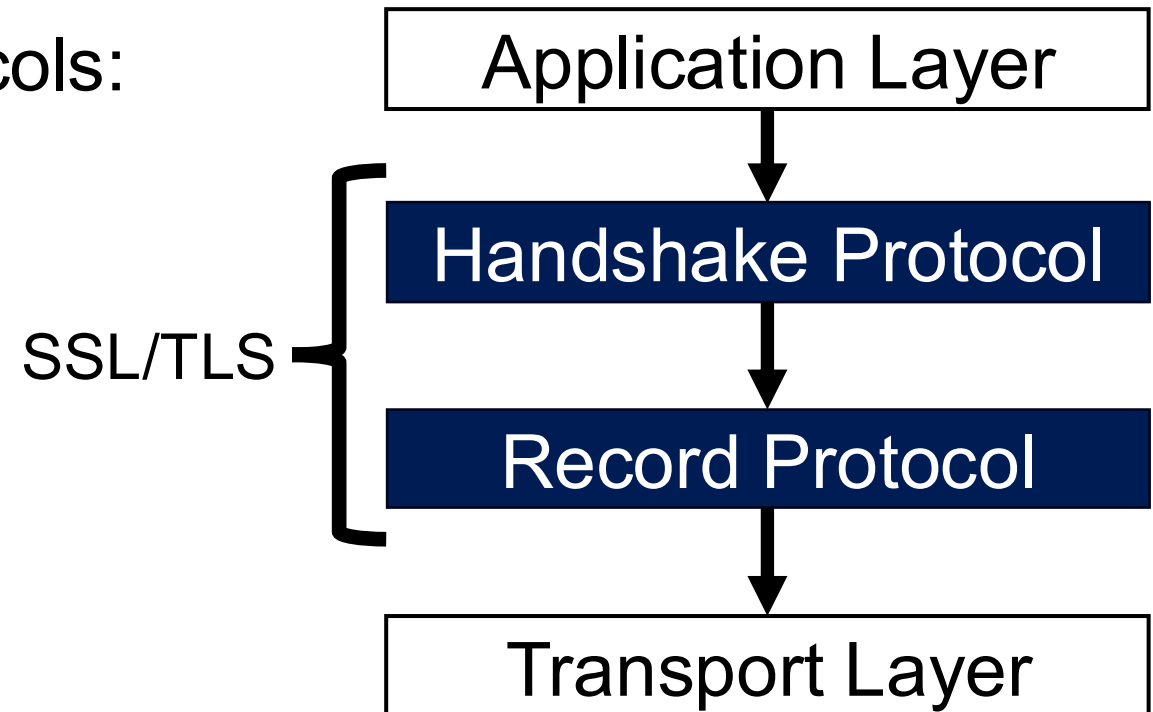
History of the Protocol

18



SSL/TLS Basics

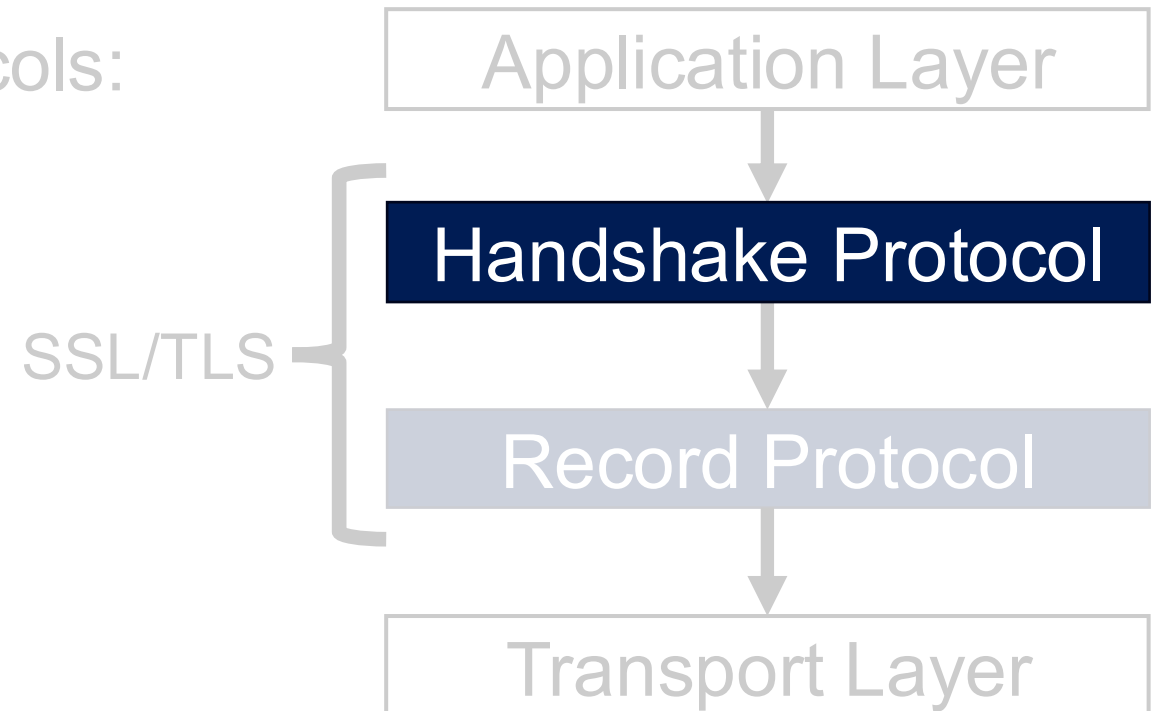
- Runs in the presentation layer
- Uses symmetric crypto, asymmetric crypto, and digital signatures
- Composed of two layers of protocols:
 1. Handshake protocol
 2. Record protocol



SSL/TLS Basics



- Runs in the presentation layer
- Uses symmetric crypto, asymmetric crypto, and digital signatures
- Composed of two layers of protocols:
 1. Handshake protocol
 2. Record protocol



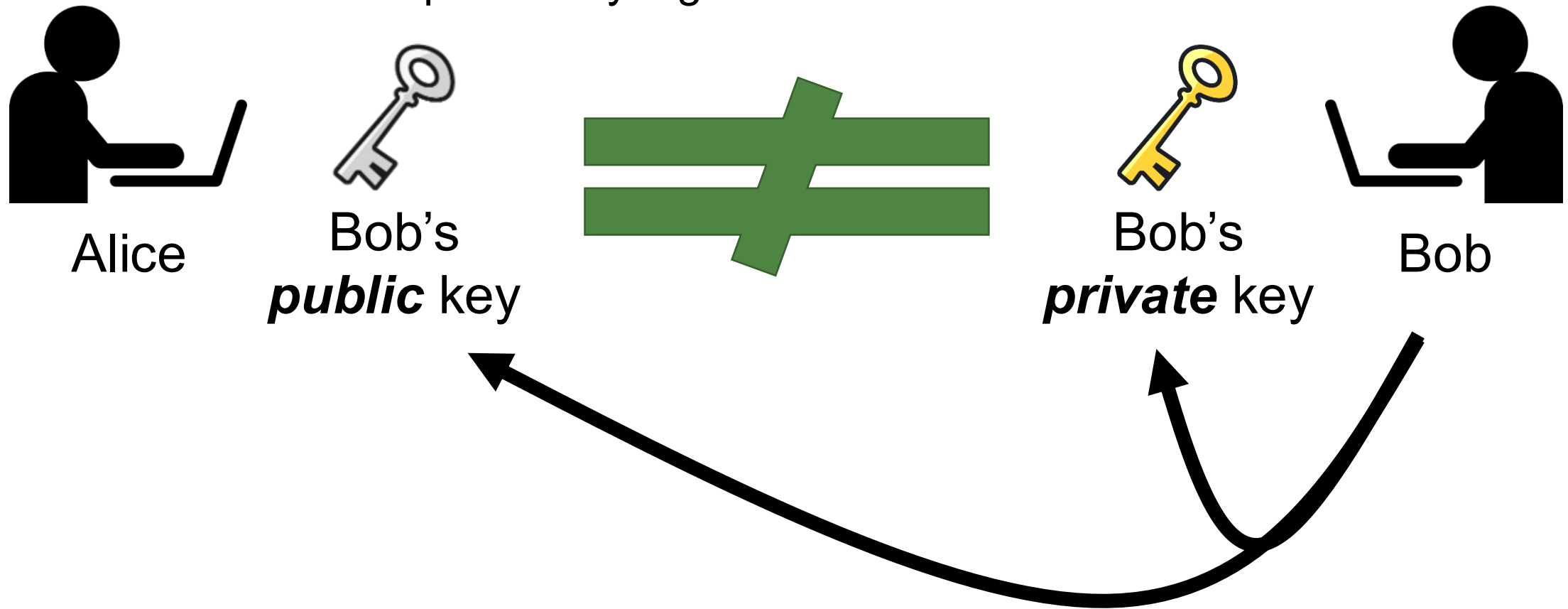
SSL/TLS Handshake Protocol



- The most complex part of SSL
- Uses asymmetric cryptography (public-key cryptography) to establish **several shared secret**

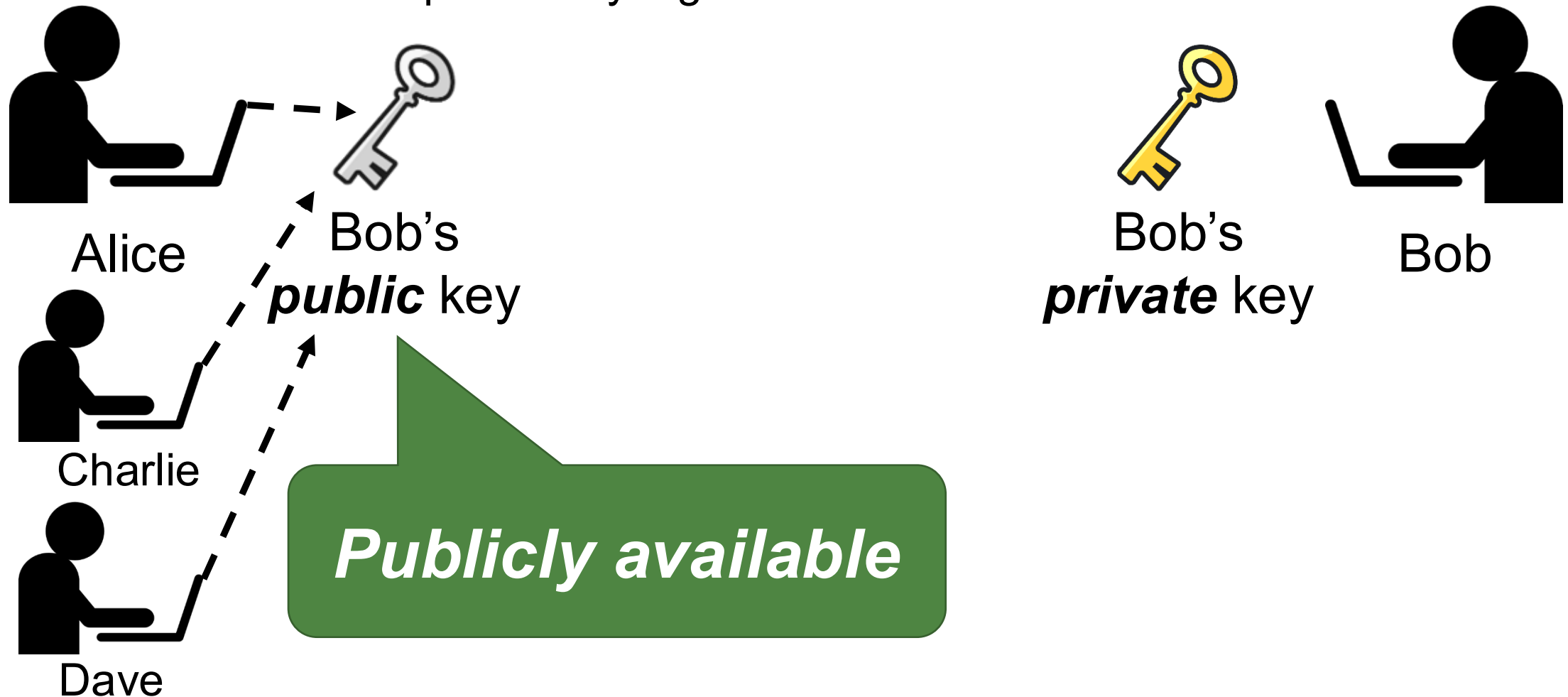
Ref: Asymmetric Key Cryptography

- Each party has two distinct keys: public key and private key
 - Also known as public-key algorithm



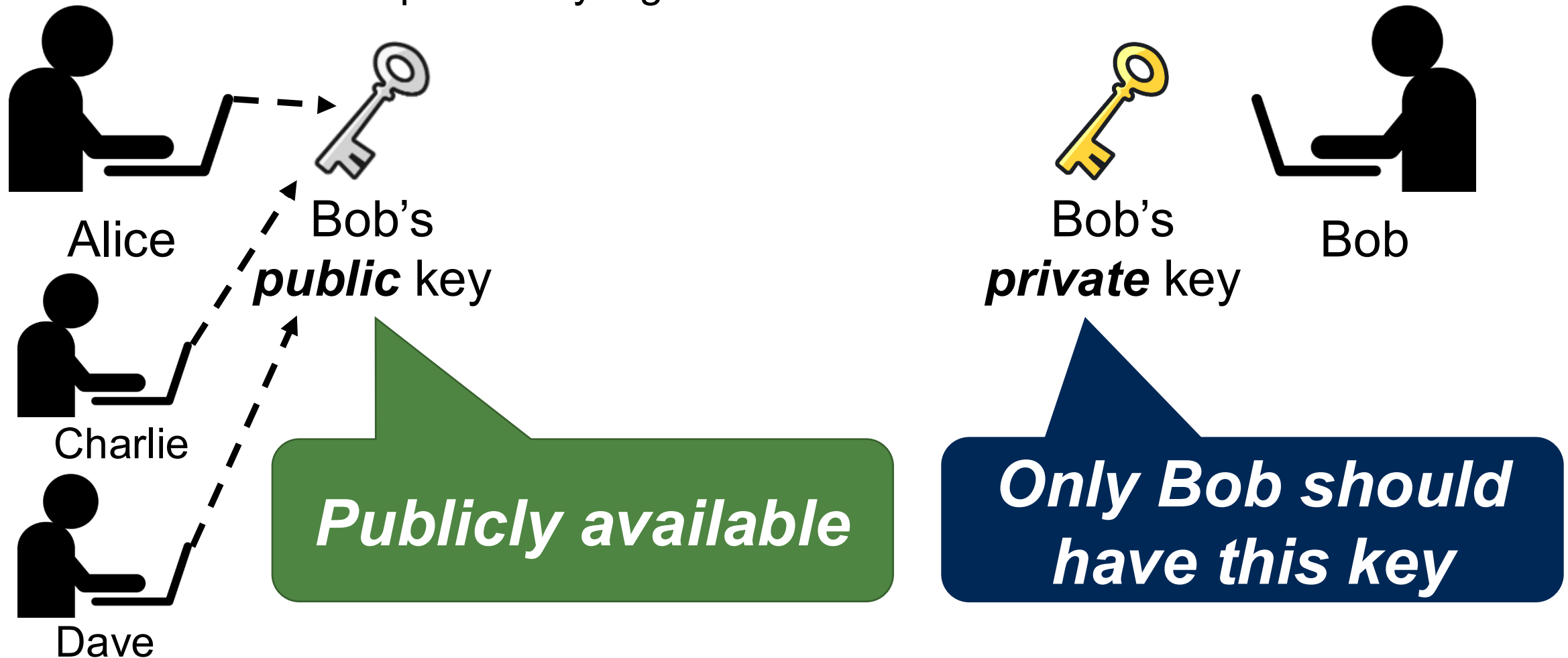
Ref: Asymmetric Key Cryptography

- Each party has two distinct keys: public key and private key
 - Also known as public-key algorithm



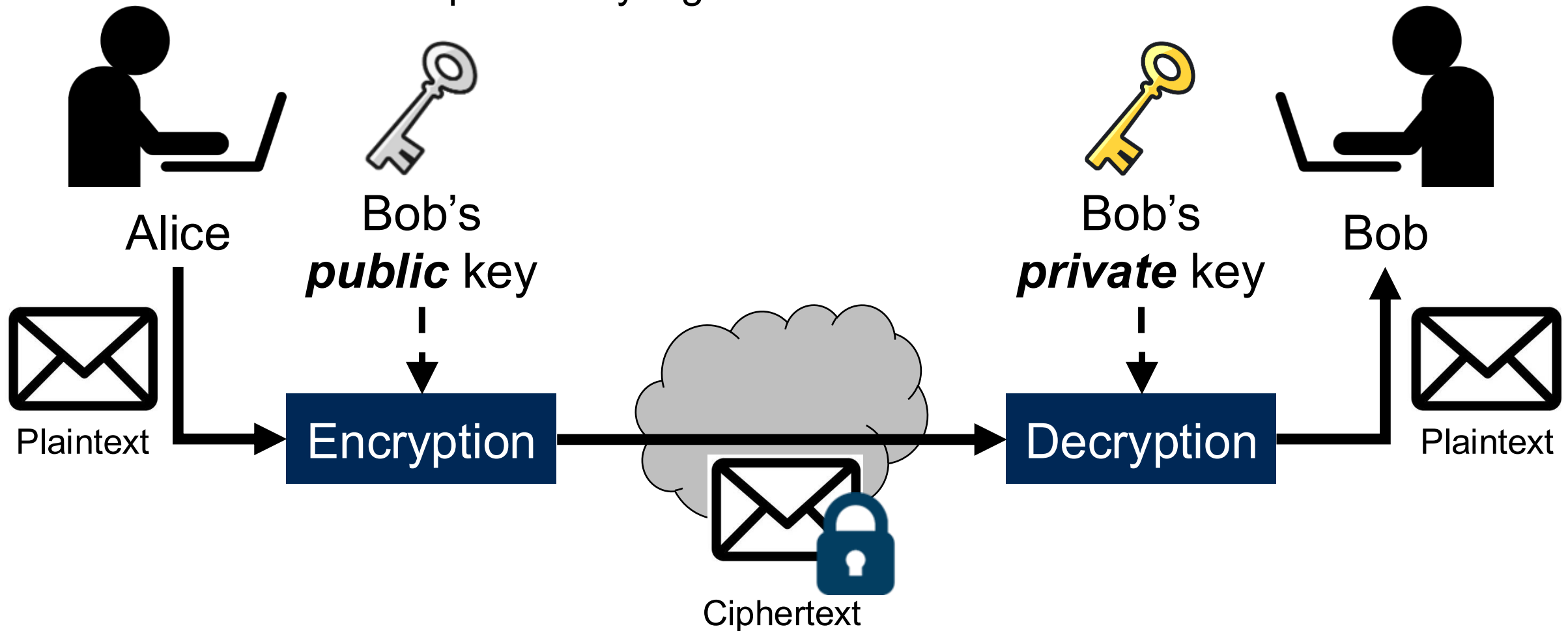
Ref: Asymmetric Key Cryptography

- Each party has two distinct keys: public key and private key
 - Also known as public-key algorithm

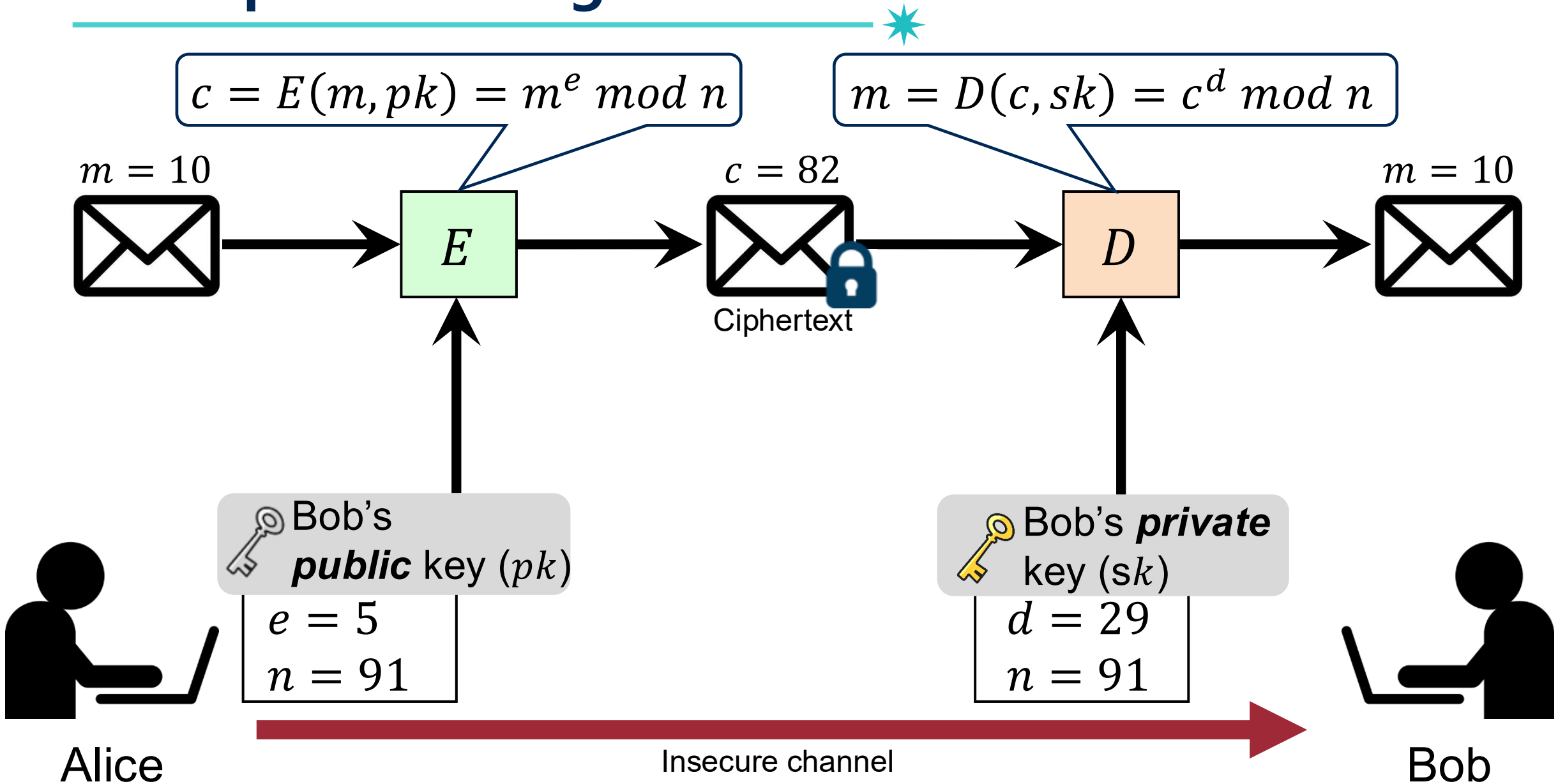


Ref: Asymmetric Key Cryptography

- Each party has two distinct keys: public key and private key
 - Also known as public-key algorithm



Example: RSA Algorithm

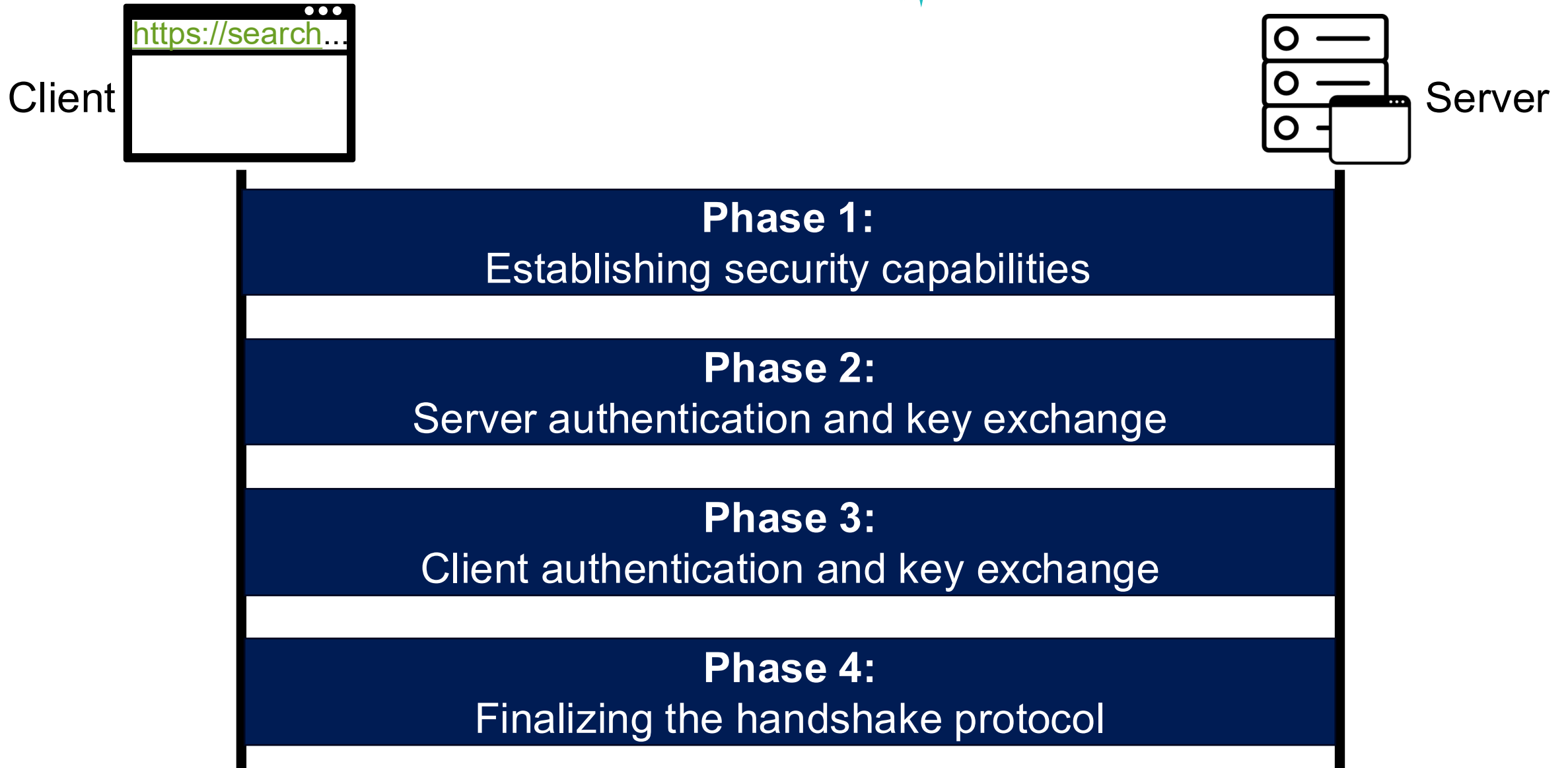


Recap: SSL/TLS Handshake Protocol



- The most complex part of SSL
- Uses asymmetric cryptography (public-key cryptography) to establish **several shared secret**

Four Phases of Handshake Protocol

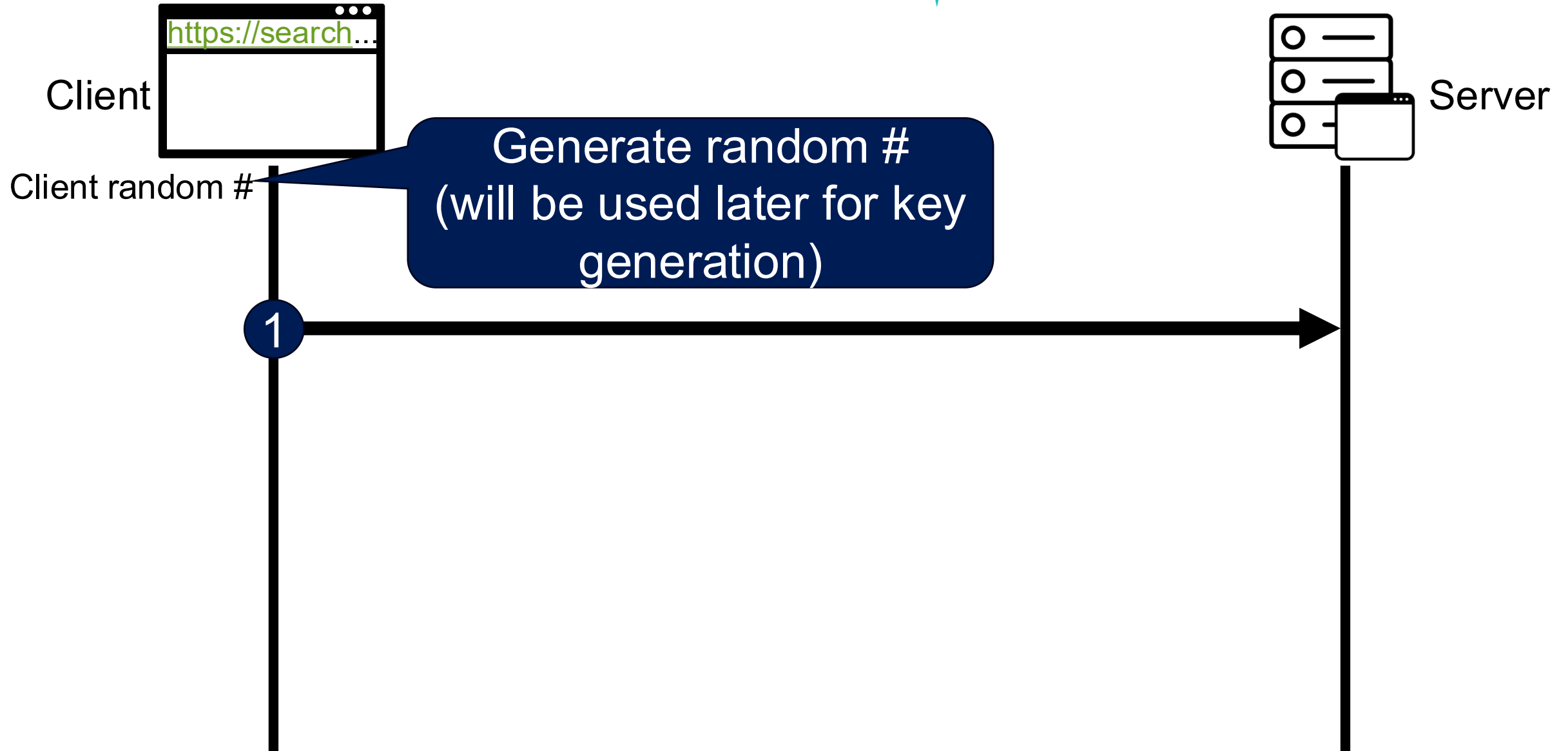


Phase 1: Establishing Security Capabilities ²⁹

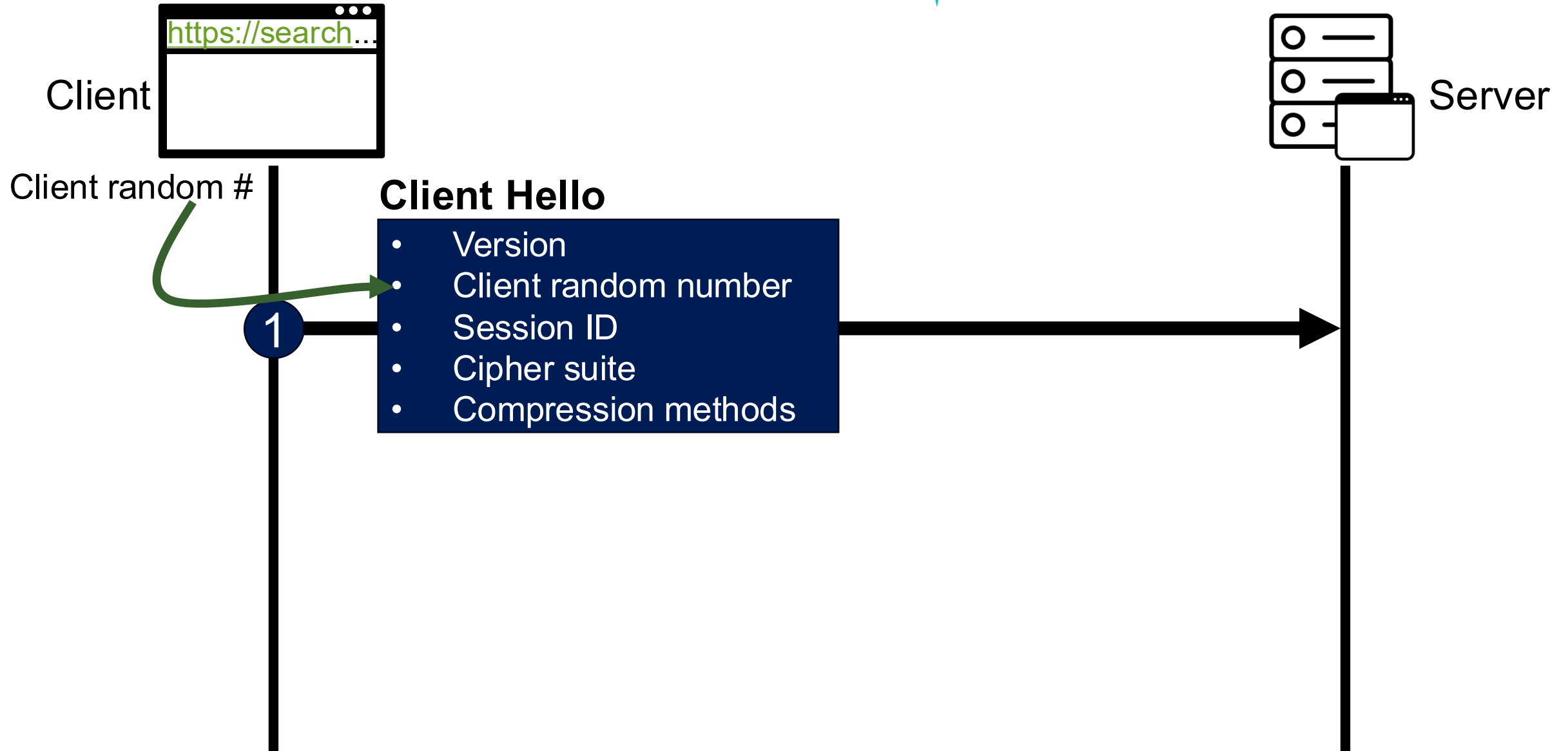


Phase 1: Establishing Security Capabilities

30



Phase 1: Establishing Security Capabilities ³¹



Phase 1 – Client Hello – Details

Client Hello – Details

- **Version**
 - Highest protocol version supported by the client
- **Client random number**
 - Random 32 bit time stamp + 28 random bytes
 - It will be used later for key generation
- **Session ID**
 - 0: establish new connection on new session
 - Non-zero: resume an old session
- **Cipher suite**
 - Set of cryptographic algorithms supported by the client
- **Compression methods**
 - Sequence of compression methods

Cipher Suites

Client Hello – Details

- **Version**
 - Highest protocol version supported by the client
- **Client random number**
 - Random 32 bit time stamp + 28 random bytes
 - It will be used later for key generation
- **Session ID**
 - 0: establish new connection on new session
 - Non-zero: resume an old session
- **Cipher suite**
 - Set of cryptographic algorithms supported by the client
- **Compression methods**
 - Sequence of compression methods

Format:

TLS_RSA_WITH_AES_128_CBC_SHA



Cipher Suites

Client Hello – Details

- **Version**
 - Highest protocol version supported by the client
- **Client random number**
 - Random 32 bit time stamp +
 - It will be used later for key generation
- **Session ID**
 - 0: establish
 - Non-zero: re
- **Cipher suite**
 - Set of cryptographic algorithms supported by the client
- **Compression methods**
 - Sequence of compression methods

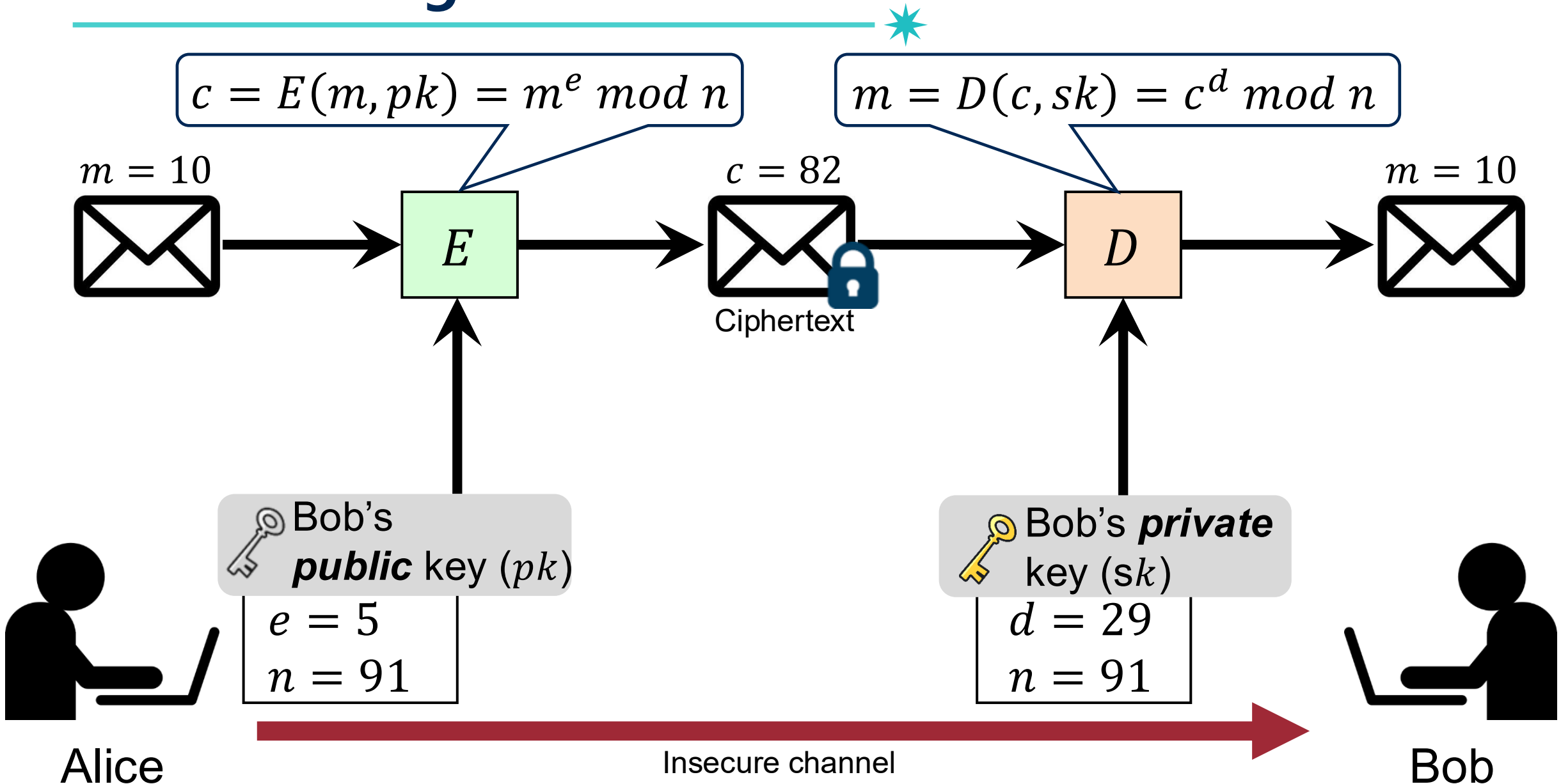
Format:

TLS_RSA_WITH_AES_128_CBC_SHA

Protocol

(Asymmetric)
Encryption/decryption algorithm
(for handshake protocol)

Ref: RSA Algorithm



Cipher Suites

Client Hello – Details

- **Version**
 - Highest protocol version supported by the client

- **Client random number**
 - Random 32 bit time stamp +
 - It will be used later for key generation

- **Session ID**
 - 0: establish
 - Non-zero: re

- **Cipher suite**

- Set of cryptographic algorithms supported by the client

- **Compression methods**
 - Sequence of compression methods

Format:

TLS_RSA_WITH_AES_128_CBC_SHA

Protocol

(Asymmetric)
Encryption/decryption algorithm
(for handshake protocol)

(Symmetric)
Encryption/decryption algorithm
(for record protocol)

Ref: Symmetric Key Cryptography

- The same key is used to encrypt/decrypt messages
 - Also known as secret key algorithm



Alice



key

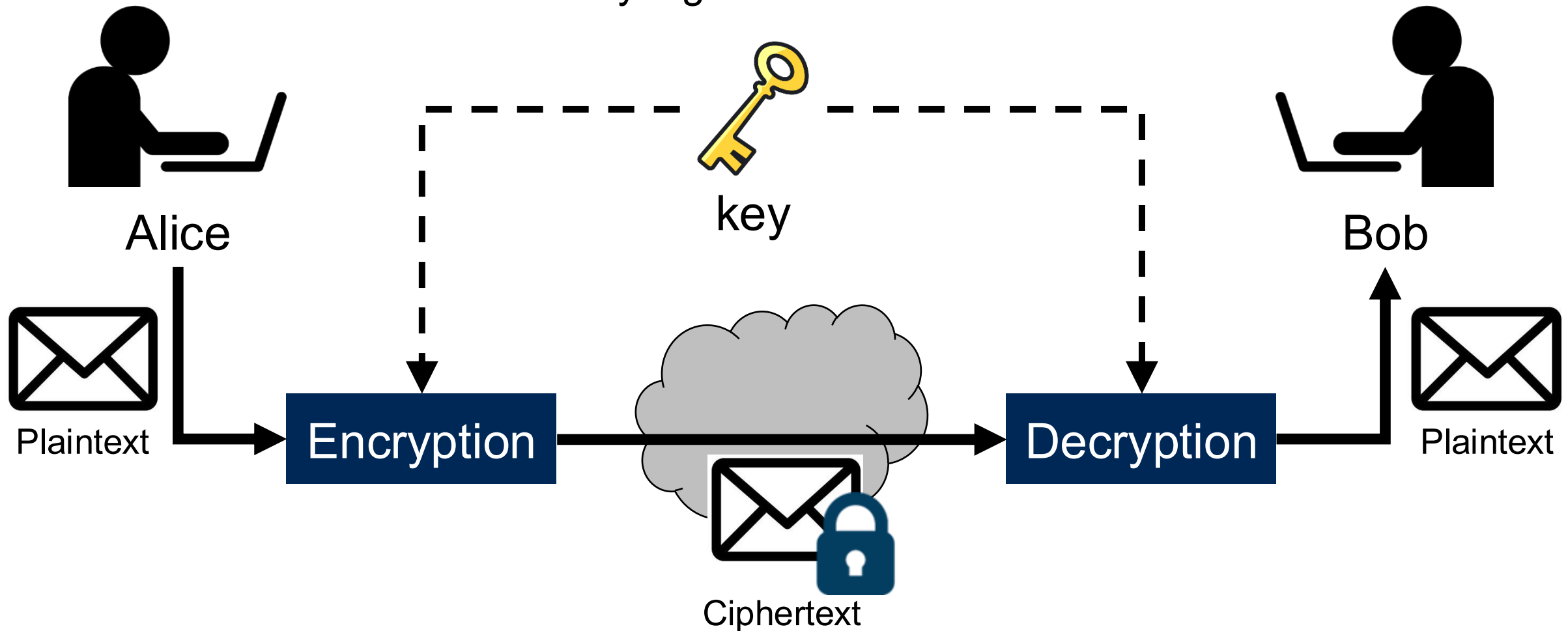


Bob

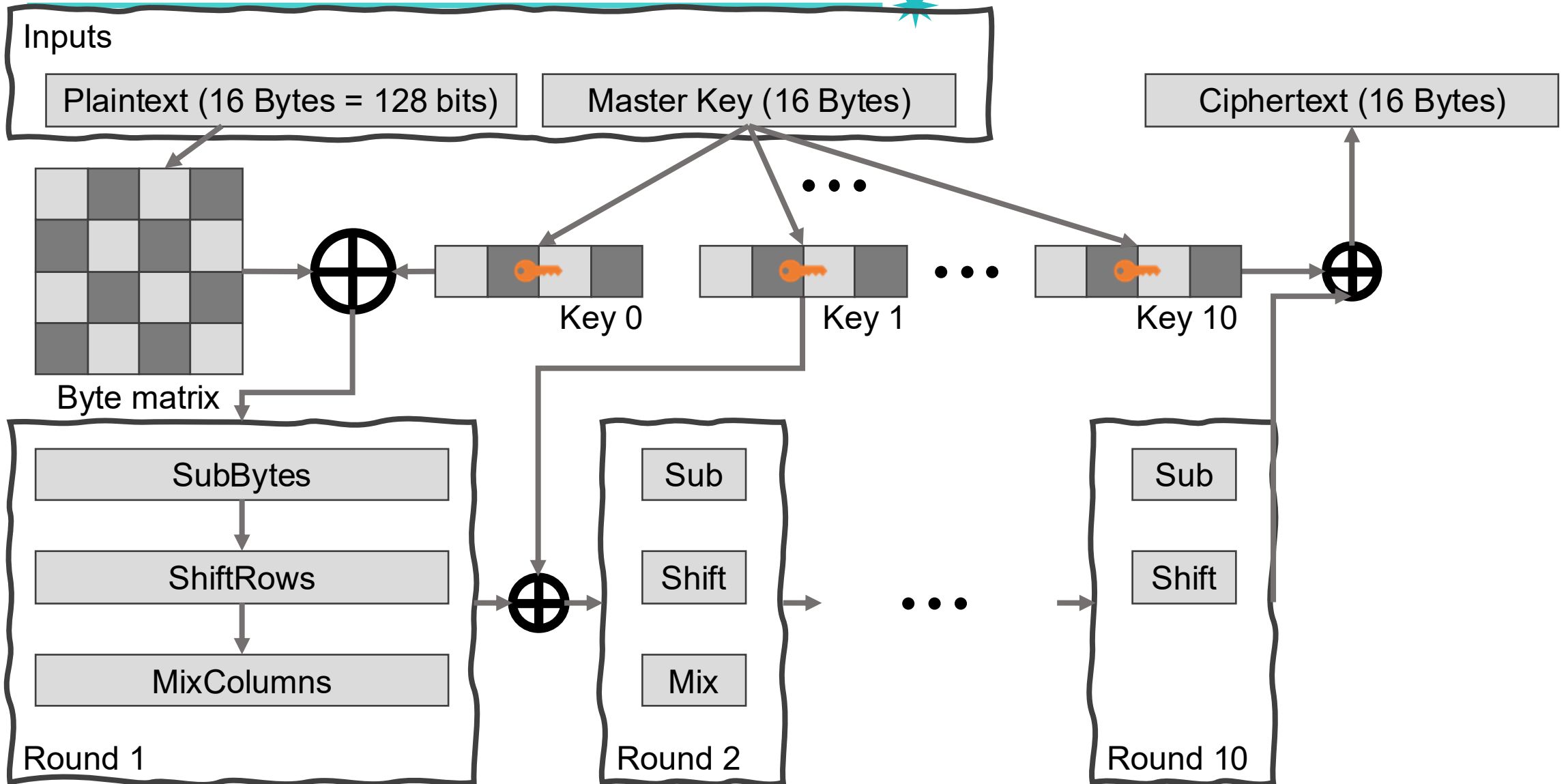
Shared secret key

Ref: Symmetric Key Cryptography

- The same key is used to encrypt/decrypt messages
 - Also known as secret key algorithm



Ref: Advanced Encryption Standard (AES) ³⁹



Cipher Suites

Client Hello – Details

- **Version**
 - Highest protocol version supported by the client

- **Client random number**
 - Random 32 bit time stamp +
 - It will be used later for key generation

Format:

TLS_RSA_WITH_AES_128_CBC_SHA

Protocol

- **Session ID**
 - 0: establish
 - Non-zero: re

(Asymmetric)
Encryption/decryption algorithm
(for key exchange)

- **Cipher suite**

- Set of cryptographic algorithms supported by the client

(Symmetric)
Encryption/decryption algorithm
(for data exchange)

- **Compression methods**
 - Sequence of compression methods

Cipher Suite – Example



Cipher Suite	Key Exchange	Cipher	MAC
TLS_NULL_WITH_NULL_NULL	NULL	NULL	NULL
TLS_RSA_WITH_NULL_MD5	RSA	NULL	MD5
TLS_RSA_WITH_NULL_SHA	RSA	NULL	SHA
TLS_RSA_WITH_NULL_SHA256	RSA	NULL	SHA256
TLS_RSA_WITH_RC4_128_MD5	RSA	RC4_128	MD5
TLS_RSA_WITH_RC4_128_SHA	RSA	RC4_128	SHA
TLS_RSA_WITH_3DES_EDE_CBC_SHA	RSA	3DES_EDE_CBC	SHA
TLS_RSA_WITH_AES_128_CBC_SHA	RSA	AES_128_CBC	SHA
TLS_RSA_WITH_AES_256_CBC_SHA	RSA	AES_256_CBC	SHA
TLS_RSA_WITH_AES_128_CBC_SHA256	RSA	AES_128_CBC	SHA256
TLS_RSA_WITH_AES_256_CBC_SHA256	RSA	AES_256_CBC	SHA256
TLS_DH_anon_WITH_RC4_128_MD5	DH_anon	RC4_128	MD5
TLS_DH_anon_WITH_3DES_EDE_CBC_SHA	DH_anon	3DES_EDE_CBC	SHA
TLS_DH_DSS_WITH_AES_128_CBC_SHA	DH_DSS	AES_128_CBC	SHA
TLS_DH_RSA_WITH_AES_128_CBC_SHA	DH_RSA	AES_128_CBC	SHA
TLS_DHE_DSS_WITH_AES_128_CBC_SHA	DHE_DSS	AES_128_CBC	SHA
TLS_DHE_RSA_WITH_AES_128_CBC_SHA	DHE_RSA	AES_128_CBC	SHA
TLS_DH_anon_WITH_AES_128_CBC_SHA	DH_anon	AES_128_CBC	SHA
TLS_DH_DSS_WITH_AES_256_CBC_SHA	DH_DSS	AES_256_CBC	SHA
TLS_DH_RSA_WITH_AES_256_CBC_SHA	DH_RSA	AES_256_CBC	SHA
TLS_DHE_DSS_WITH_AES_256_CBC_SHA	DHE_DSS	AES_256_CBC	SHA
TLS_DHE_RSA_WITH_AES_256_CBC_SHA	DHE_RSA	AES_256_CBC	SHA
TLS_DH_anon_WITH_AES_256_CBC_SHA	DH_anon	AES_256_CBC	SHA

No protection

Uses RSA (certificate) for key exchange, AES 256 in CBC mode for encryption and SHA256 as MAC

Uses ephemeral Diffie- Hellman with RSA for key exchange, AES 256 CBC for encryption and SHA256 as MAC

Cipher Suites

Client Hello –

- **Version**
 - Highest protocol version supported by the client
- **Client random number**
 - 32-byte random number generated by the client
- **Session ID**
 - 0: establish new connection
 - Non-zero: resume an old session
- **Cipher suite**
 - Set of cryptographic algorithms supported by the client
- **Compression methods**
 - Sequence of compression methods supported by the client

In decreasing order of preference

Transport Layer Security

TLSv1.2 Record Layer: Handshake Protocol: Client Hello

Content Type: Handshake (22)

Version: TLS 1.0 (0x0301)

Length: 512

Handshake Protocol: Client Hello

Handshake Type: Client Hello (1)

Length: 508

Version: TLS 1.2 (0x0303)

> Random: 1396873af8d56db07f55a31afba6c98a04e00025005764fe...

Session ID Length: 32

Session ID: fe329526917d48c5af72228bdc801142894fe91f4a548f7...

Cipher Suites Length: 34

Cipher Suites (17 suites)

Cipher Suite: Reserved (GREASE) (0x3a3a)

Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)

Cipher Suite: TLS_AES_256_GCM_SHA384 (0x1302)

Cipher Suite: TLS_CHACHA20_POLY1305_SHA256 (0x1303)

Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)

Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)

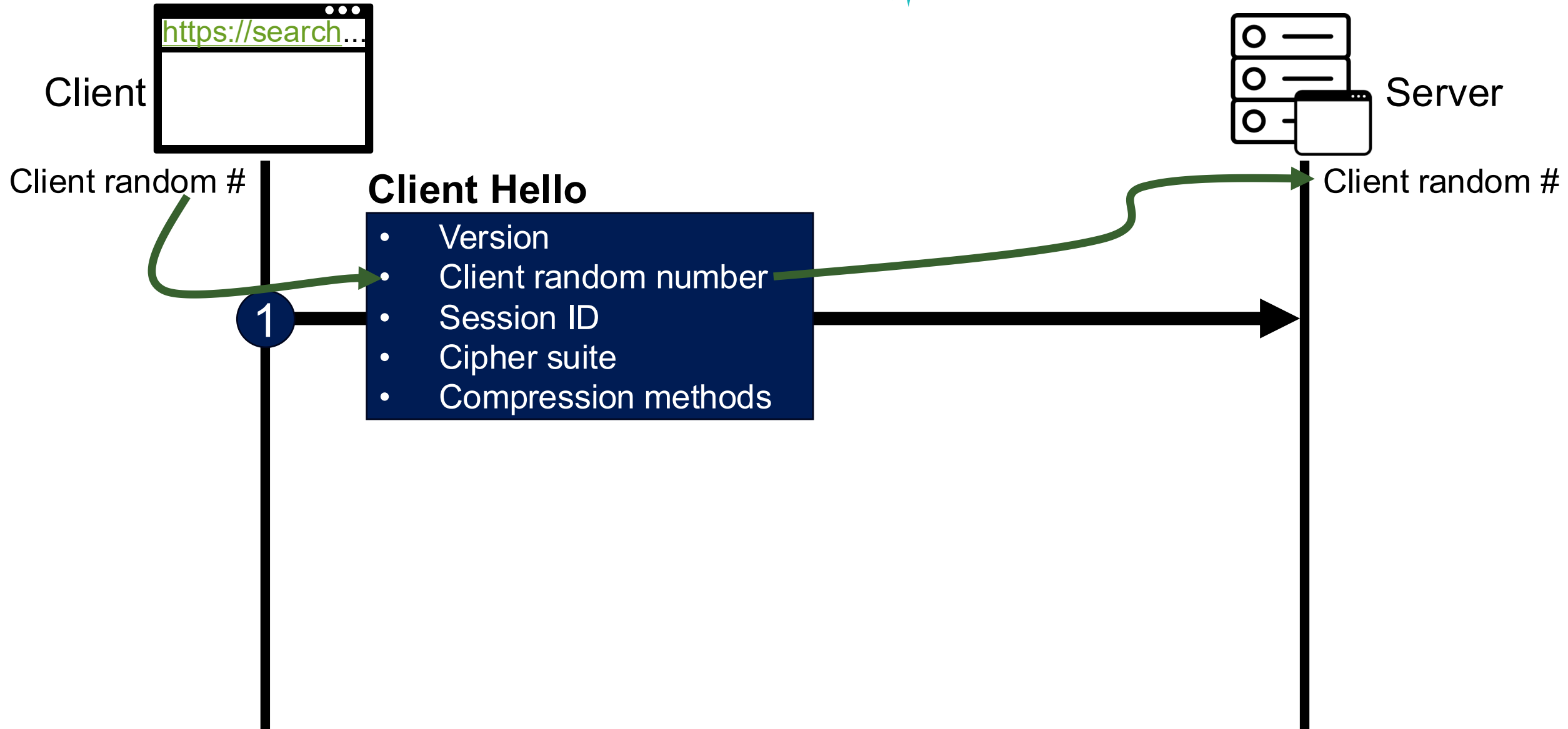
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c)

Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)

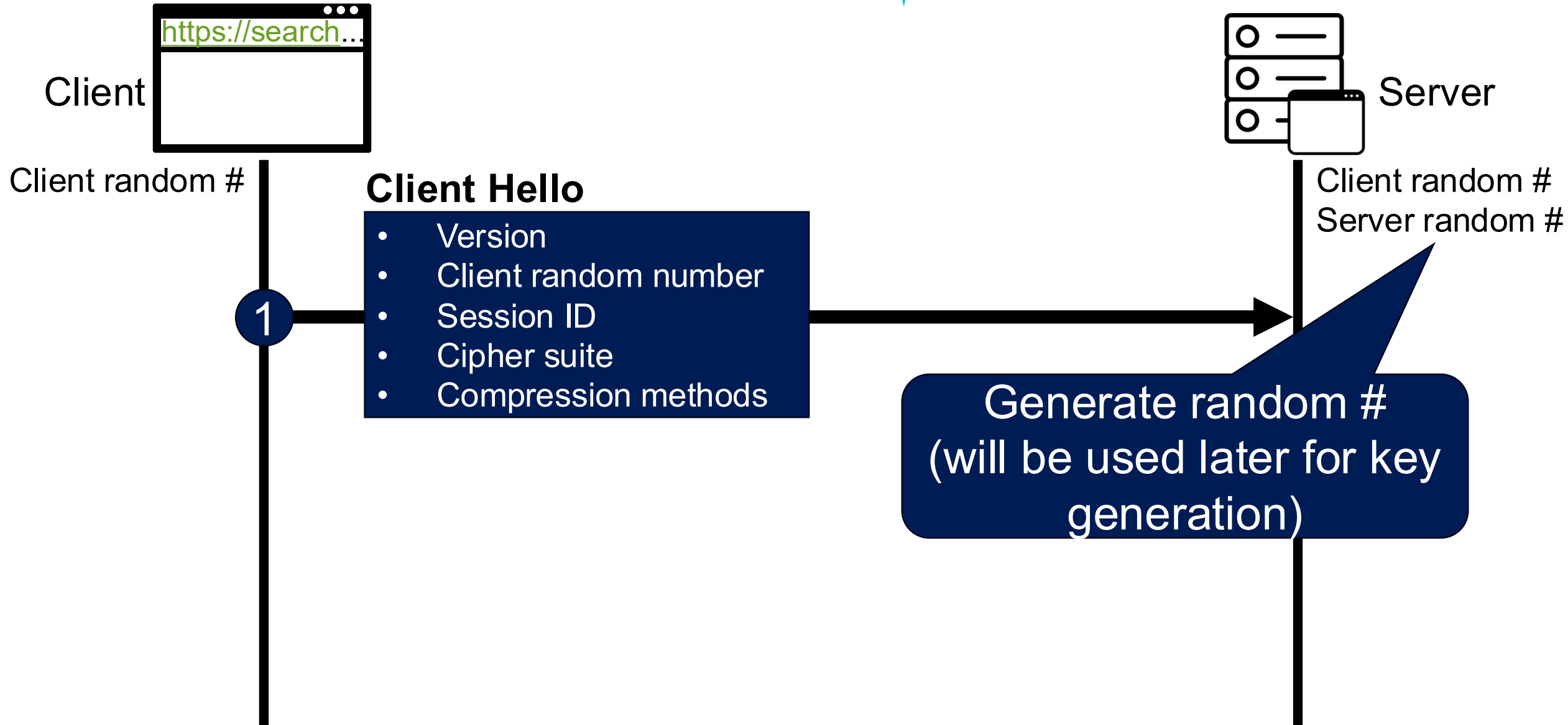
Cipher Suite: TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca9)

Cipher Suite: TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8)

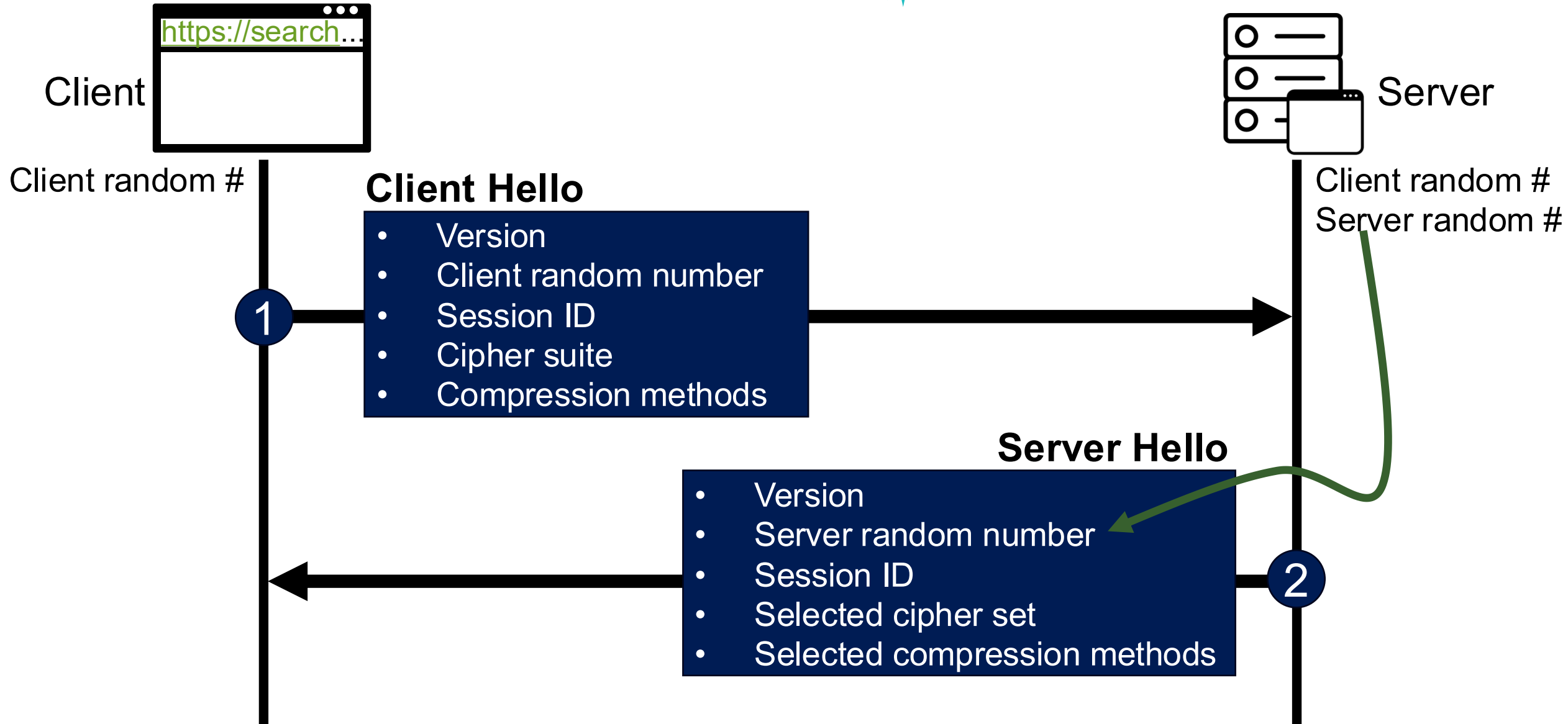
Phase 1: Establishing Security Capabilities ⁴³



Phase 1: Establishing Security Capabilities ⁴⁴

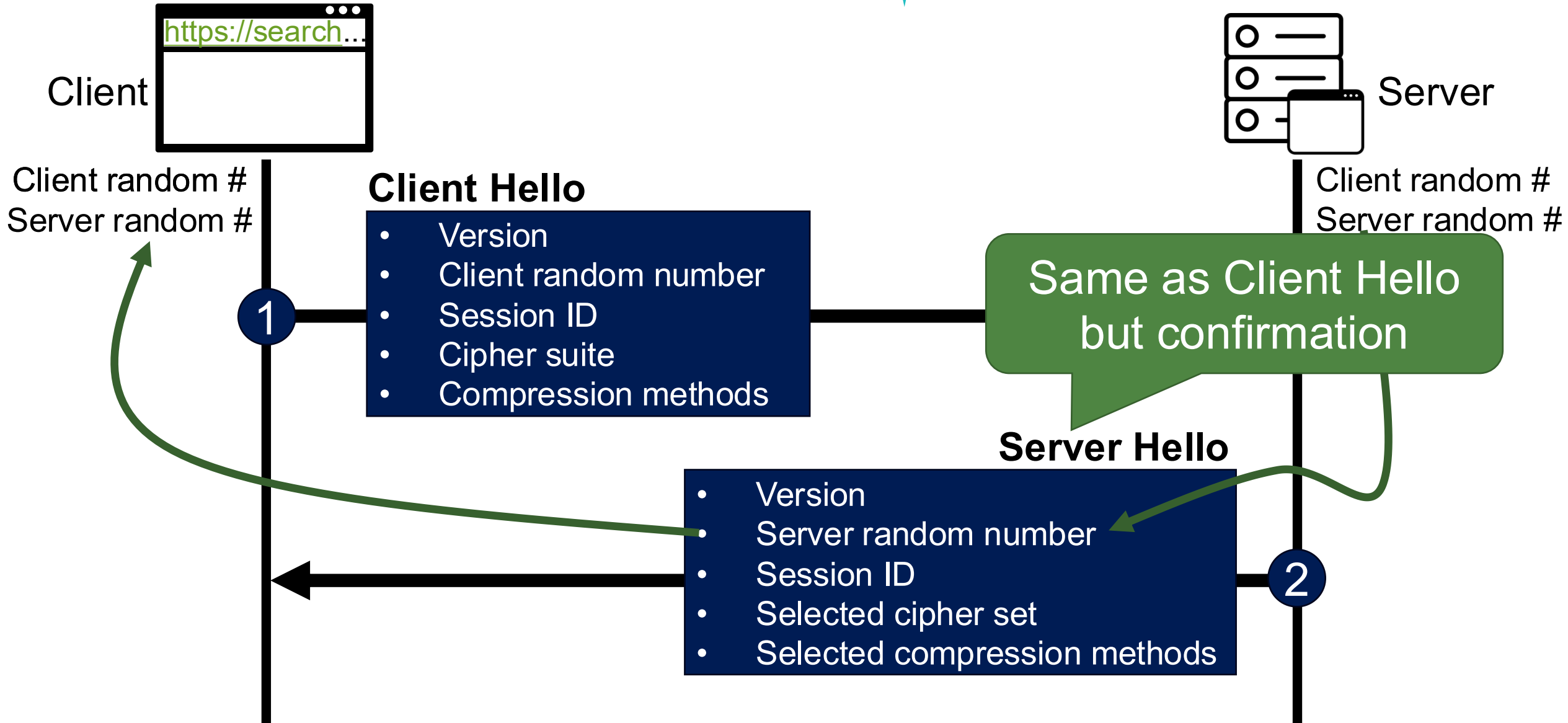


Phase 1: Establishing Security Capabilities ⁴⁵



Phase 1: Establishing Security Capabilities

46



Phase 1 – Server Hello – Details

Client Hello – Details

- **Version**
 - Highest protocol version supported by the client
- **Client random number**
 - Random 32 bit time stamp + 28 random bytes
 - It will be used later for key generation
- **Session ID**
 - 0: establish new connection on new session
 - Non-zero: resume an old session
- **Cipher suite**
 - Set of cryptographic algorithms supported by the client
- **Compression methods**
 - Sequence of compression methods

Server Hello – Details

- **Version**
 - Highest common version
- **Server random number**
 - Random 32 bit time stamp + 28 random bytes
 - It will be used later for key generation
- **Session ID**
 - New session ID if zero, old session ID otherwise
- **Cipher suite**
 - The selected cipher suite
- **Compression methods**
 - The selected compression technique

▼ TLSv1.2 Record Layer: Handshake Protocol: Server Hello

Content Type: Handshake (22)

Version: TLS 1.2 (0x0303)

Length: 78

▼ Handshake Protocol: Server Hello

Handshake Type: Server Hello (2)

Length: 74

Version: TLS 1.2 (0x0303)

> Random: 3896a769b30ae8f9cd0dcd3eb1d58aa4d7a12e2c5ca7...47b...

Session ID Length: 0

Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)

Compression Method: null (0)

Extensions Length: 34

> Extension: renegotiation_info (len=1)

> Extension: server_name (len=0)

> Extension: ec_point_formats (len=4)

> Extension: session_ticket (len=0)

> Extension: application_layer_protocol_negotiation (len=5)

> Extension: extended_master_secret (len=0)

Selected cipher suite

number

timestamp + 28 random bytes
for key generation

zero, old session ID

suite

methods

session technique

Phase 1: Establishing Security Capabilities ⁴⁹



After Phase 1, the client and server know the followings:

- The version of SSL/TLS
- The algorithms for key exchange and encryption
- The compression method
- The two random numbers for key generation

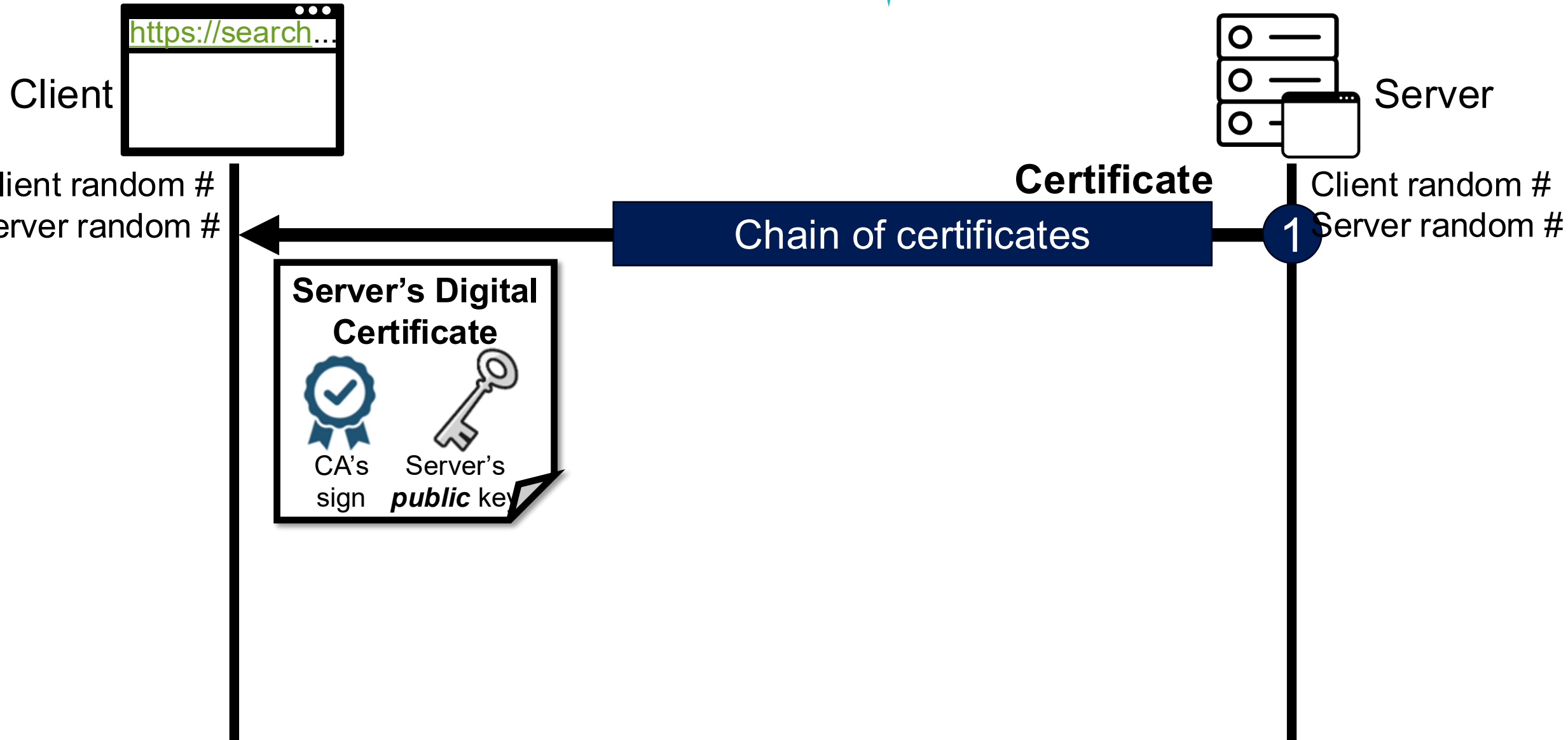
Phase 2: Server Auth. and Key Exchange

50



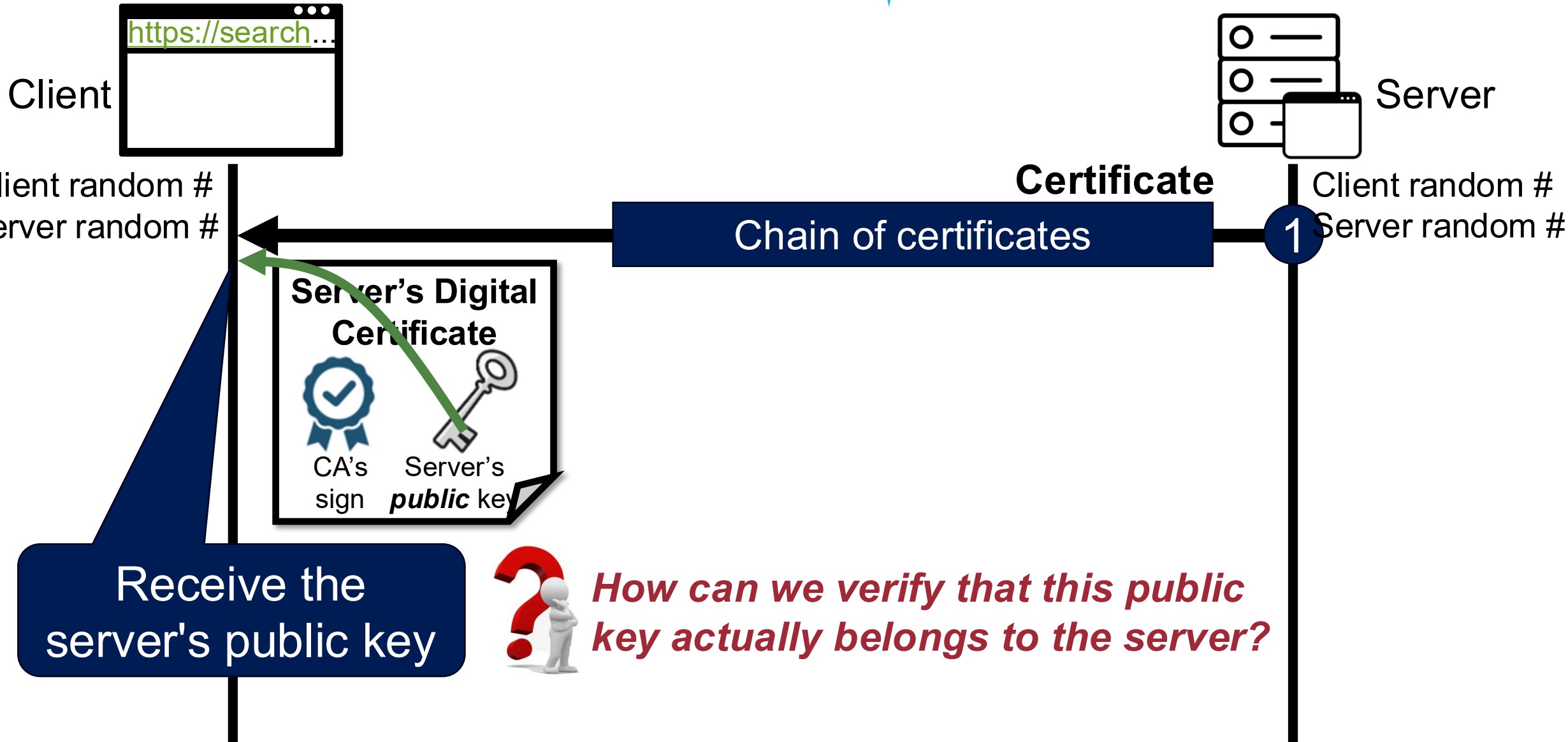
Phase 2: Server Auth. and Key Exchange

51



Phase 2: Server Auth. and Key Exchange

52



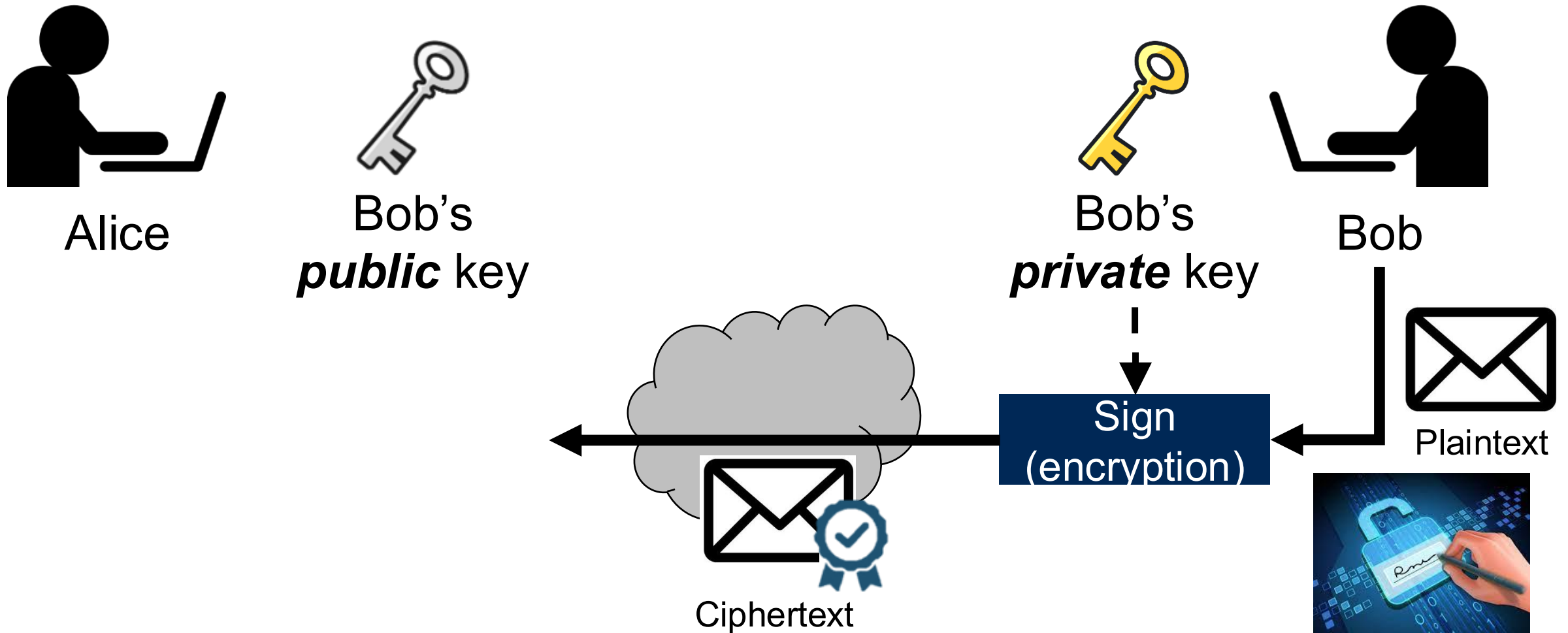
Digital Certificate



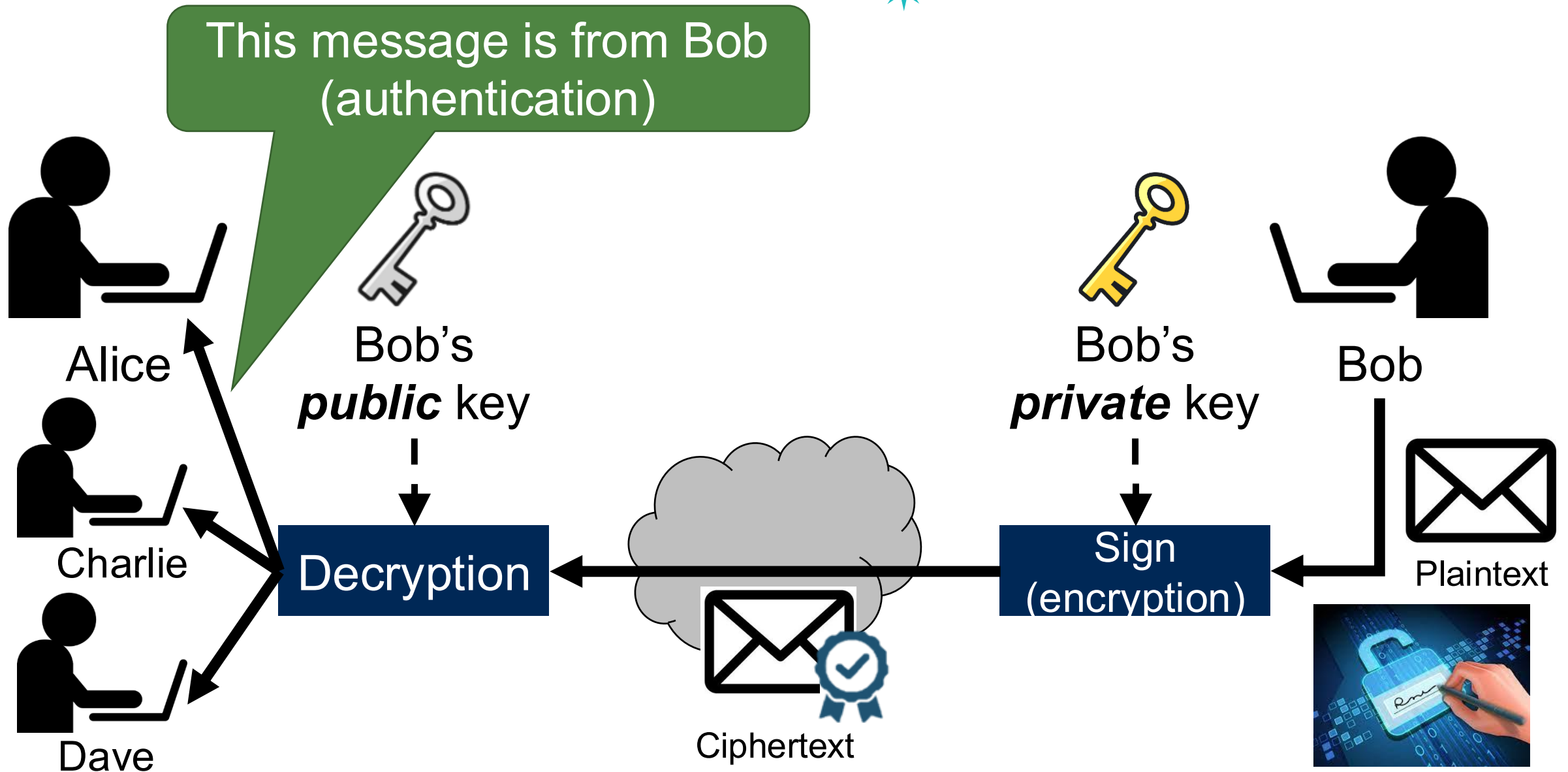
- A document certifying that the public key included inside does **belong to the identity described in the document**
- The Certificate Authority (CA), a trusted third party, certifies the server's public key using a **digital signature**

Digital Signature

54

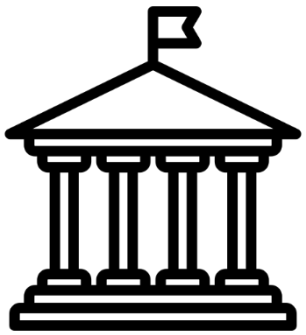


Digital Signature



Digital Certificate

Signing



Certificate
Authority (CA)

Trusted 3rd-party authority
(KISA, yesSign, Verisign ...)

Digital Certificate

- ✓ **Subject:** Server
- ✓ **Expires:** 11/25/2034
- ✓ **Bob's public key:**
ADFECDBBF...



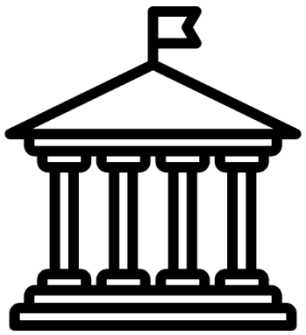
Hash
function

0101000010...

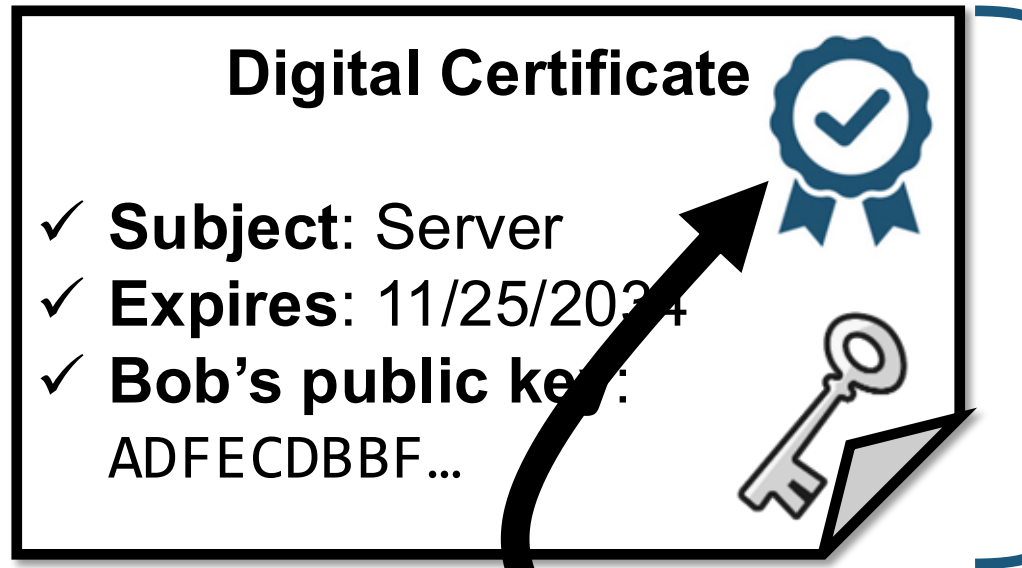
Encrypt with
CA's **private key**
(Sign)

Digital Certificate

Signing



Certificate
Authority (CA)



Append

Encrypt with
CA's **private key**
(Sign)

Hash
function

0101000010...

Hash-based Digital Signature

Verification



Alice



Digital Certificate

- ✓ **Subject:** Server
- ✓ **Expires:** 11/25/2034
- ✓ **Bob's public key:**
ADFECDBBF...



Hash-based Digital Signature

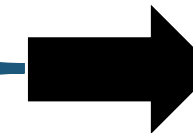
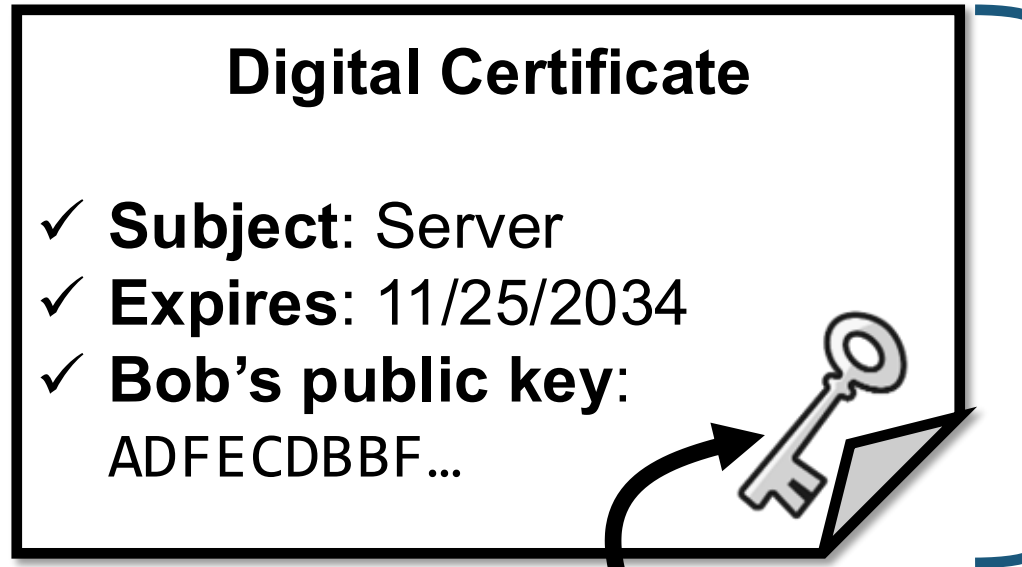
Verification



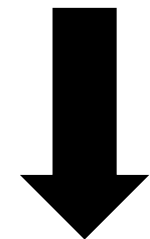
Any problems?



Alice



**Hash
function**

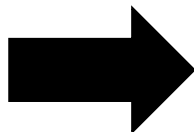


?

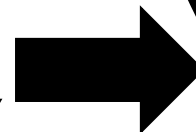
0101000010...



CA's sign



Decrypt with
CA's *public key*



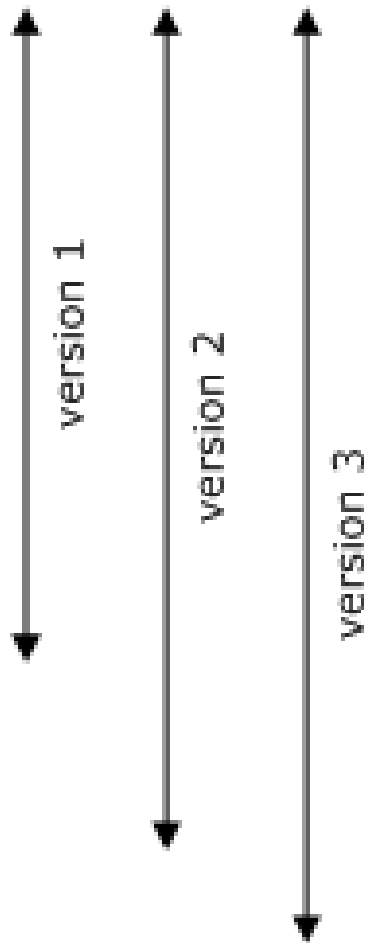
0101000010...



Authentication: Confirm
Server's public key

X.509 Certificate (Demo)

Version
Serial Number
Signature Algorithm Identifier
Issuer Name
Validity Period
Subject Name
Public Key Information
Issuer Unique ID
Subject Unique ID
Extensions



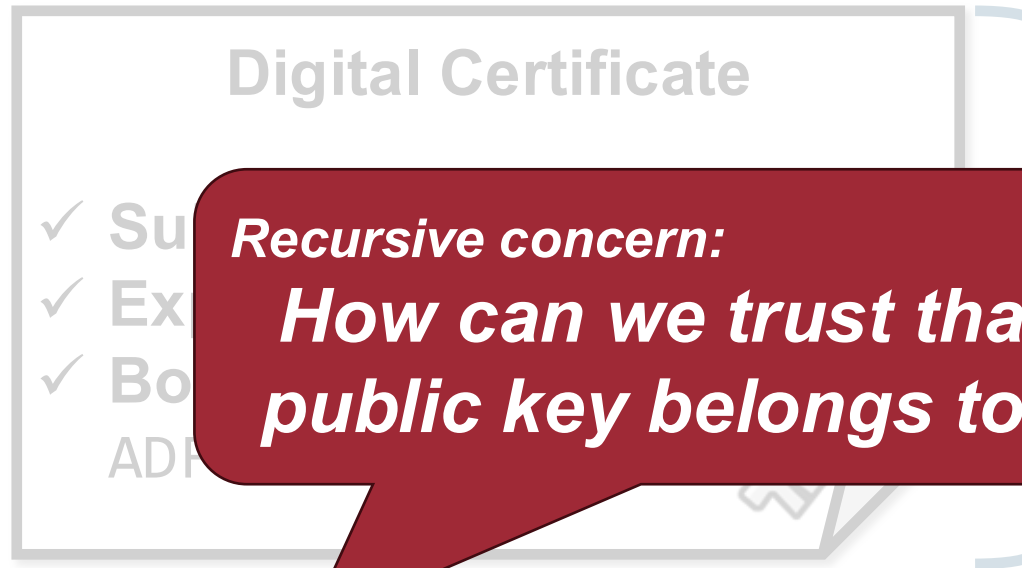
구분	
일반	
필드	값
버전	3
일련번호	09575a3e
서명 알고리즘	SHA1 + RSA
발급자	cn=yessignCA,ou=Accredited...
다음부터 유효함	2009-05-19 00:00:00
다음까지 유효함	2010-05-25 23:59:59
주체	cn=...()0020045200505177...
공개키 알고리즘	RSA
공개키	3081890281810080270c78b6e91...
서명	07c8512b0c4615f4b8576ddd8c...
CA 키 고유번호	4afb5d332d85b1d18c946bffe04...
이진서 전채	1 2 410 200005 1 1 4

One Concern:

Verification



Alice



Recursive concern:

How can we trust that the public key belongs to CA?

Hash function



?



CA's sign

Decrypt with
CA's **public key**

0101000010.

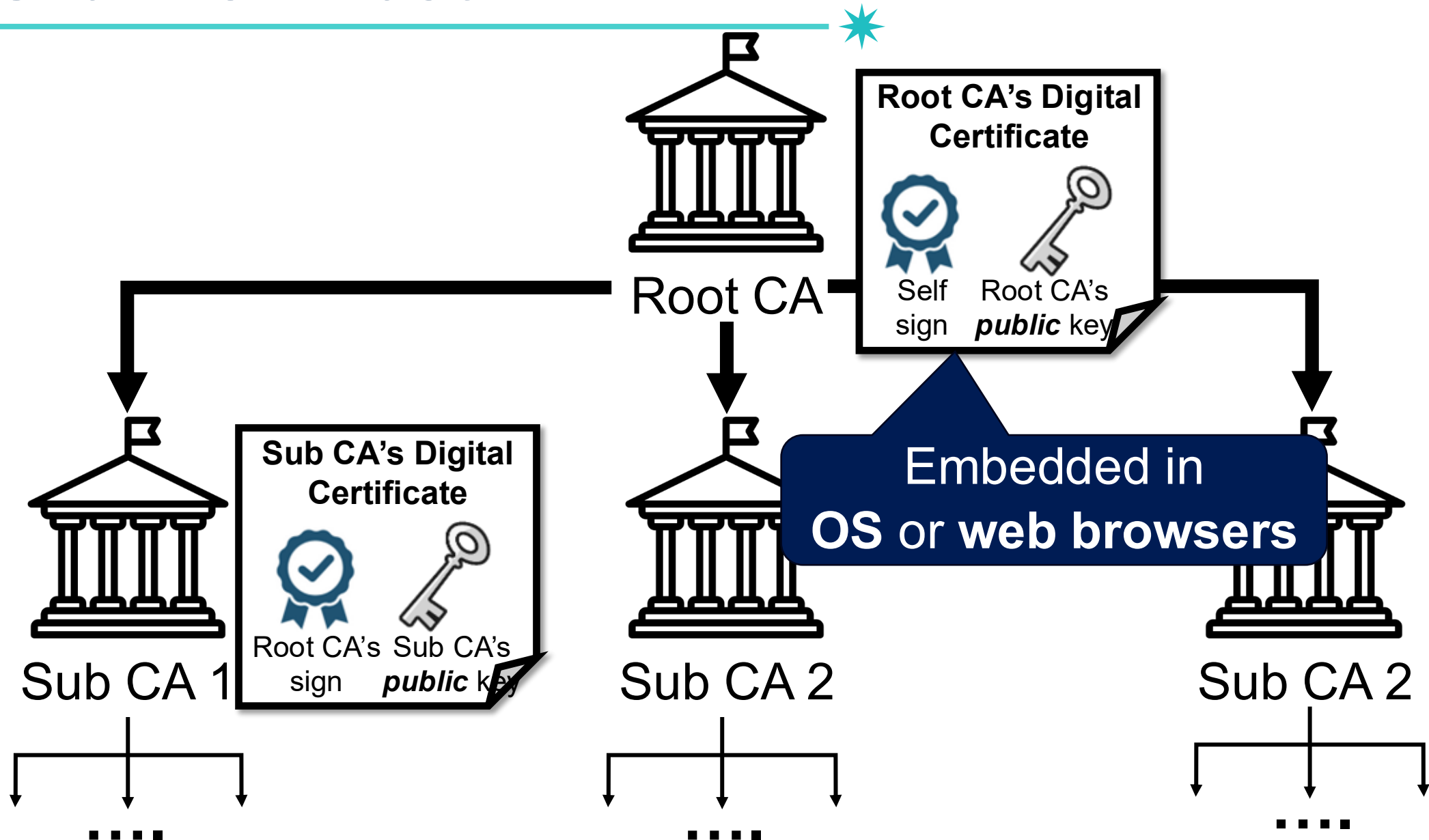
=

0101000010.

Authentication: Confirm
Server's public key

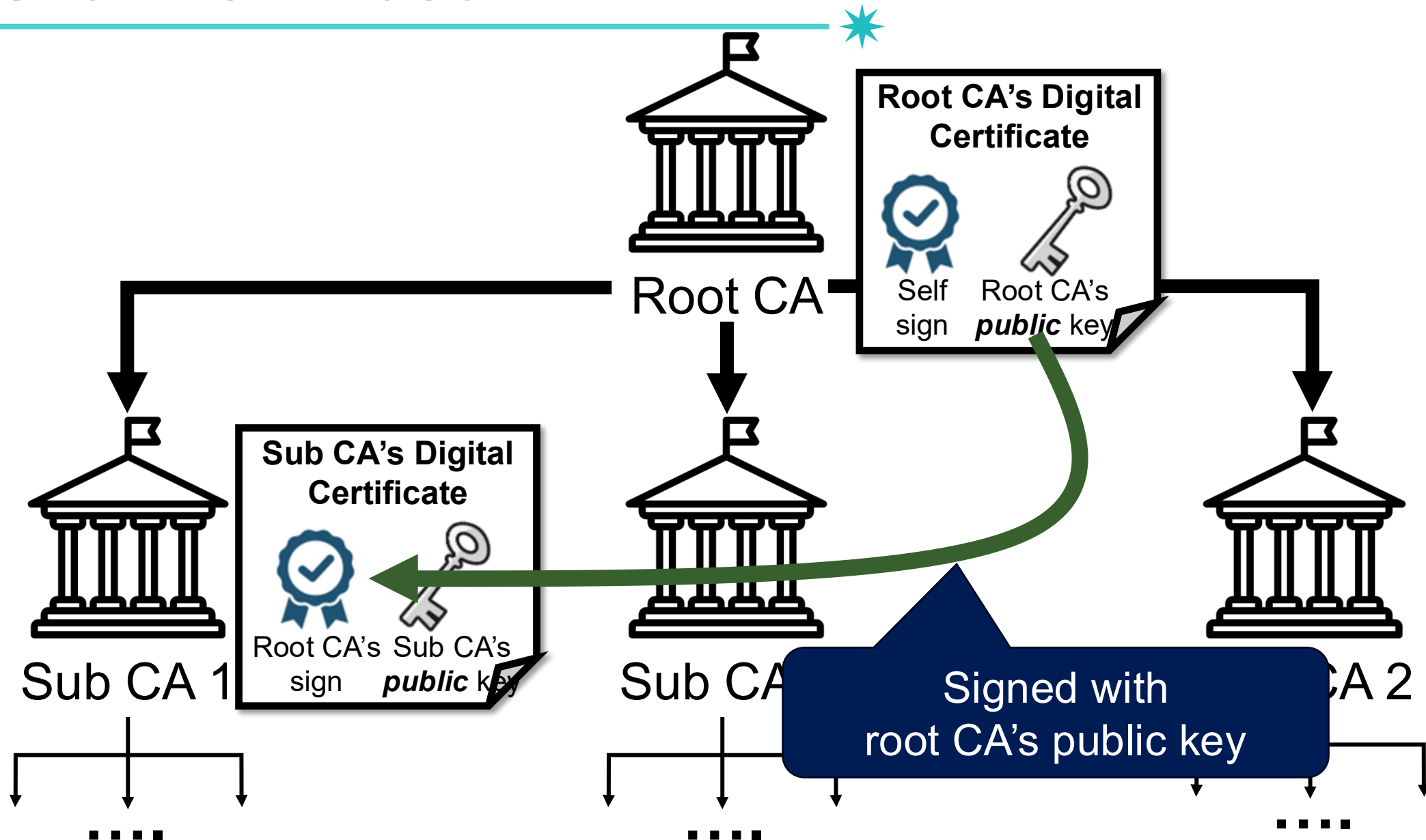
Chain of Trust

62



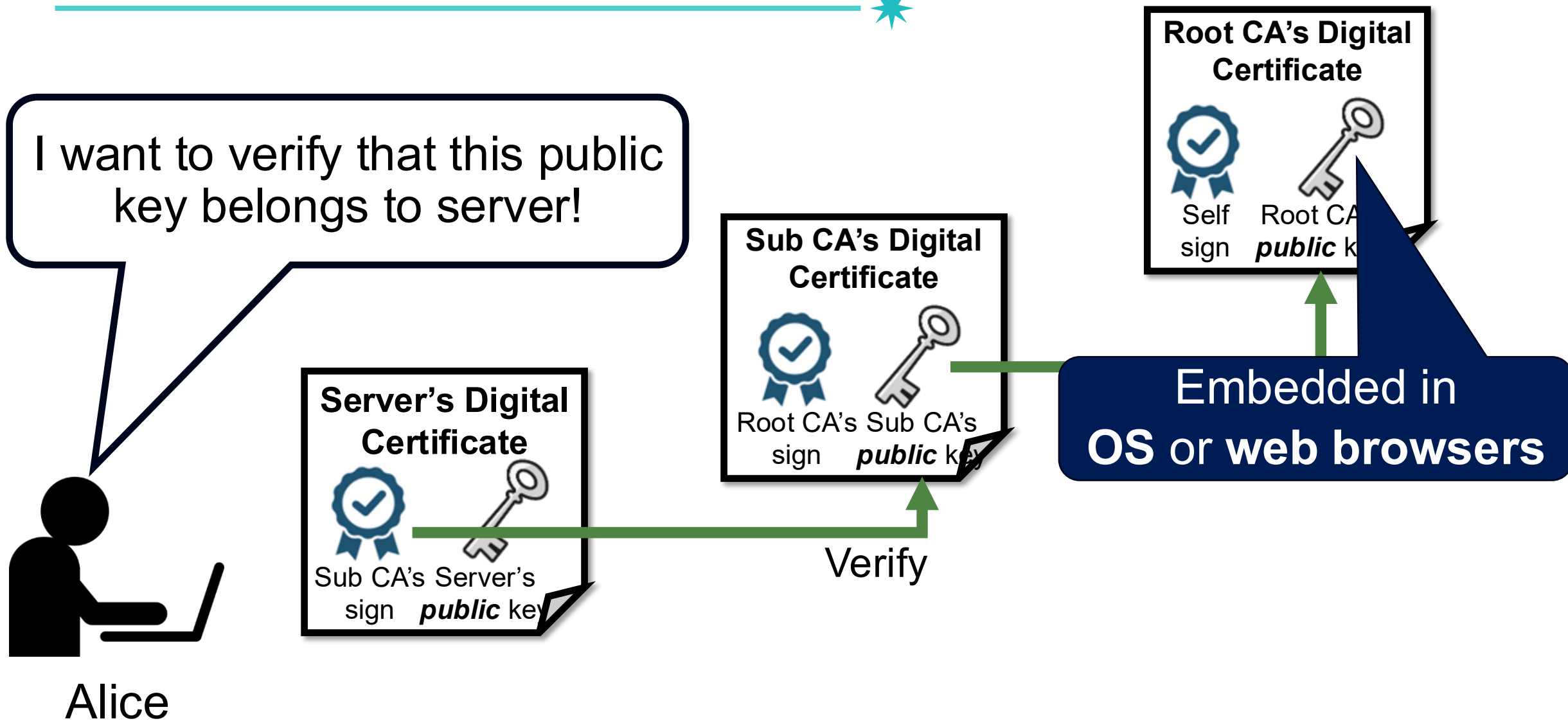
Chain of Trust

63



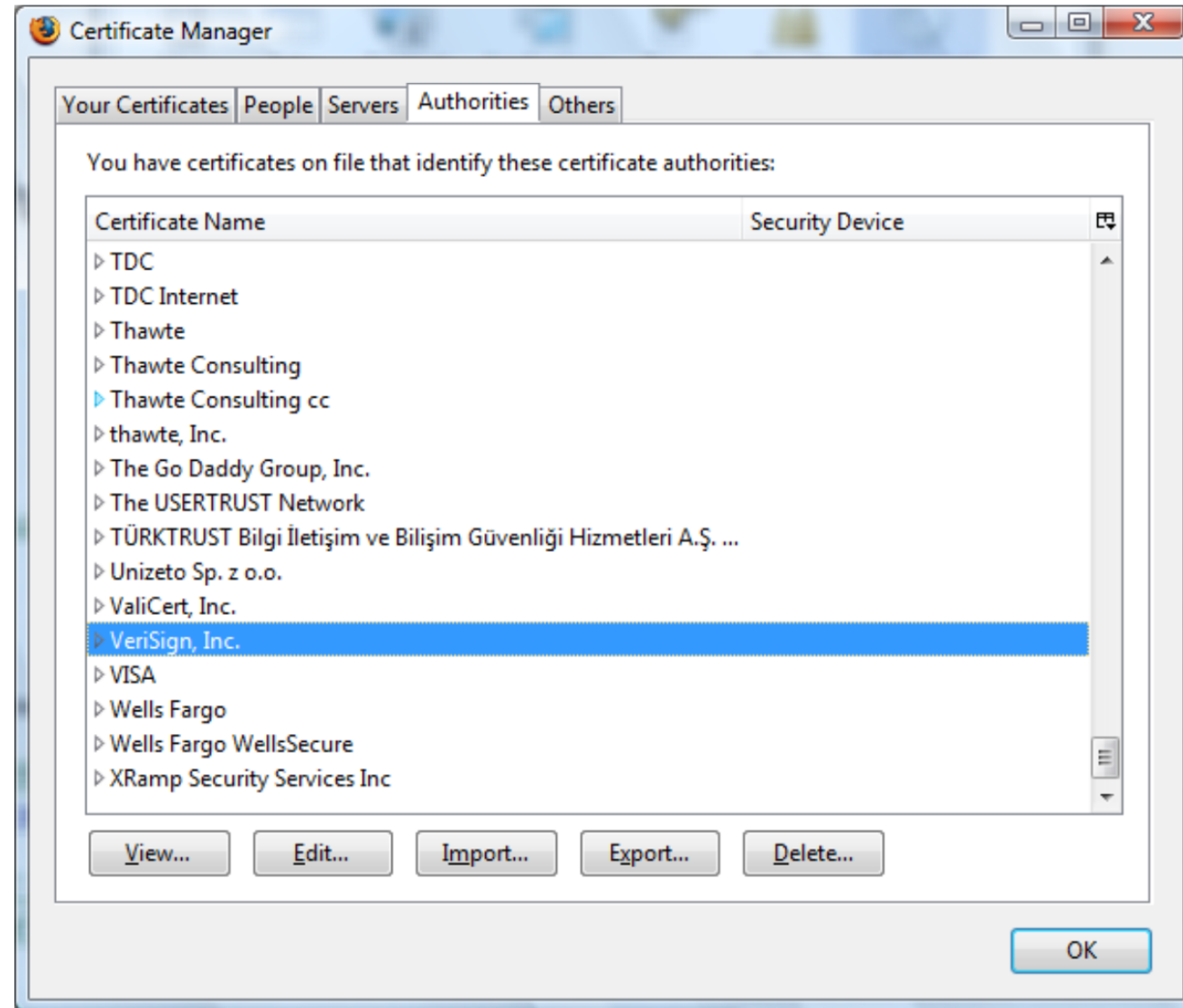
Recap: Chain of Trust

64



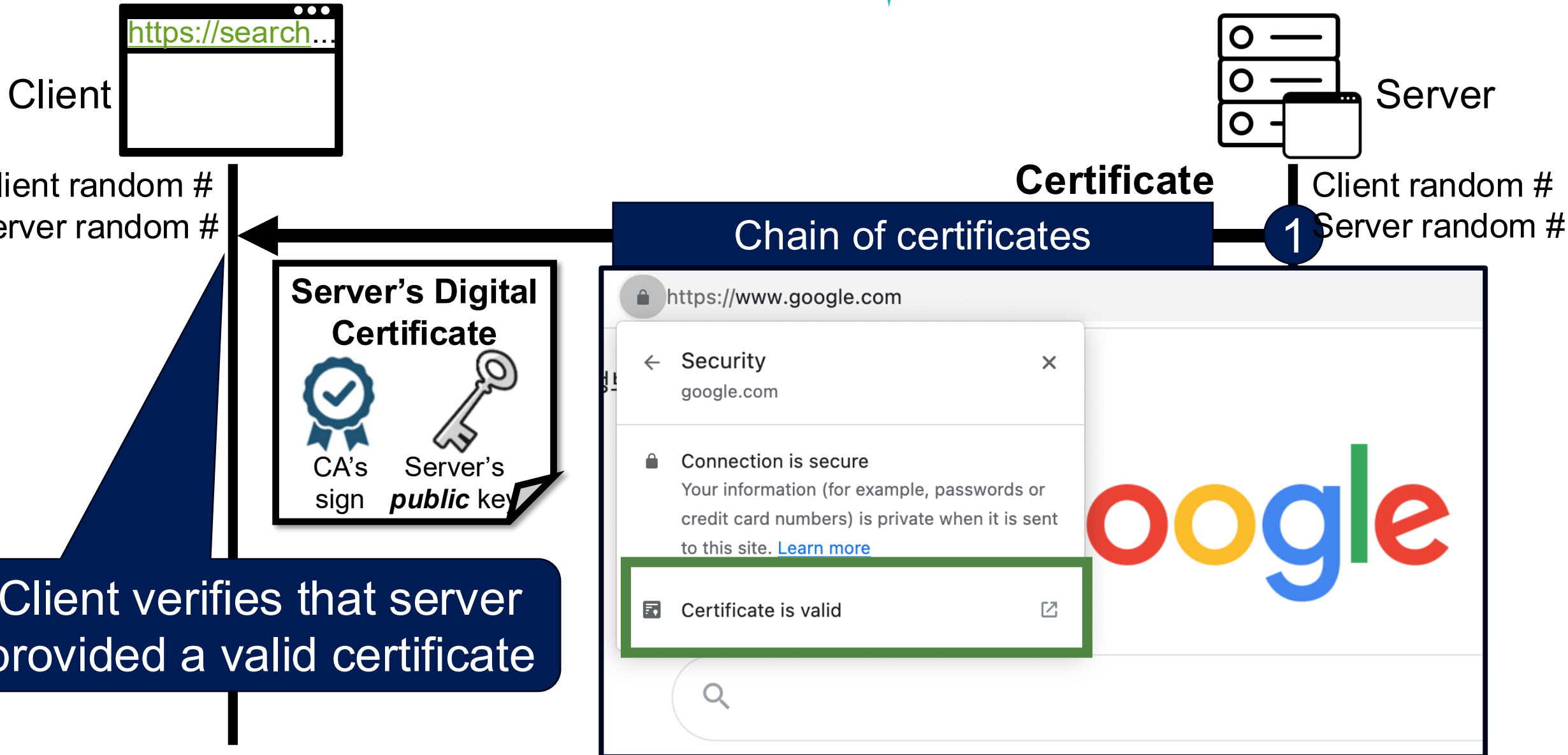
Browsers are Pre-configured with 100+ Trusted CAs

65



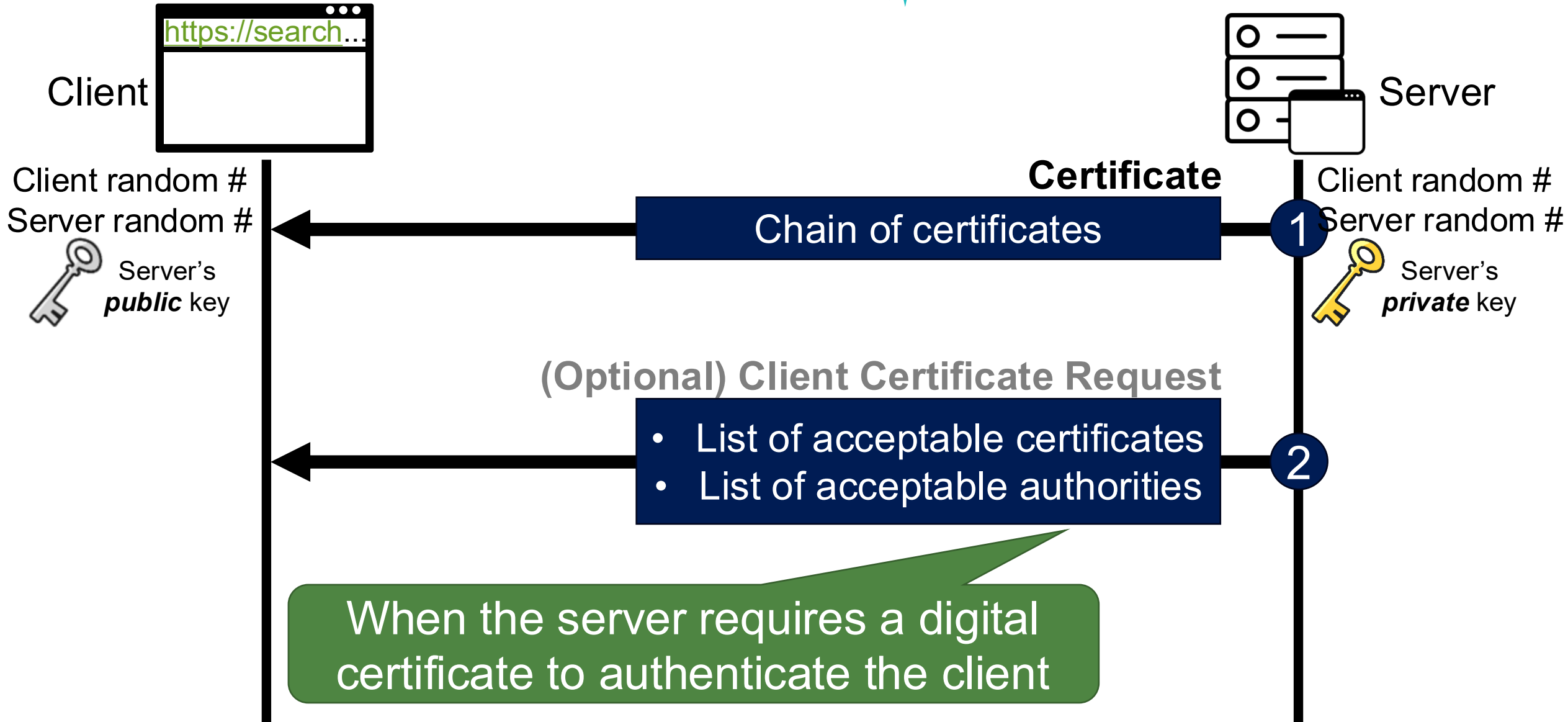
Phase 2: Server Auth. and Key Exchange

66



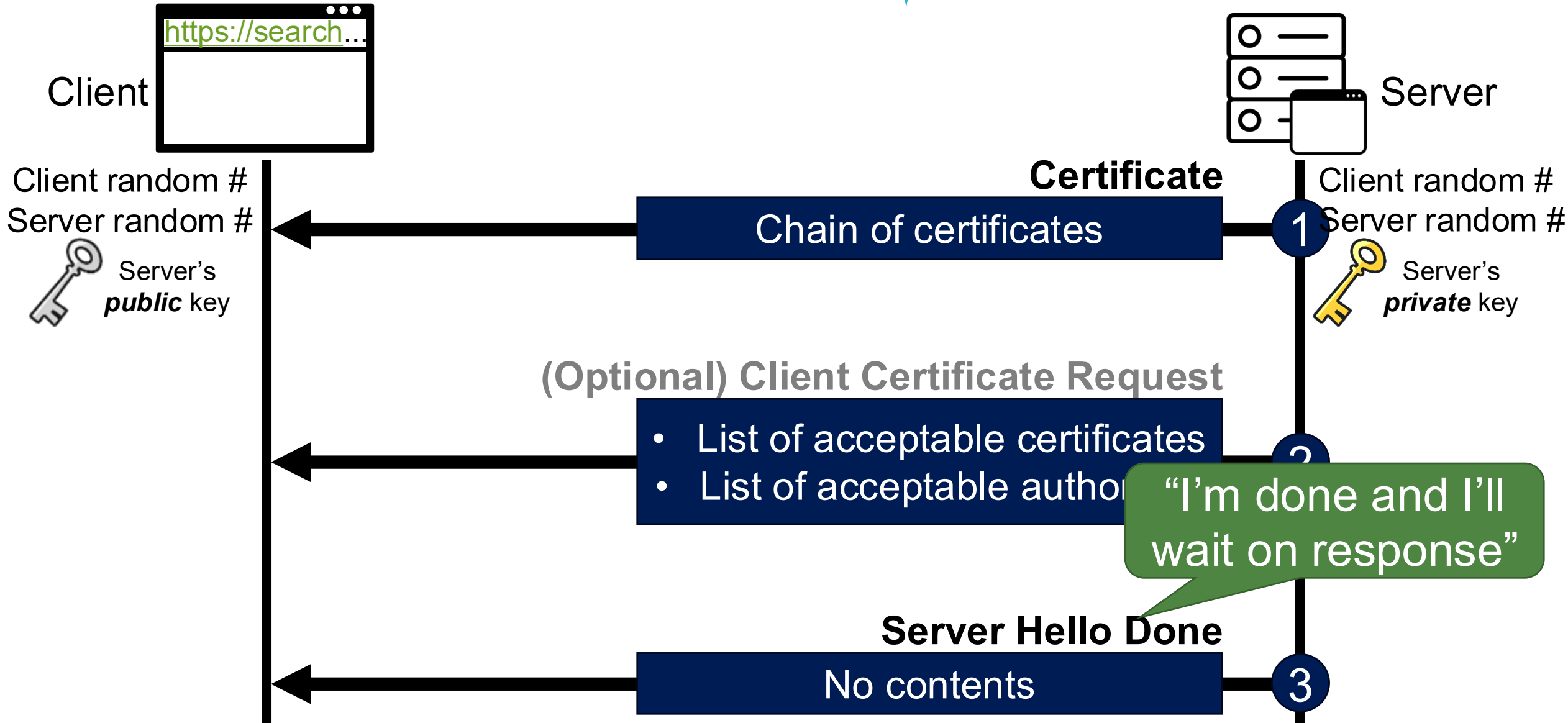
Phase 2: Server Auth. and Key Exchange

67

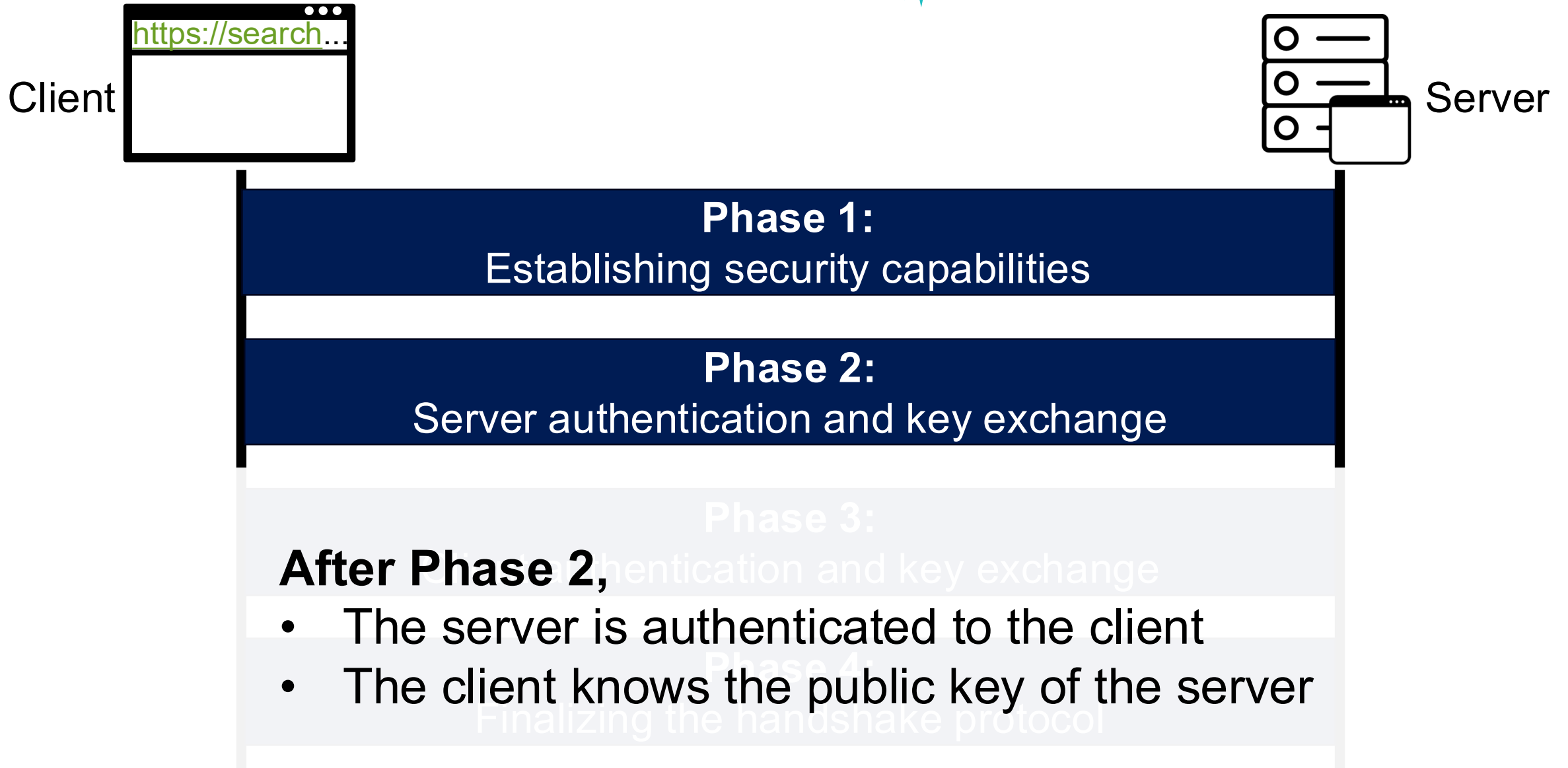


Phase 2: Server Auth. and Key Exchange

68

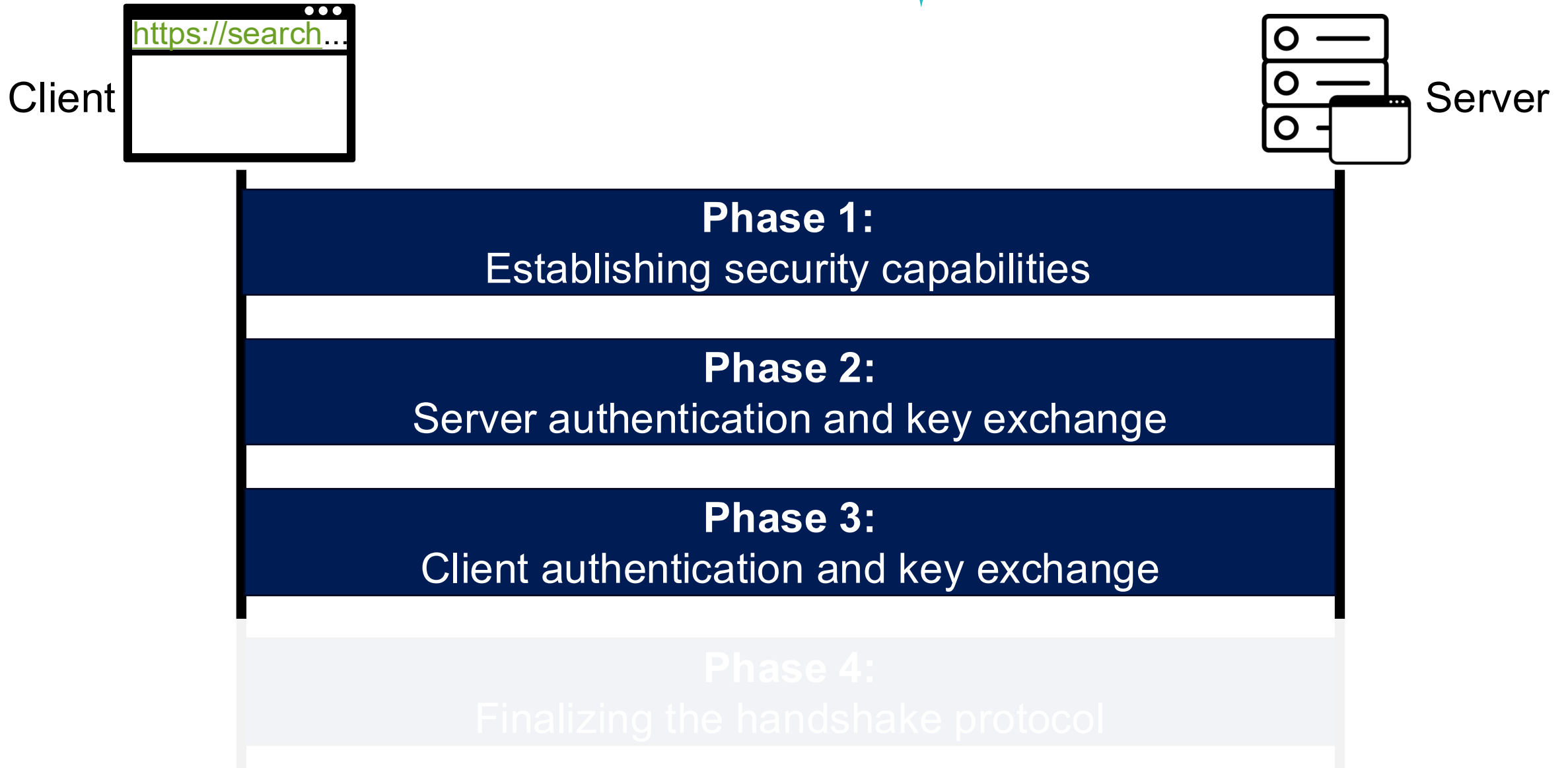


Phase 1: Establishing Security Capabilities ⁶⁹



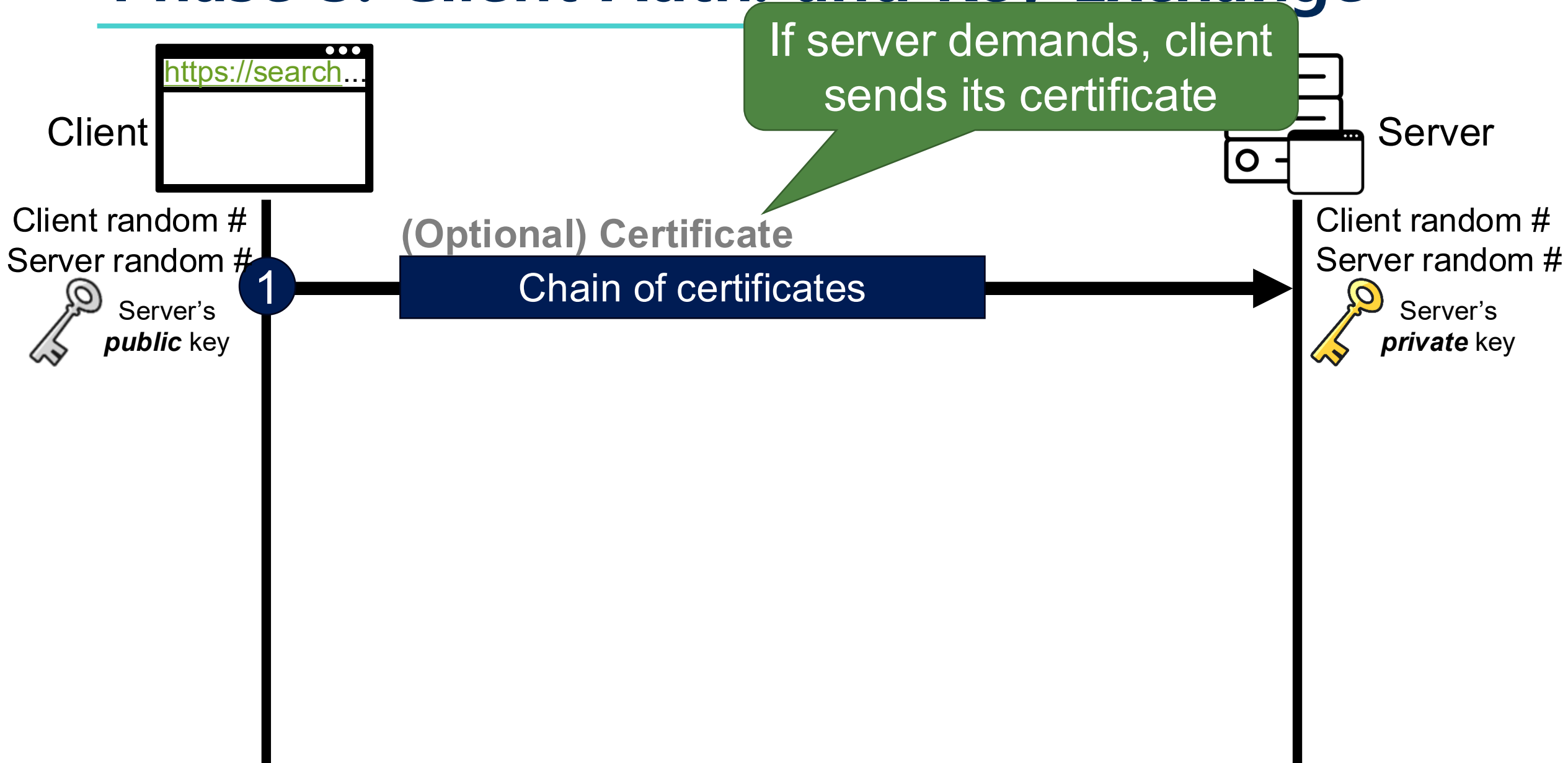
Phase 3: Client Auth. and Key Exchange

70



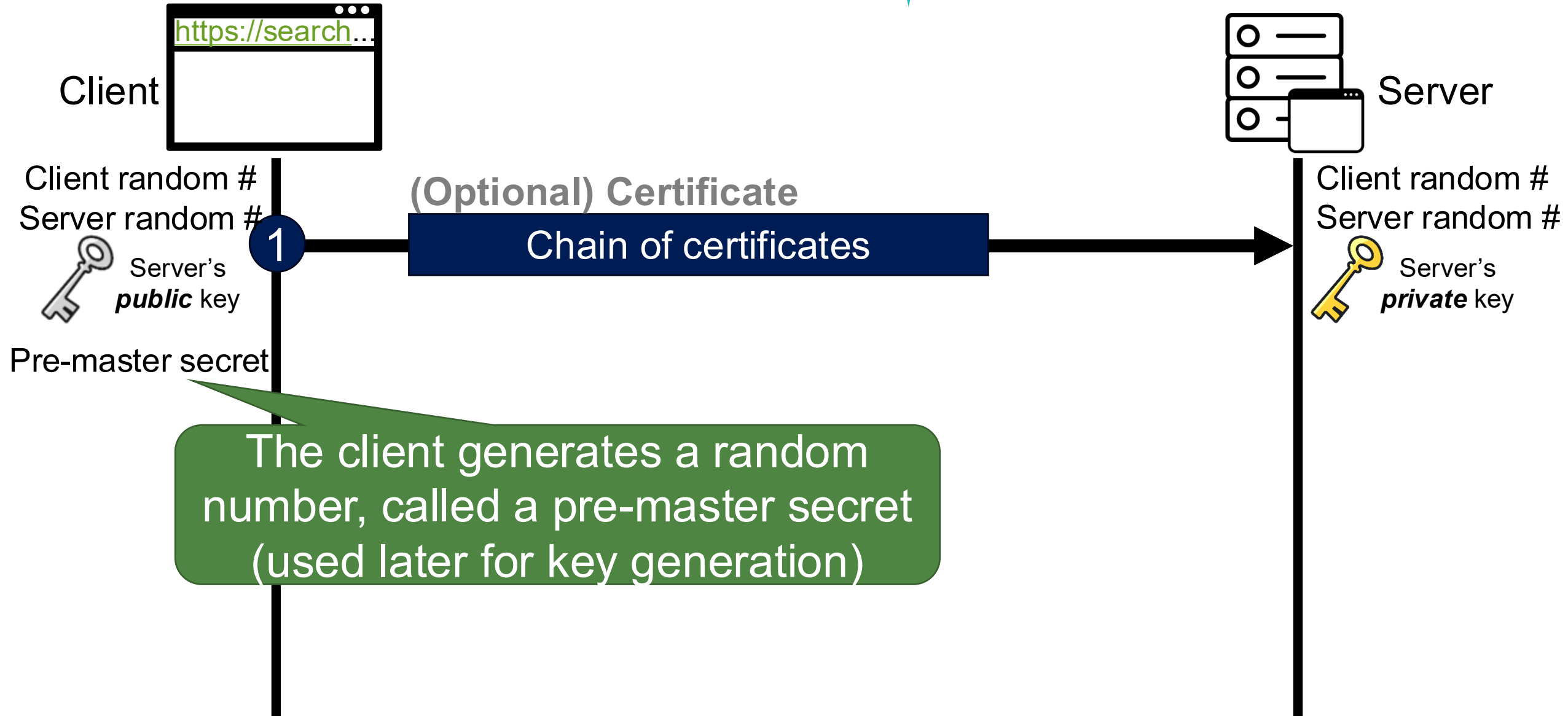
Phase 3: Client Auth. and Key Exchange

71



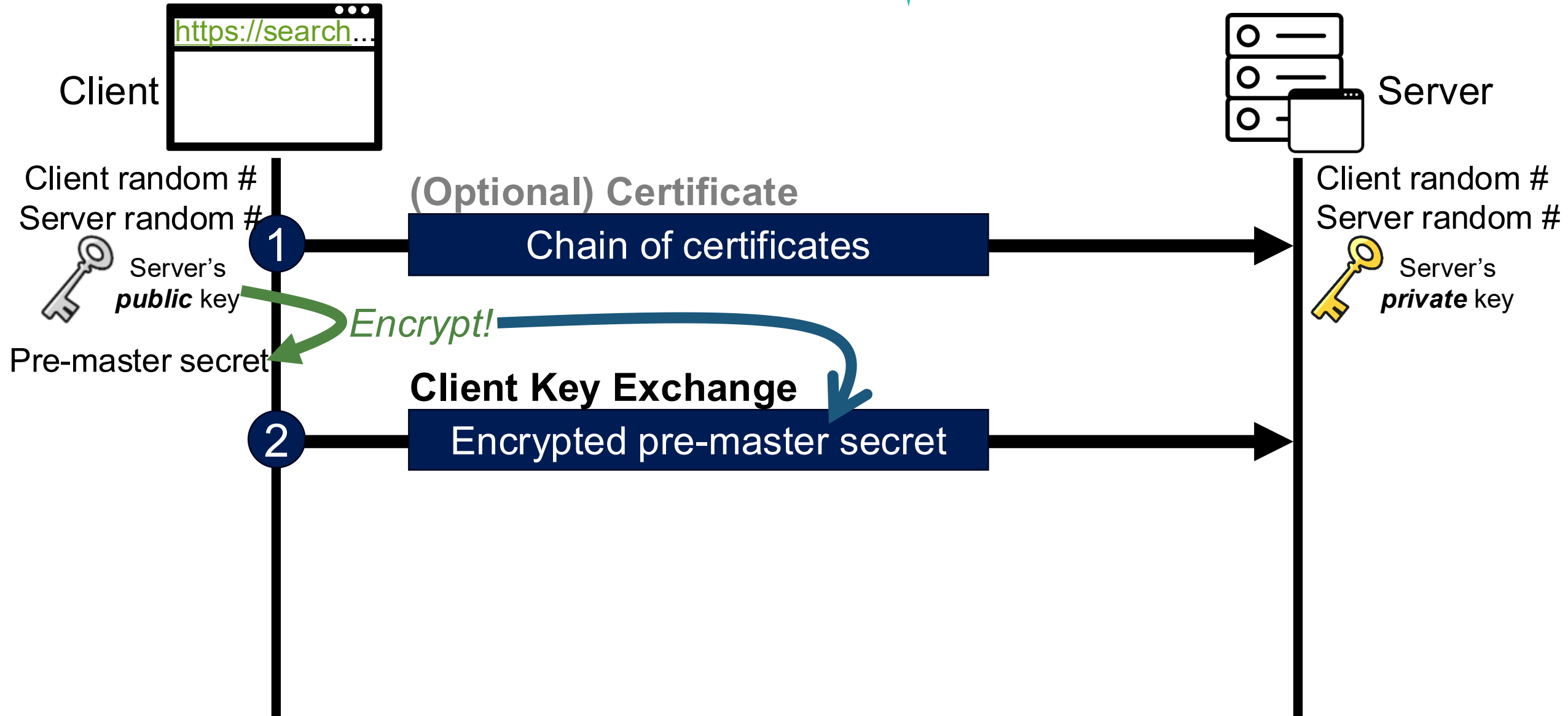
Phase 3: Client Auth. and Key Exchange

72



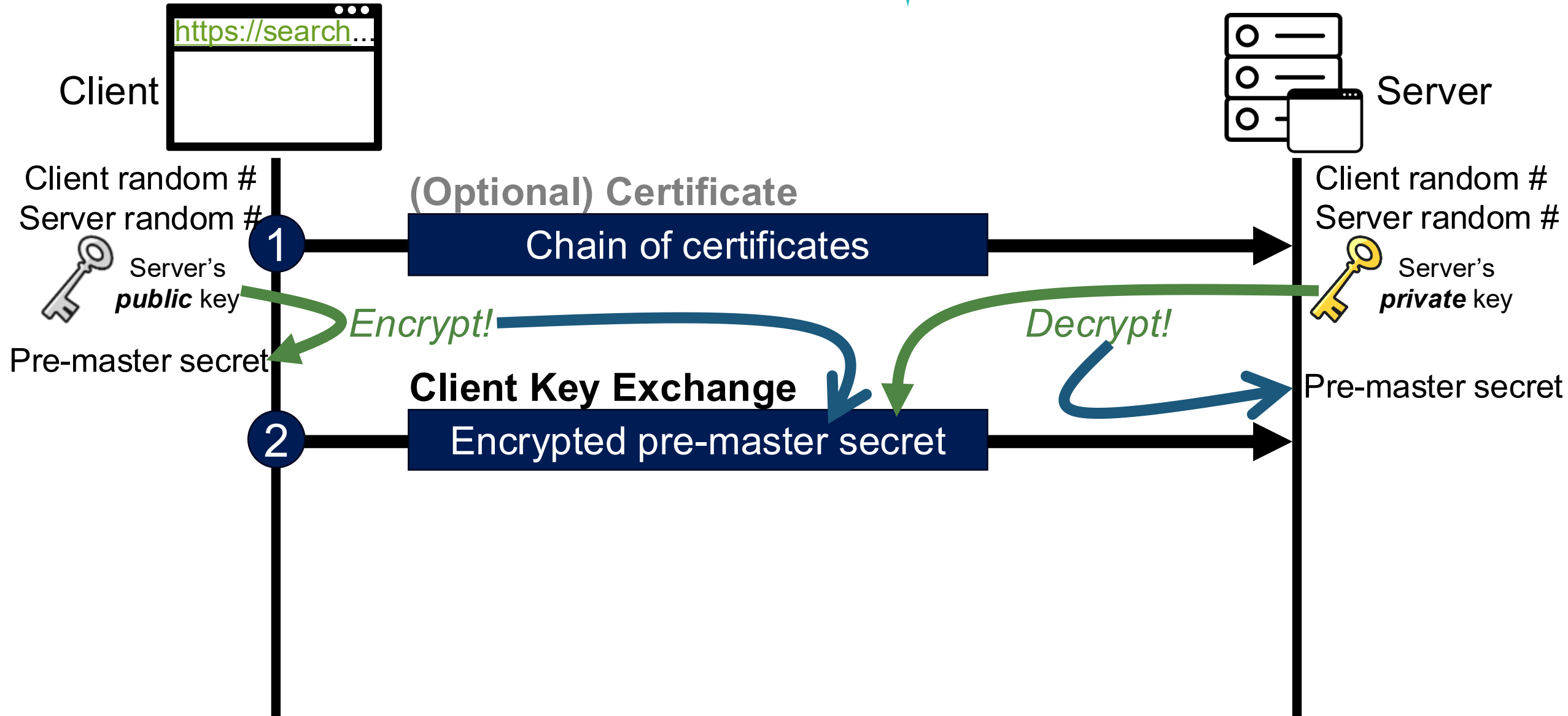
Phase 3: Client Auth. and Key Exchange

73



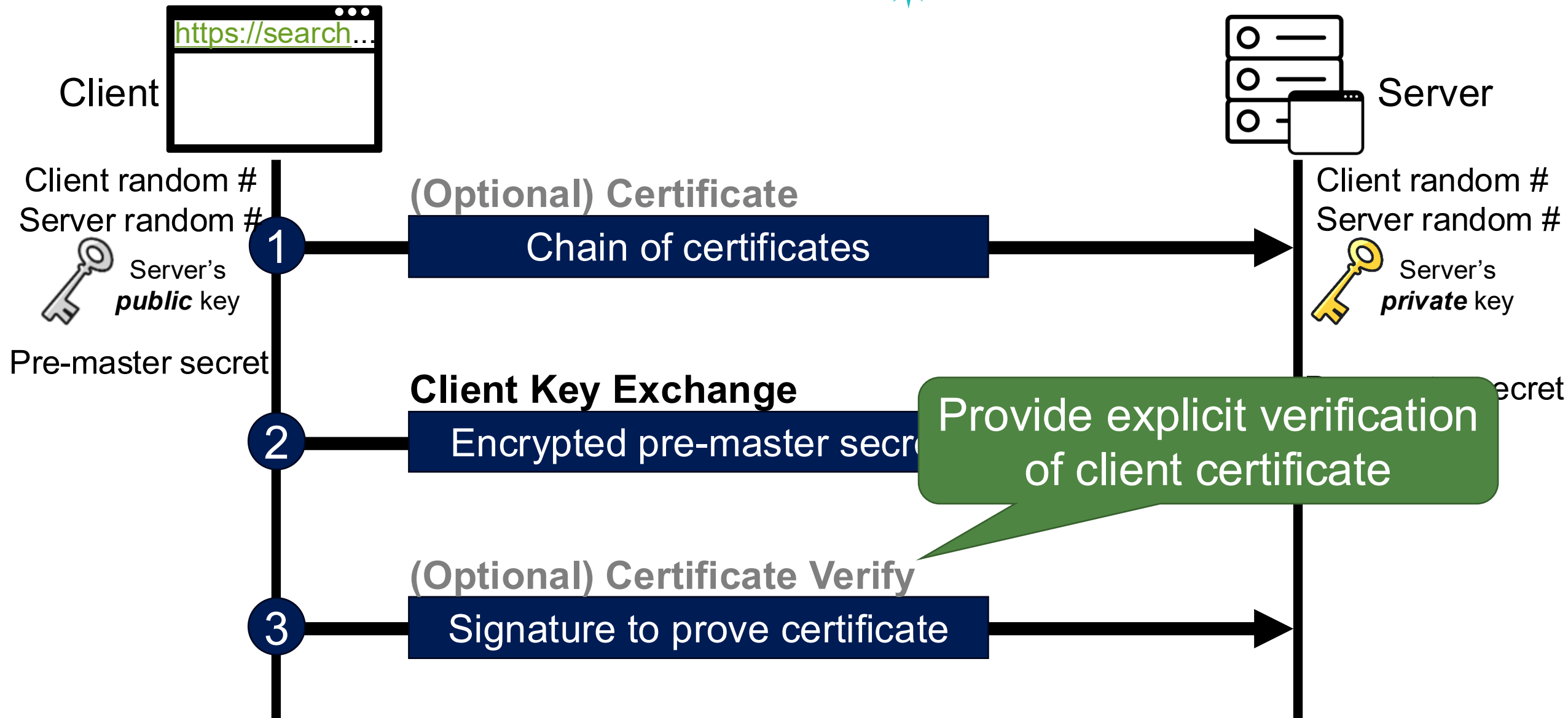
Phase 3: Client Auth. and Key Exchange

74



Phase 3: Client Auth. and Key Exchange

75



Phase 3: Client Auth. and Key Exchange

76



After Phase 3,

- (Optional) The client is authenticated for the server
- Both the client and the server know the pre-master secret

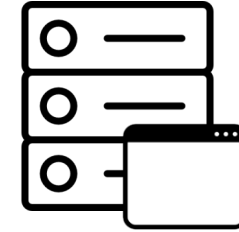
Phase 3: Client Auth. and Key Exchange

77

Client



Server



Client random #
Server random #



Server's
public key

Pre-master secret

***Before move on Phase 4,
let's make symmetric key***

*Why do we need a symmetric key
even though we already have asymmetric key?*

Client random #
Server random #



Server's
private key

Pre-master secret

Pros and Cons: Asymmetric-key Cryptography



- Pros

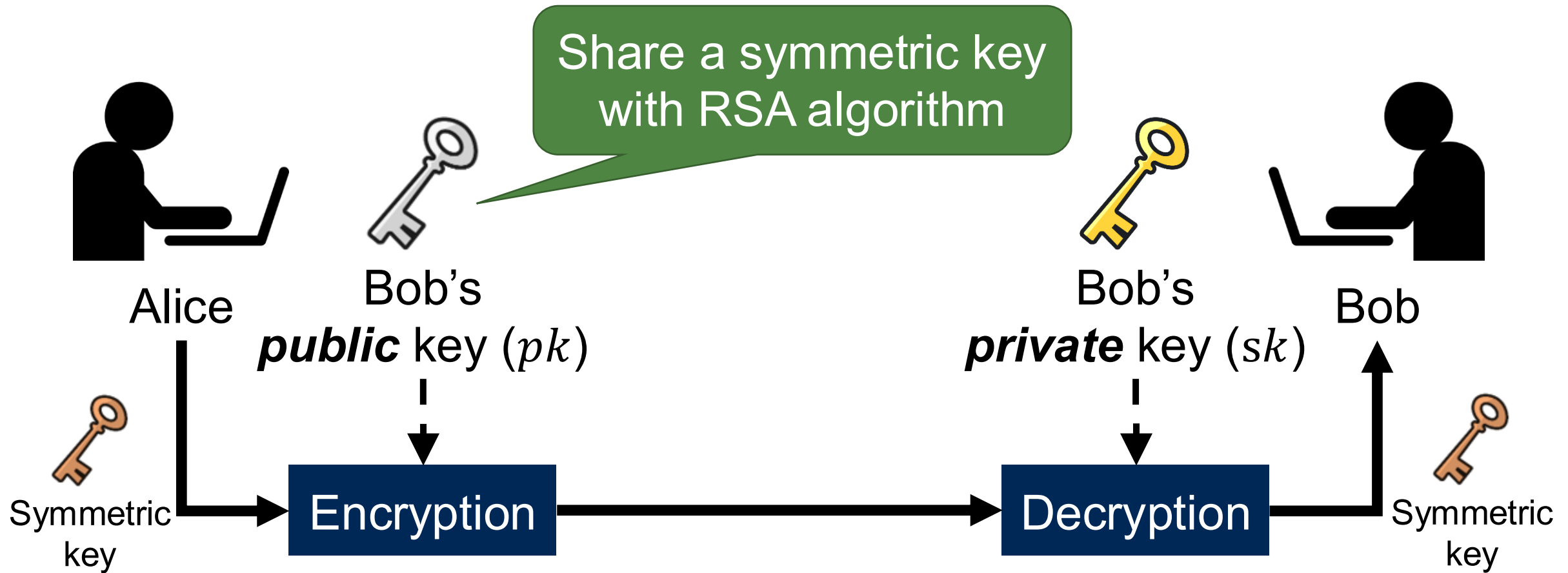
- No need to share a secret
- Enable multiple senders to communicate privately with a single receiver
- More applications: Digital sign

- Cons

- **Slower in general**: due to the larger key
 - Roughly 2-3 orders of magnitude slower

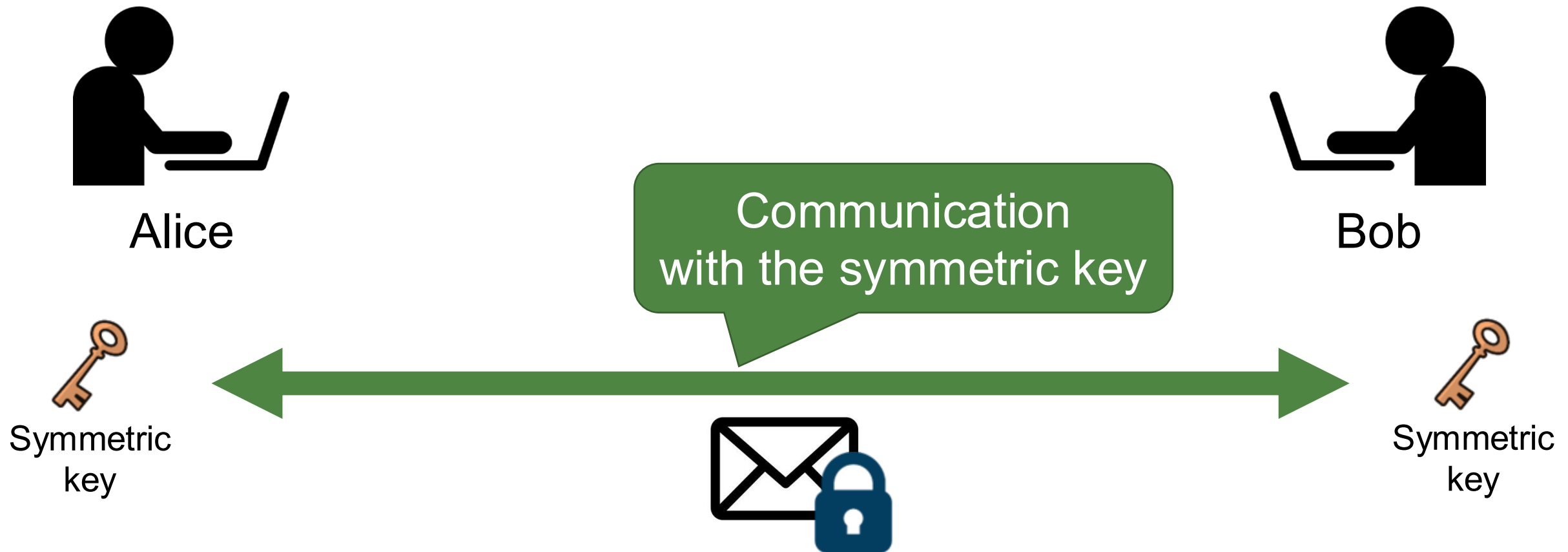
Solution: Combination of Two Schemes

79



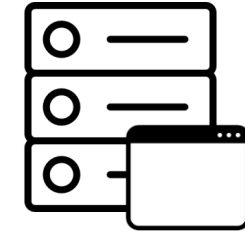
Solution: Combination of Two Schemes

80



Phase 3: Client Auth. and Key Exchange

81

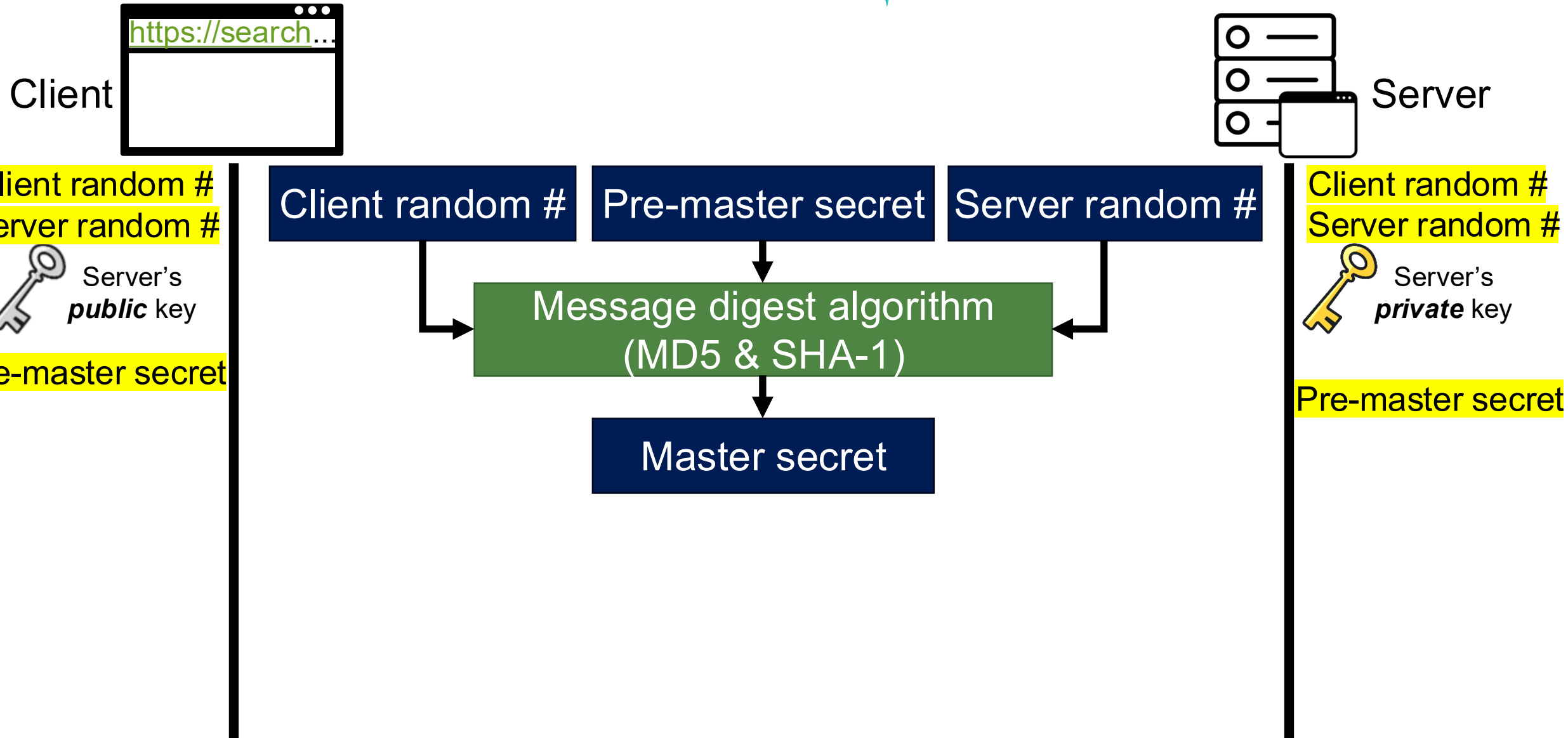


Client random #
Server random #
 Server's **public** key
Pre-master secret

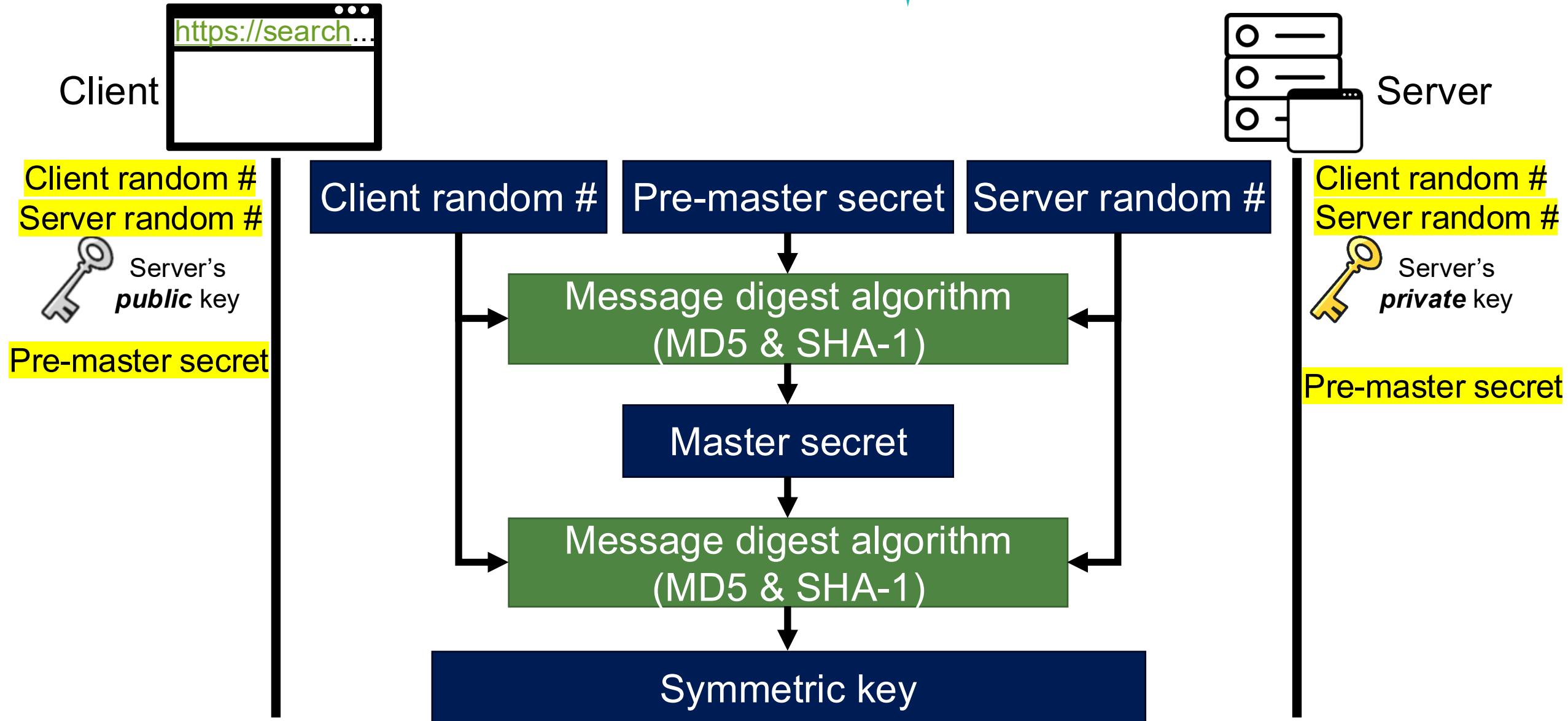
***Before move on Phase 4,
let's make symmetric key***

Client random #
Server random #
 Server's **private** key
Pre-master secret

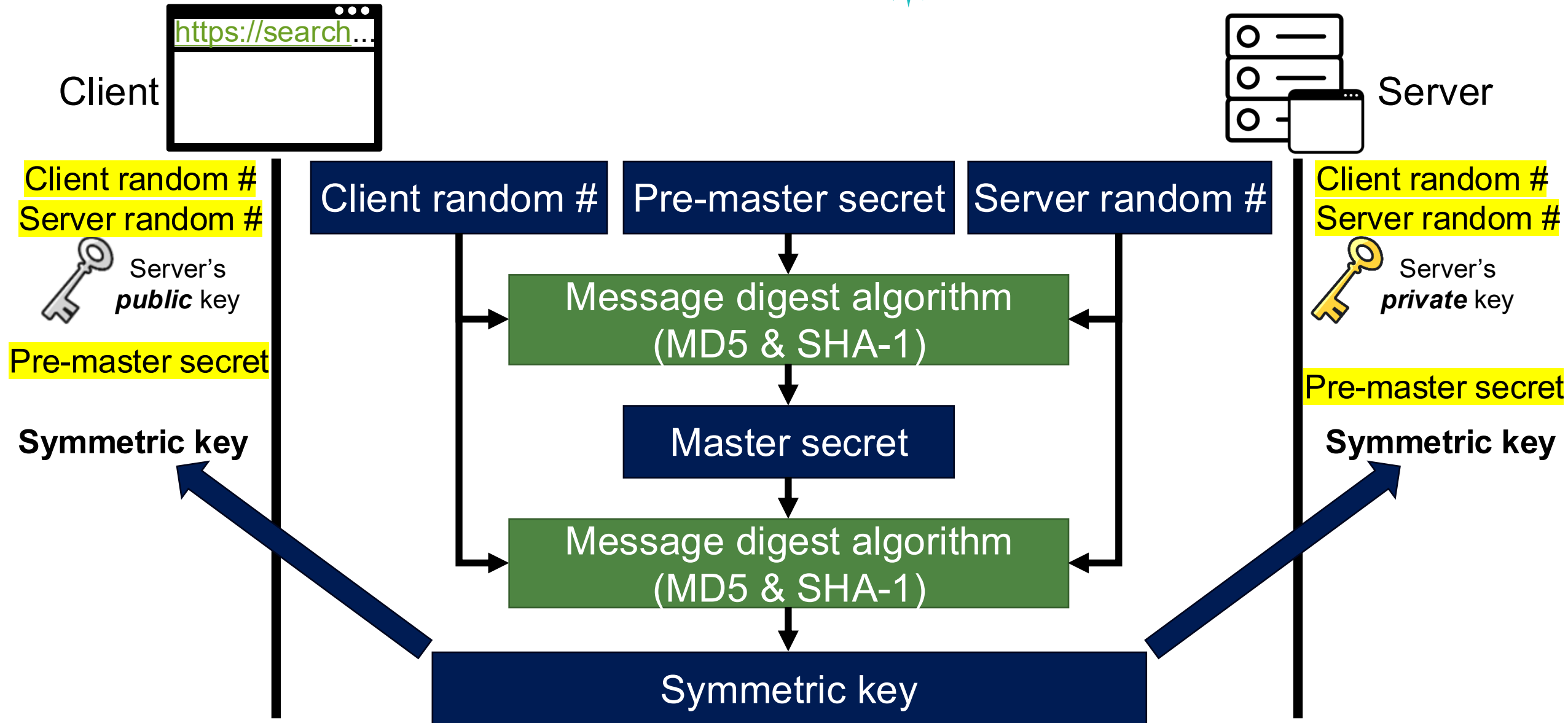
Calculation of Master Secret



Calculation of Symmetric Key

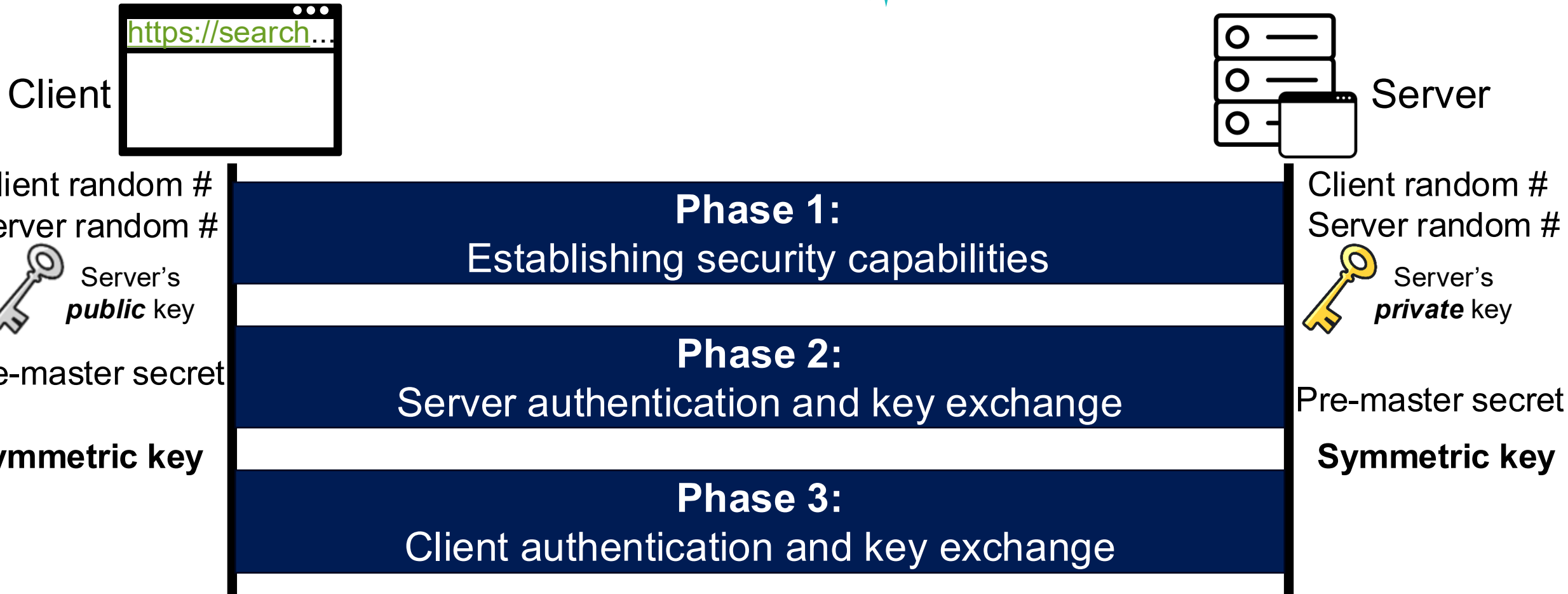


Calculation of Symmetric Key



Phase 3: Client Auth. and Key Exchange

85



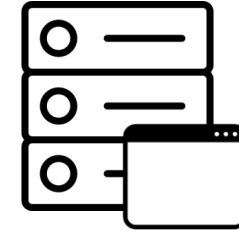
After Phase 3,

- (Optional) The client is authenticated for the server
- Both the client and the server know the pre-master secret

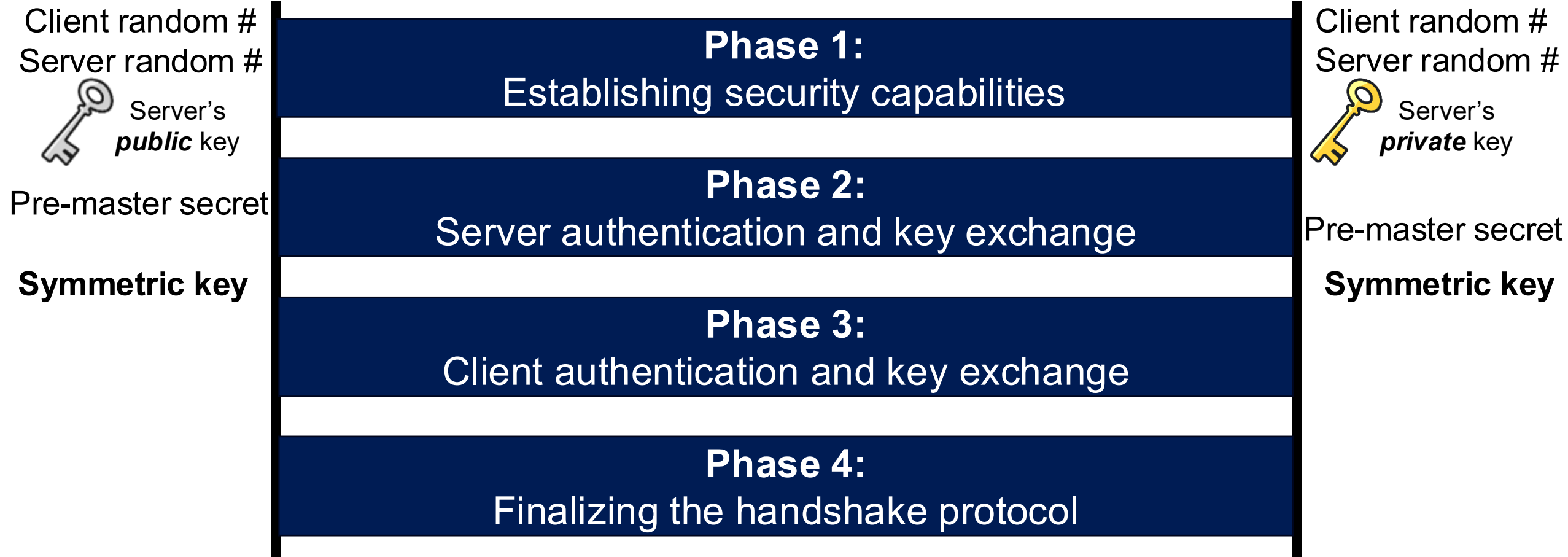
Phase 4: Finalizing the Handshake Protocol

86

Client

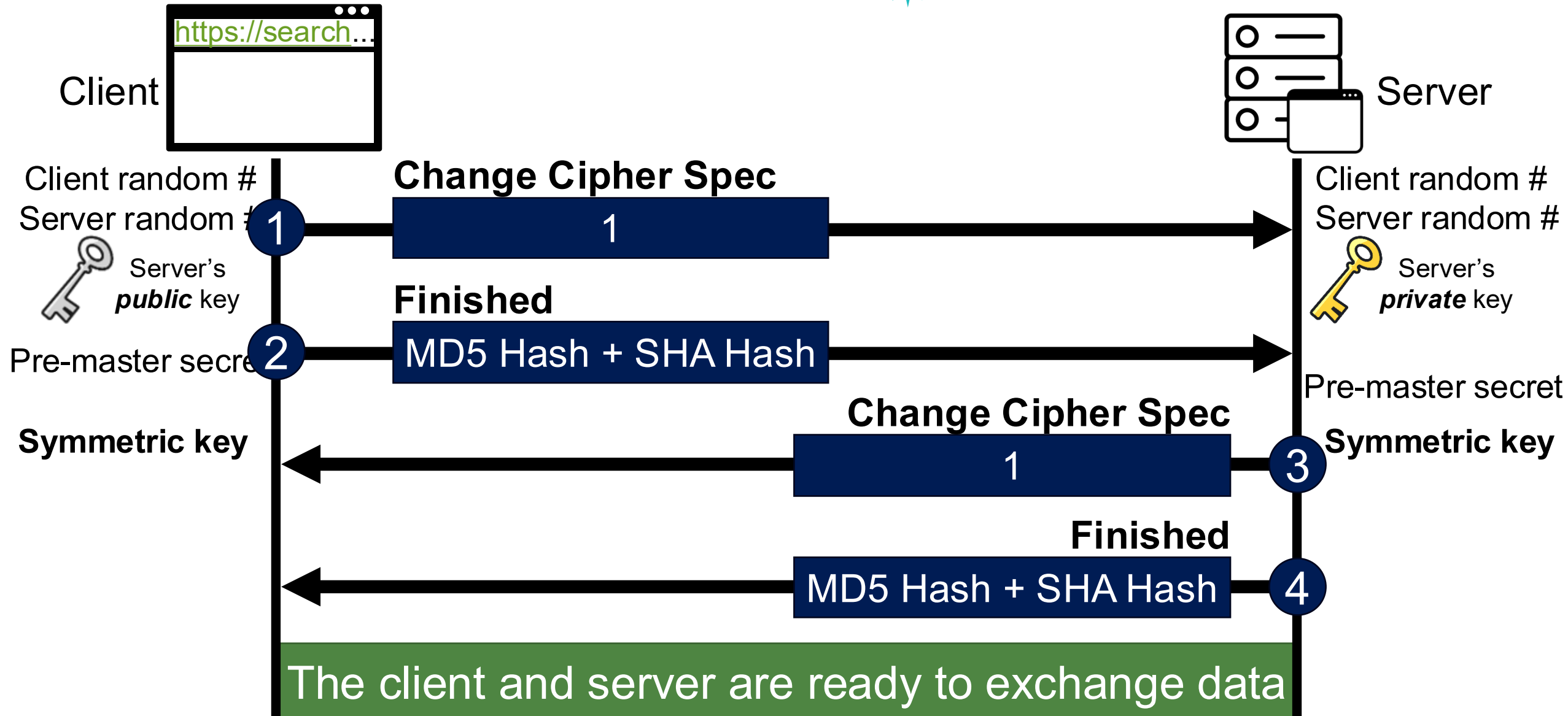


Server

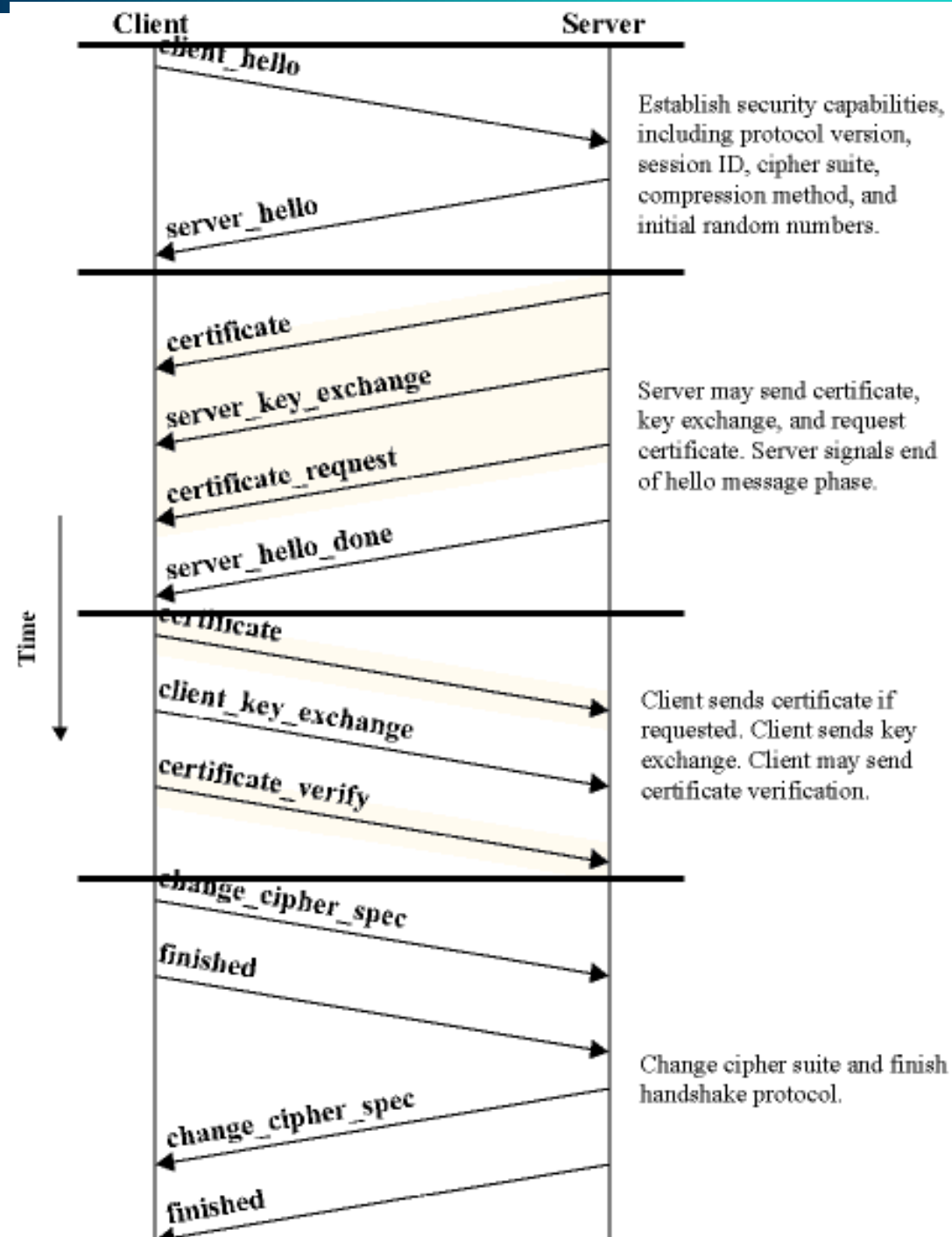


Phase 4: Finalizing the Handshake Protocol

87



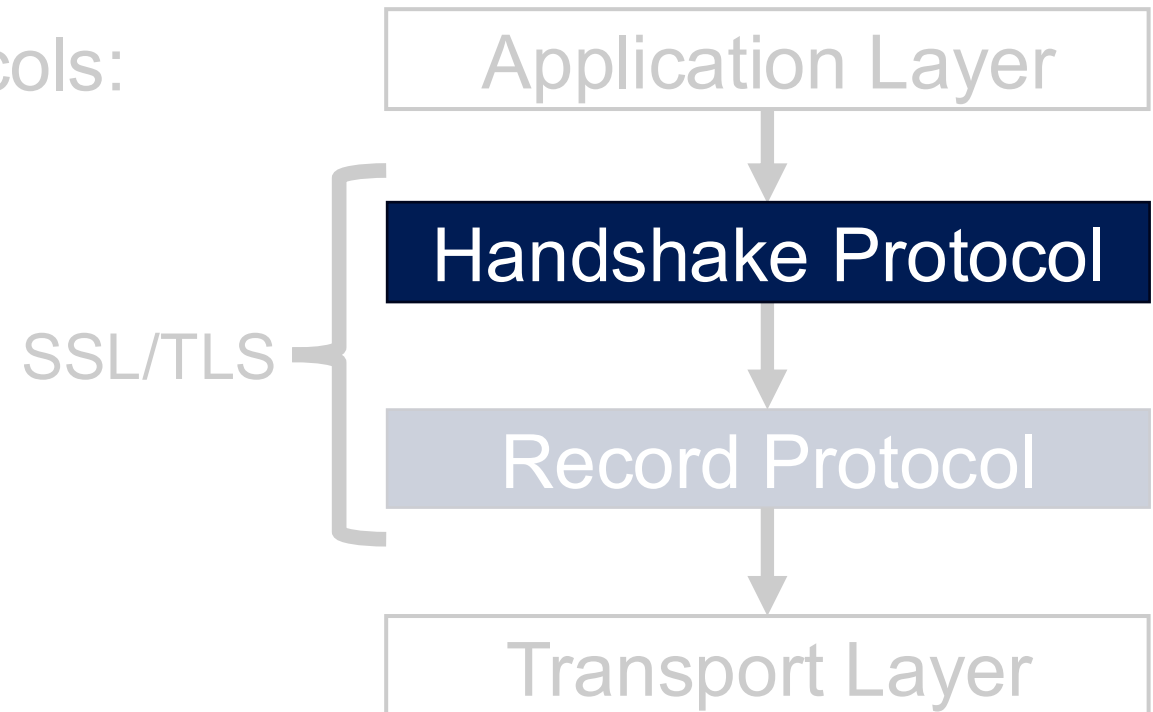
Handshake Protocol Summary



SSL/TLS Basics



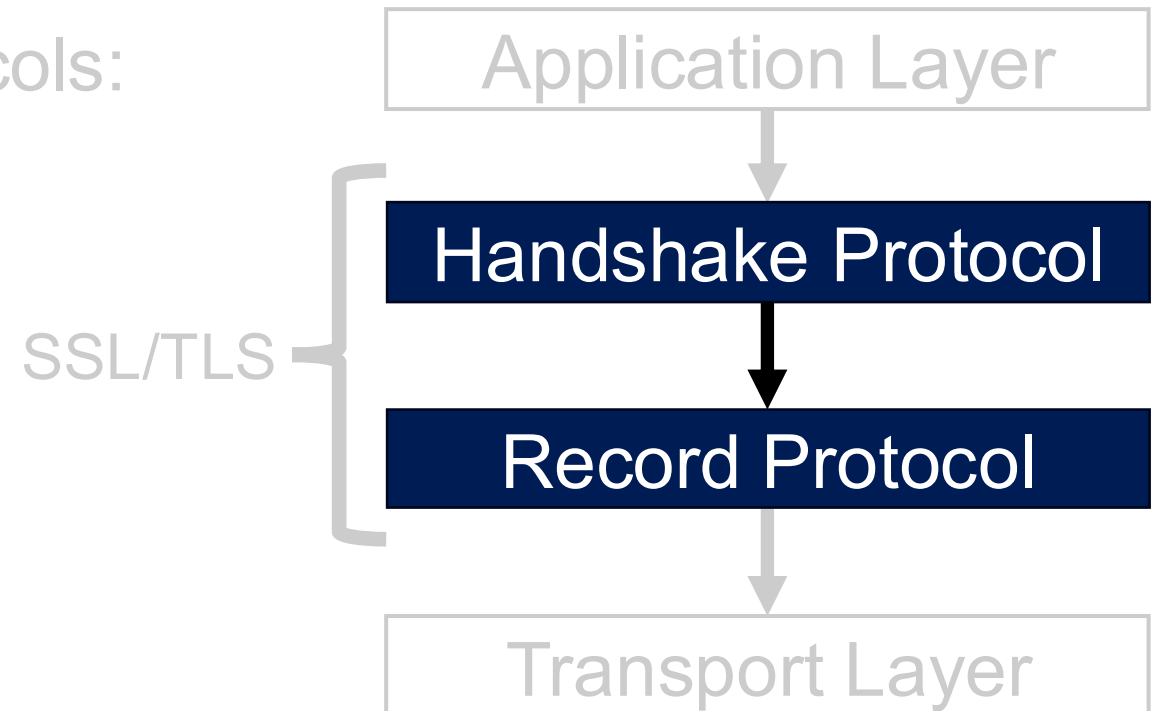
- Runs in the presentation layer
- Uses symmetric crypto, asymmetric crypto, and digital signatures
- Composed of two layers of protocols:
 1. Handshake protocol
 2. Record protocol



SSL/TLS Basics



- Runs in the presentation layer
- Uses symmetric crypto, asymmetric crypto, and digital signatures
- Composed of two layers of protocols:
 1. Handshake protocol
 2. Record protocol



SSL Record Protocol Operation

91

Application Data

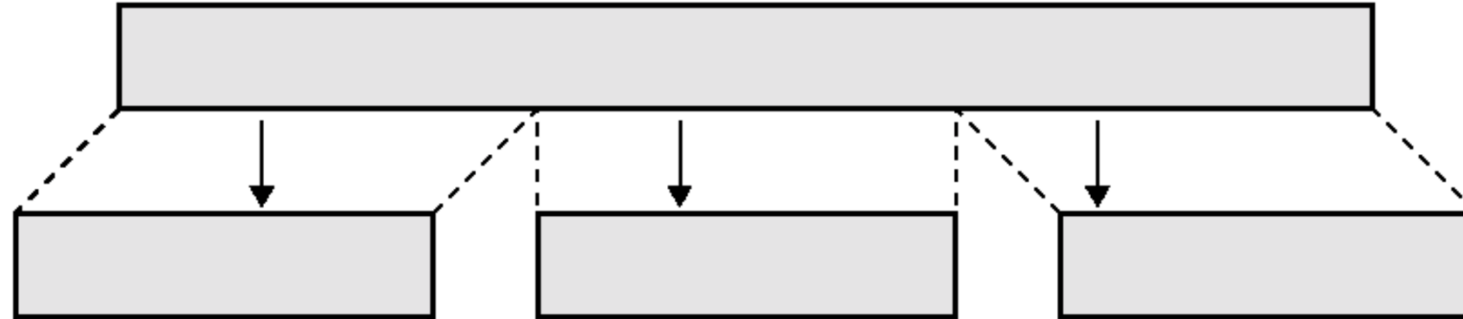


SSL Record Protocol Operation

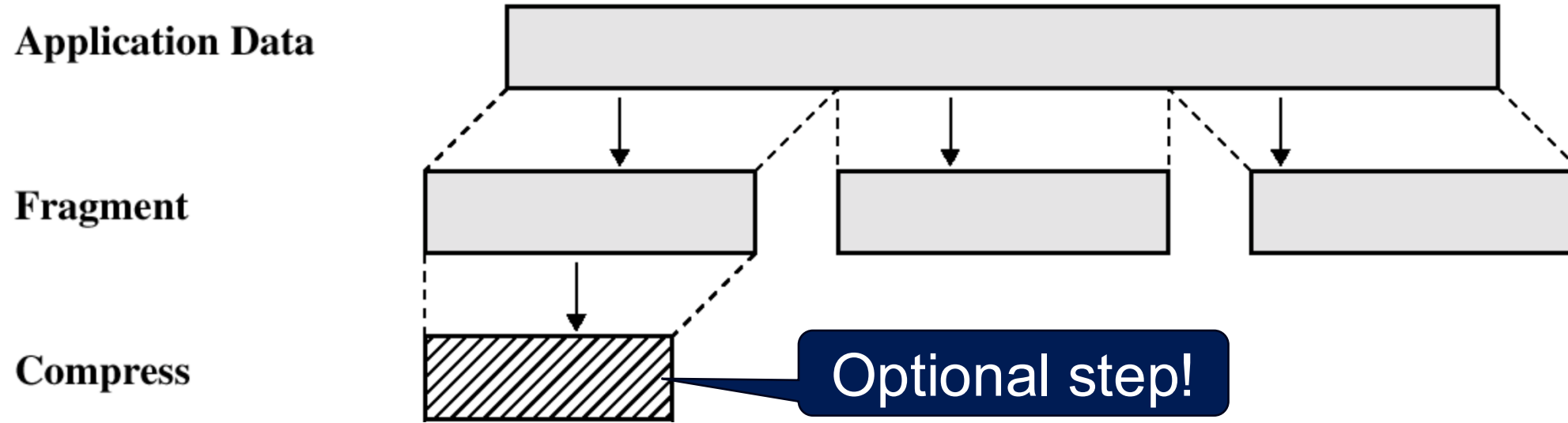
92

Application Data

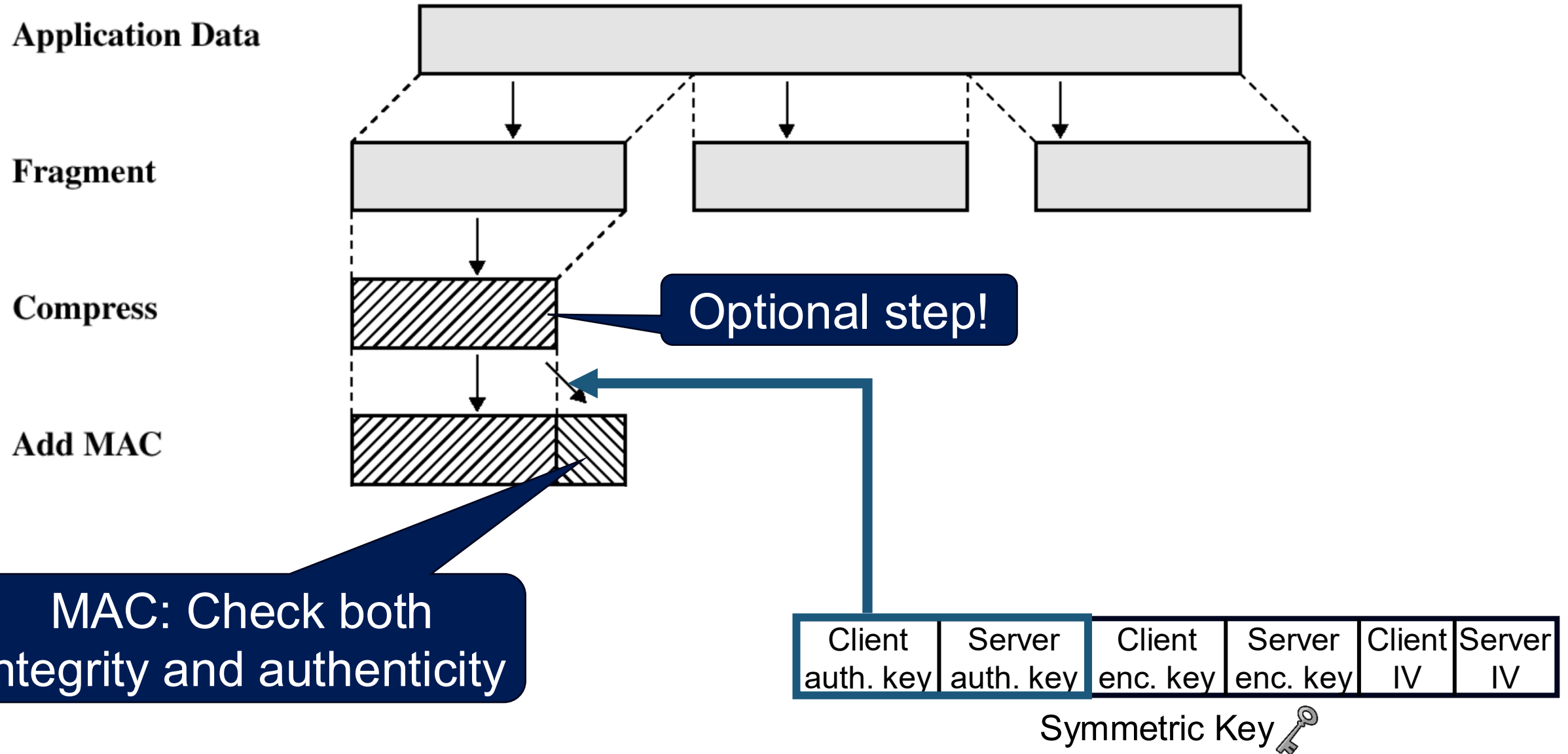
Fragment



SSL Record Protocol Operation



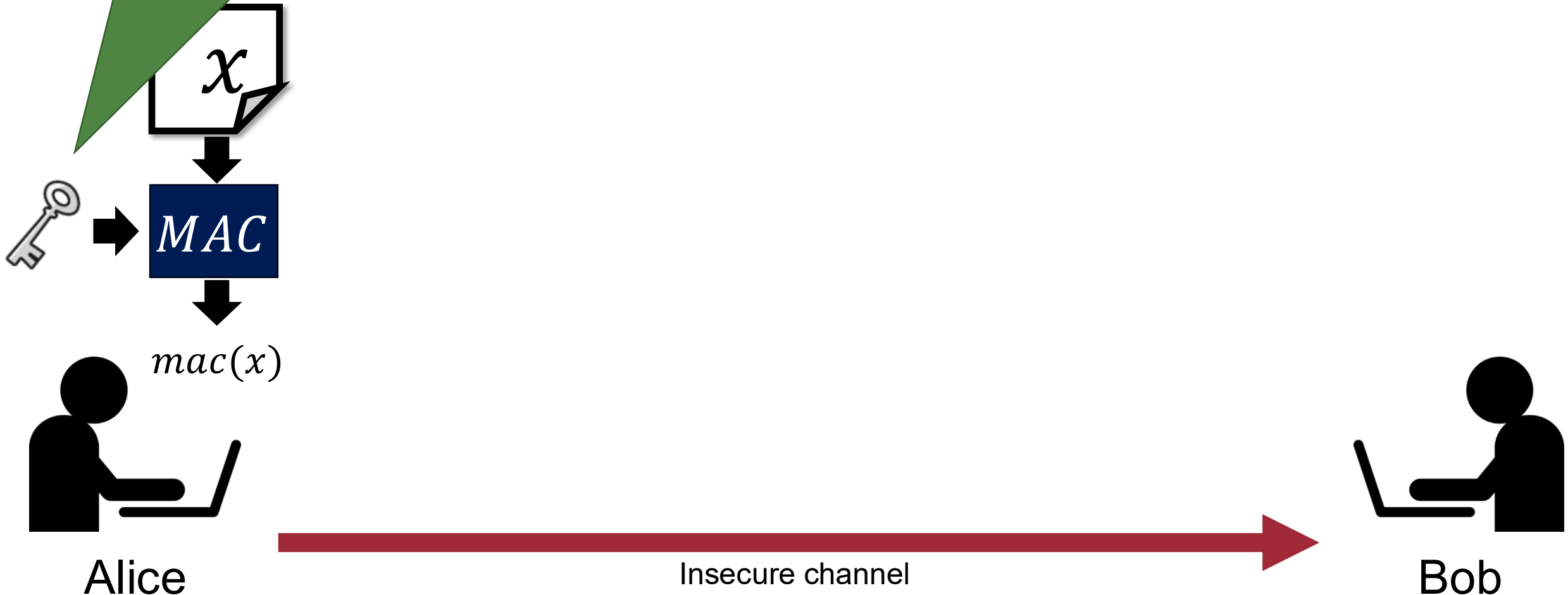
SSL Record Protocol Operation



Ref: Message Authentication Codes (MAC)

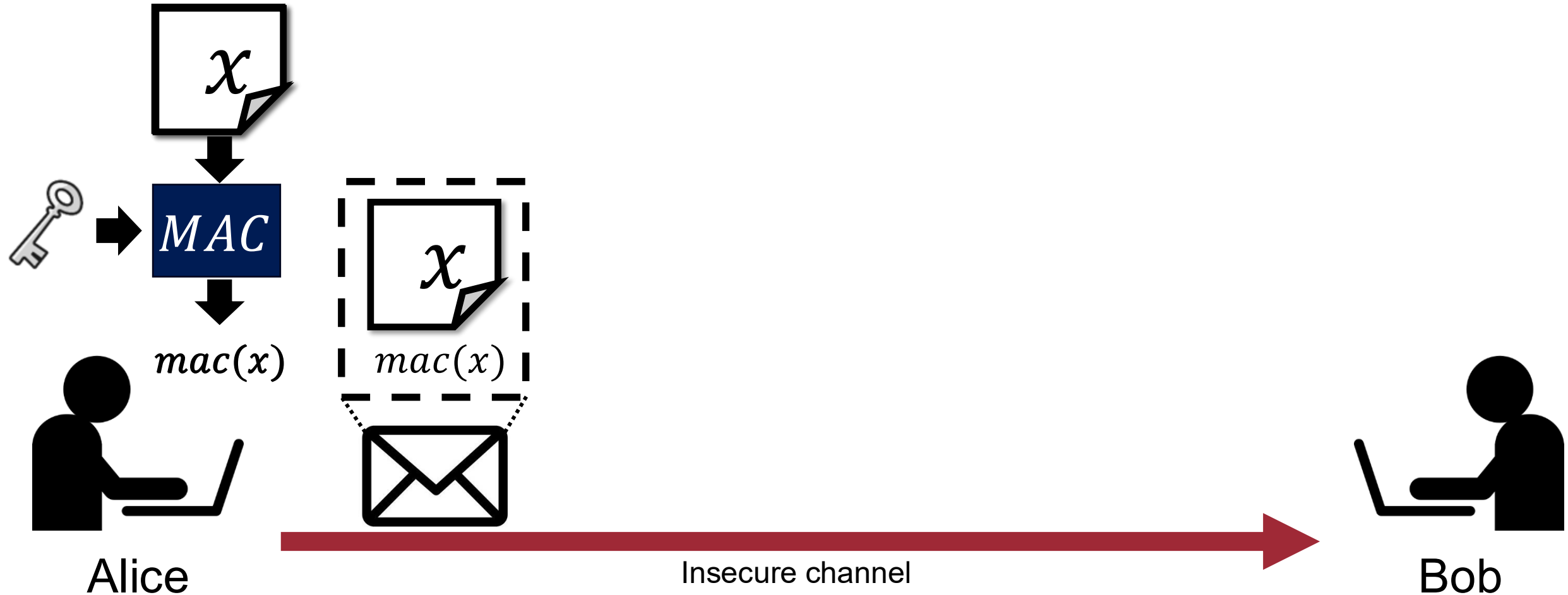
5

Use the symmetric key!



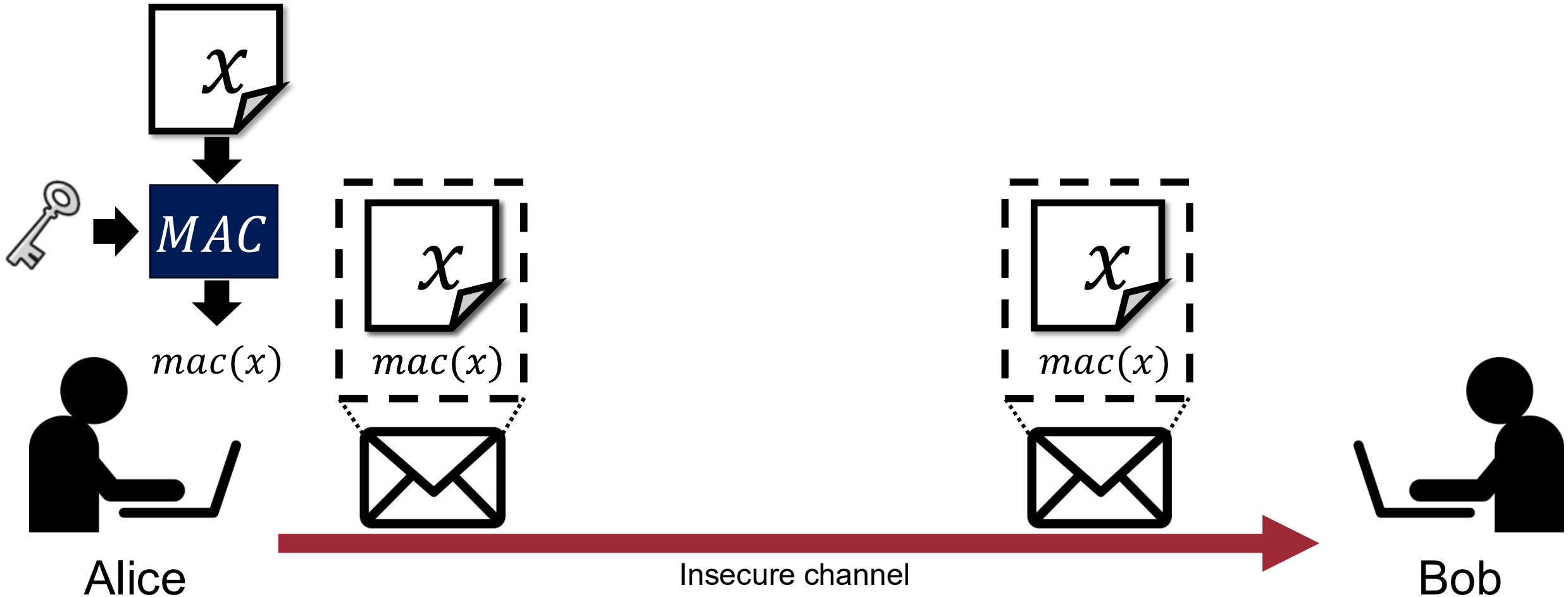
Ref: Message Authentication Codes (MAC)

56



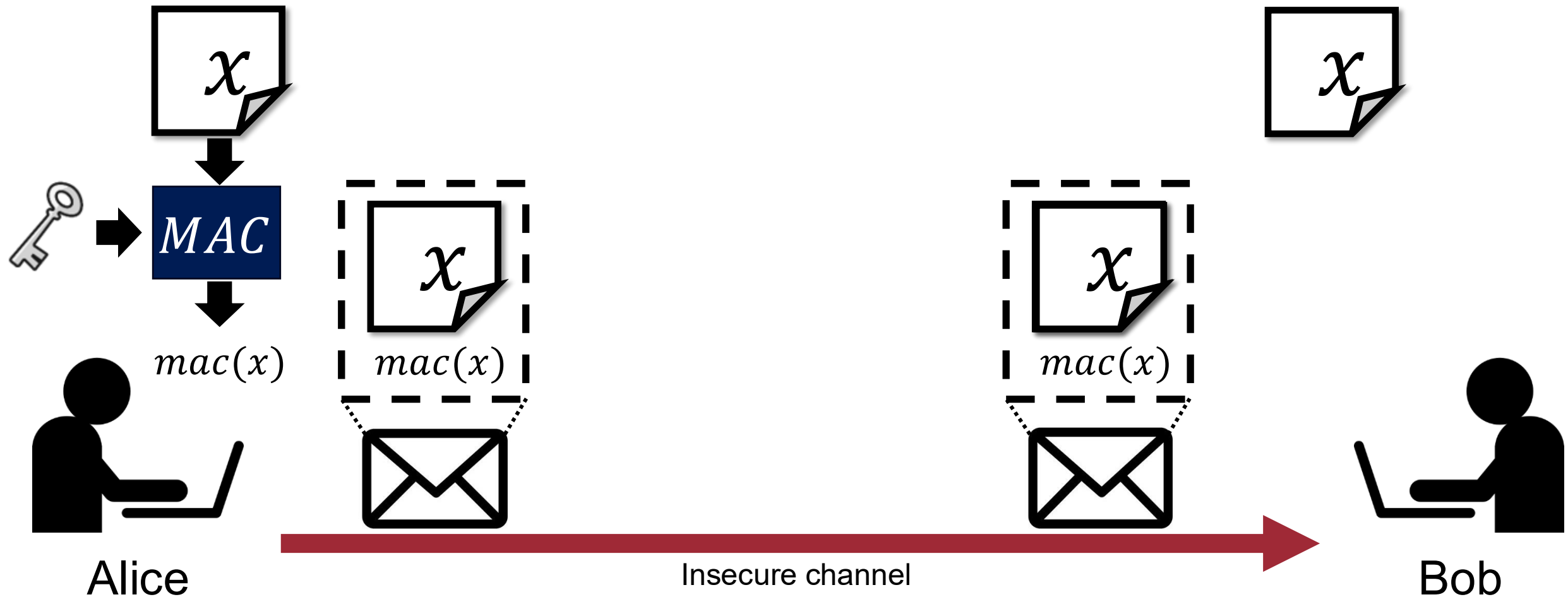
Ref: Message Authentication Codes (MAC)

97



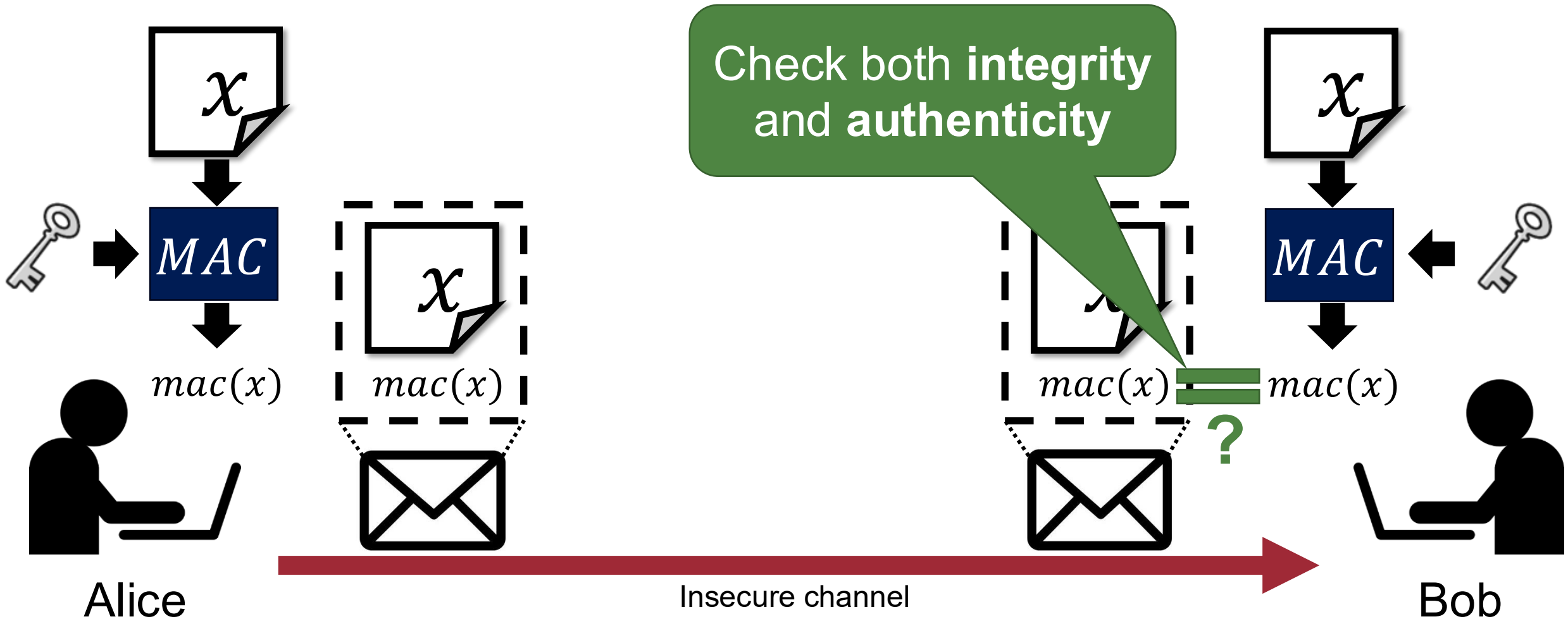
Ref: Message Authentication Codes (MAC)

53



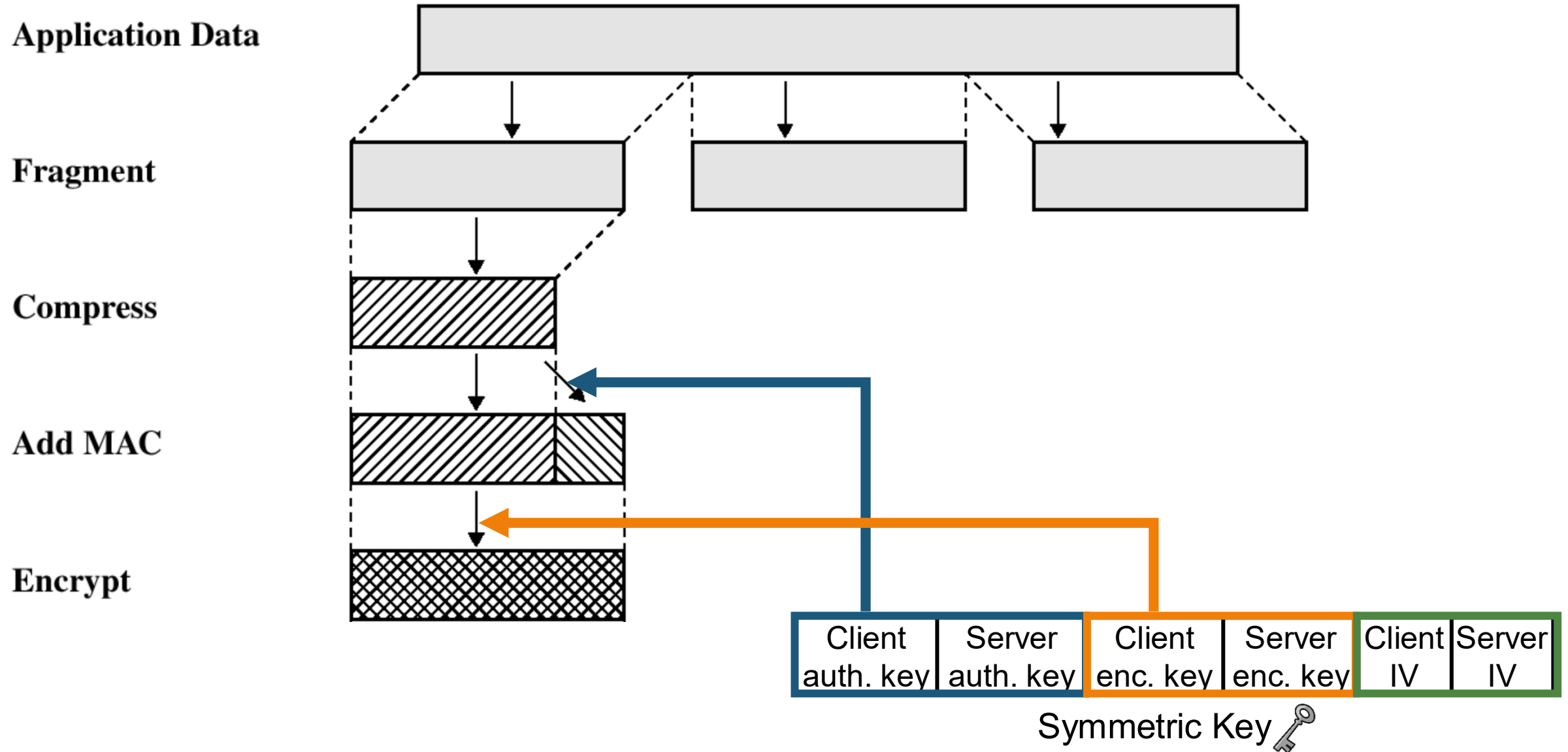
Ref: Message Authentication Codes (MAC)

99



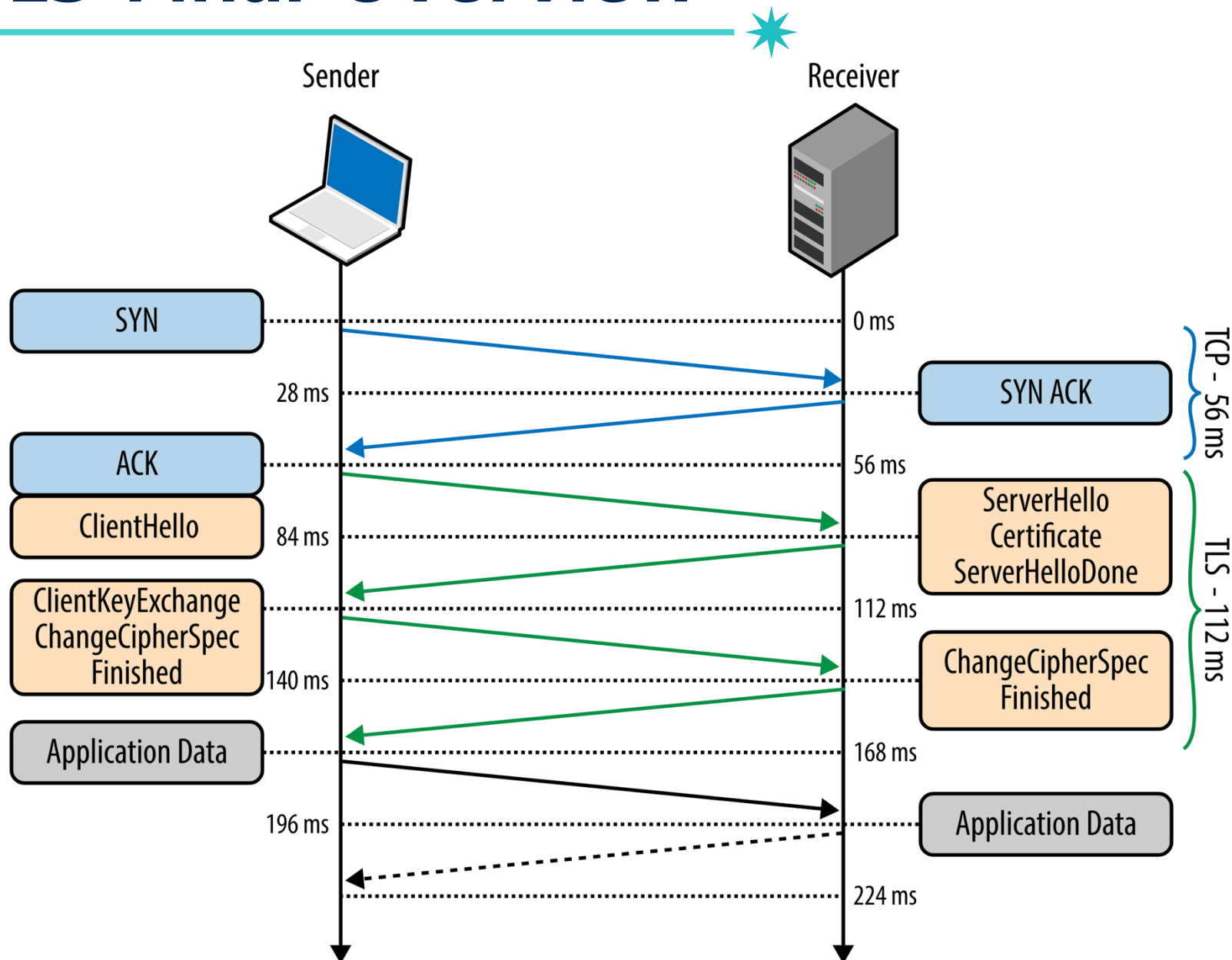
SSL Record Protocol Operation

100



SSL/TLS Final Overview

10



How SSL/TLS Provides Security Properties?

- Security goals: achieving confidentiality, integrity, and authentication

- **Confidentiality**

- Asymmetric-key algorithm for key exchange (pre-master key)
 - Symmetric-key algorithm for data exchange

- **Integrity:**

- Finished message verifies the integrity of the handshake
 - MAC for data exchange
 - If an attacker modifies the message, the recipient can detect the modification

- **Authentication**

- Authenticate the identity of the server using the server's certificate
 - MAC for data exchange

How SSL/TLS Provides Security Properties?

- Security goals: achieving confidentiality, integrity, and authentication

–Confidentiality

- Asymmetric key algorithm for key exchange (pre-master key)

Are we safe now?

- Finished message verifies the integrity of the handshake
- MAC for data exchange
- If an attacker modifies the message, the recipient can detect the modification

–Authentication

- Authenticate the identity of the server using the server's certificate
- MAC for data exchange

SSL/TLS Implementations

- Many open-source implementations of SSL/TLS are available for developers



**Can We Believe the
SSL/TLS Implementations?**

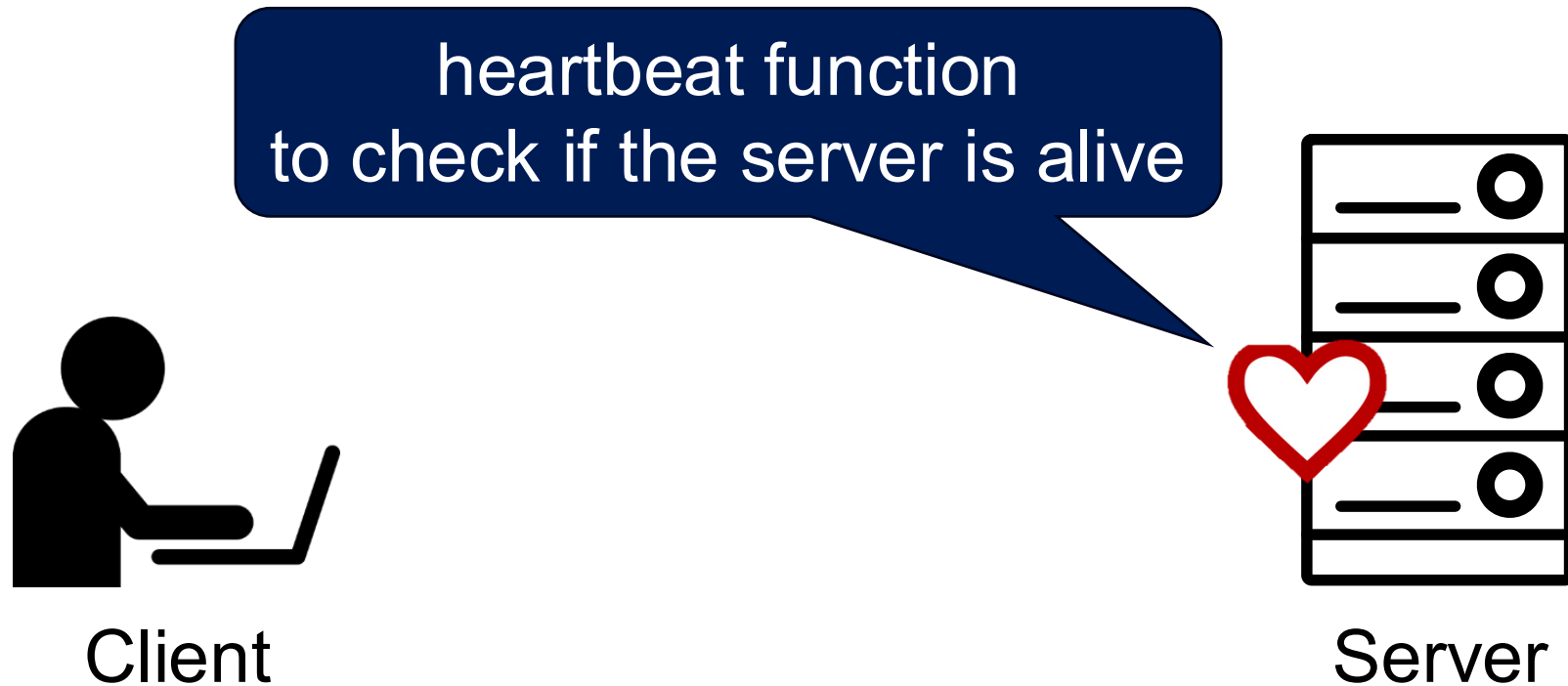
Bypass Example #1: Heartbleed Bug (in 2014)

- Famous bug in OpenSSL (in TLS *heartbeat*)
- An attacker can steal private keys



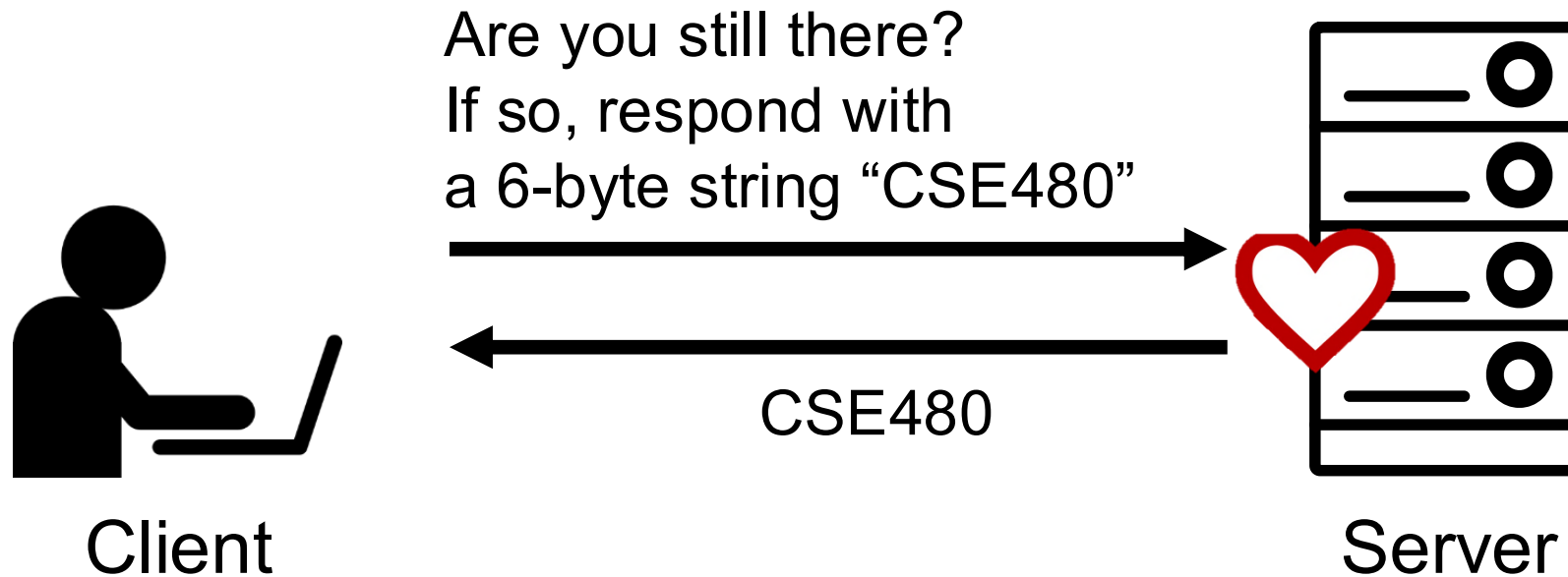
Heartbleed Bug: High-level Workflow

107



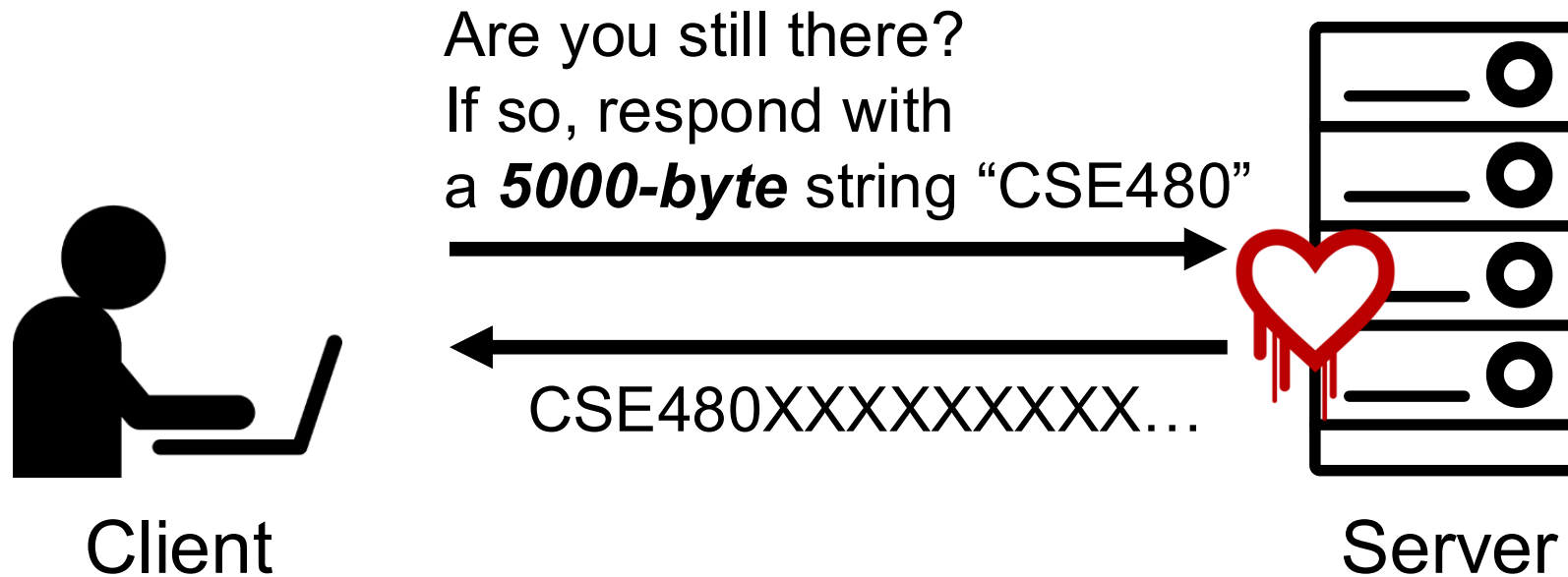
Heartbleed Bug: High-level Workflow

108



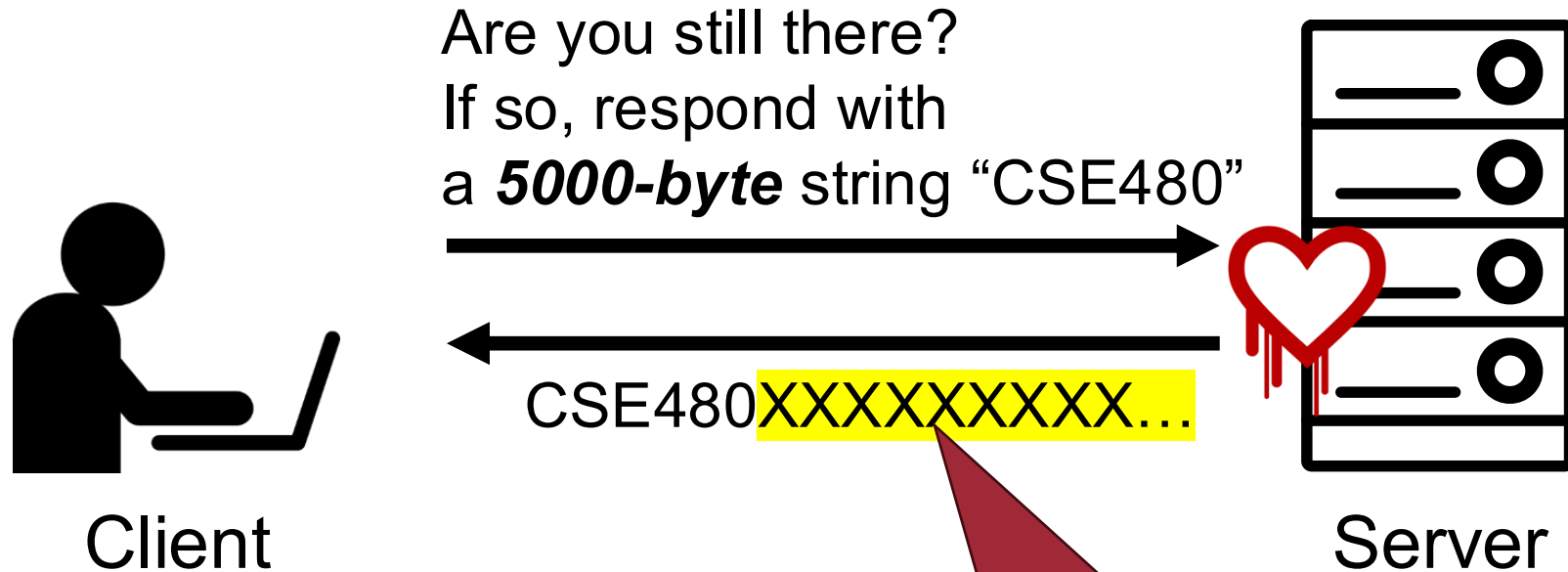
Heartbleed Bug: High-level Workflow

109



Heartbleed Bug: High-level Workflow

110



Memory disclosure!
(leak private keys)

The Bug



```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[padding_length];  
} HeartbeatMessage;
```

```
struct {  
    unsigned int length;  
    unsigned char *data;  
    ...  
} SSL3_RECORD;
```

The Bug

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[HeartbeatMessage.payload_length];  
} HeartbeatMessage;
```

Calculated from
the user's payload (i.e., 6)

Payload obtained from
HeartbeatMessage (i.e., CSE480)

```
struct {  
    unsigned int length;  
    unsigned char *data;  
    ...  
} SSL3_RECORD;
```

Obtained from
the user's input (i.e., 5000)

```
memcpy(bp, pl, length); // vulnerable spot! 
```

Copy arbitrary memory contents of a
server! TLS secret key may be available

The Bug

```
struct {  
    HeartbeatMessageType type;  
    uint16 payload_length;  
    opaque payload[HeartbeatMessage.payload_length];  
    opaque padding[HeartbeatMessage.payload_length];  
} HeartbeatMessage;
```

Calculated from
the user's payload (i.e., 6)

Payload obtained from
HeartbeatMessage (i.e., CSE480)

```
struct {  
    unsigned int length;  
    unsigned char *data;  
    ...  
} SSL3_RECORD;
```

Obtained from
the user's input (i.e., 5000)

```
memcpy(bp, pl, length); // vulnerable spot! 🐛
```

Copy arbitrary memory contents of a
server! TLS secret key may be available

Root cause:

Did not check the
consistency of the values
of the two variables!

Bypass Example #2: Fingerprint Attack

114

- Beauty and the Burst: Remote Identification of Encrypted Video Streams, **USENIX SEC'17**

Beauty and the Burst: Remote Identification of Encrypted Video Streams

Roei Schuster

Tel Aviv University, Cornell Tech

rs864@cornell.edu

Vitaly Shmatikov

Cornell Tech

shmat@cs.cornell.edu

Eran Tromer

Tel Aviv University, Columbia University

tromer@cs.tau.ac.il

Abstract

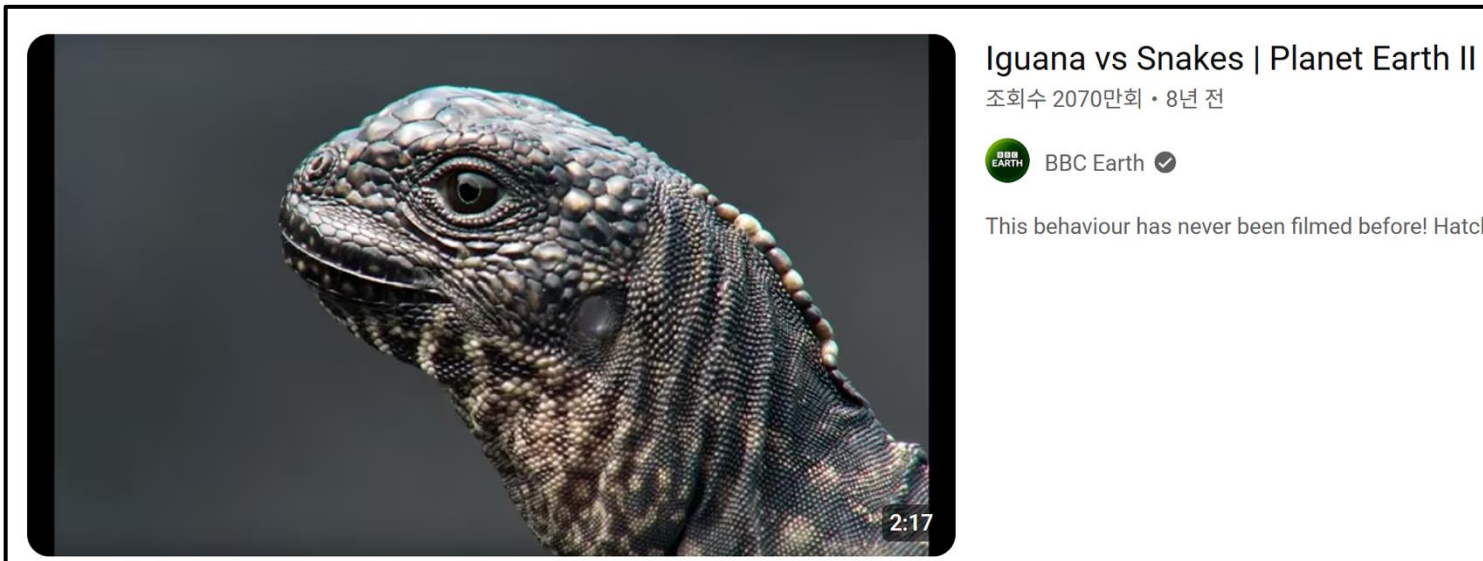
The MPEG-DASH streaming video standard contains an information leak: even if the stream is encrypted, the segmentation prescribed by the standard causes content-dependent packet bursts. We show that many video streams are uniquely characterized by their burst pat-

Our contributions. First, we analyze the root cause of the bursty, on-off patterns exhibited by encrypted video streams. The MPEG-DASH streaming standard (1) creates video segments whose size varies due to variable-rate encoding, and (2) prescribes that clients request content at segment granularity. We demonstrate that packet bursts in encrypted streams correspond to segment re-

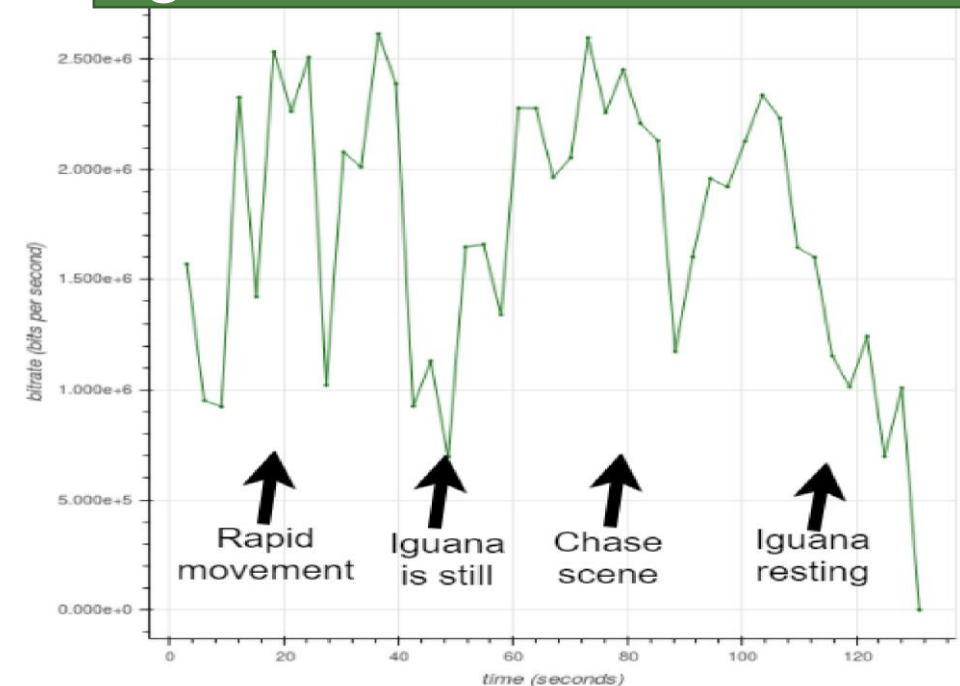
Beauty and the Burst, *USENIX SEC '17*

115

- Observation: Many video streams are uniquely characterized by their **burst patterns (Fingerprintable patterns)**
 - Even if packets are encrypted at the transport layer (e.g., using TLS), their sizes and times of arrival are visible to anyone watching the network



Iguana vs. Snakes bitrate



Scenery, movement, tension rising

116



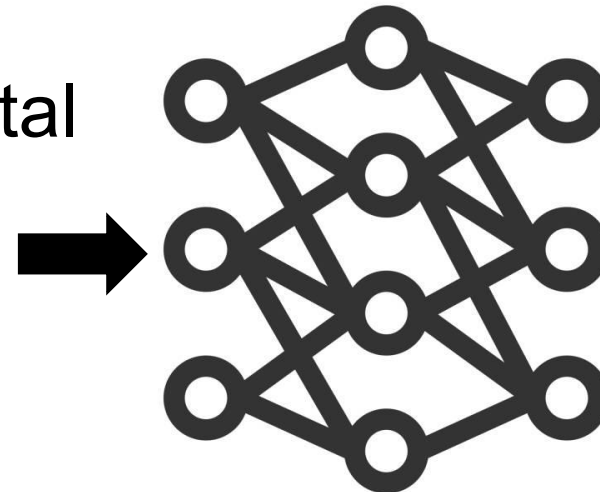
Beauty and the Burst, *USENIX SEC '17*

117

- Observation: Many video streams are uniquely characterized by their **burst patterns (Fingerprintable patterns)**
 - Even if packets are encrypted at the transport layer (e.g., using TLS), their sizes and times of arrival are visible to anyone watching the network
- Approach: ML-based video fingerprinting

Features

- Downstream/upstream/total values of bps
- Packet per second
- Average packet length
- ...



Classify the video the victim is watching

Video →

Beauty and the Burst, *USENIX SEC '17*

118



- Observation: Many video streams are uniquely characterized by their **burst patterns (Fingerprintable patterns)**
 - Even if packets are encrypted at the transport layer (e.g., using TLS), their sizes and times of arrival are visible to anyone watching the network
- Approach: ML-based video fingerprinting
- Results:
 - Youtube: 0 false positives with 0.988 recall
 - Netflix: 0.0005 false positive rate with 0.93 recall

Lesson

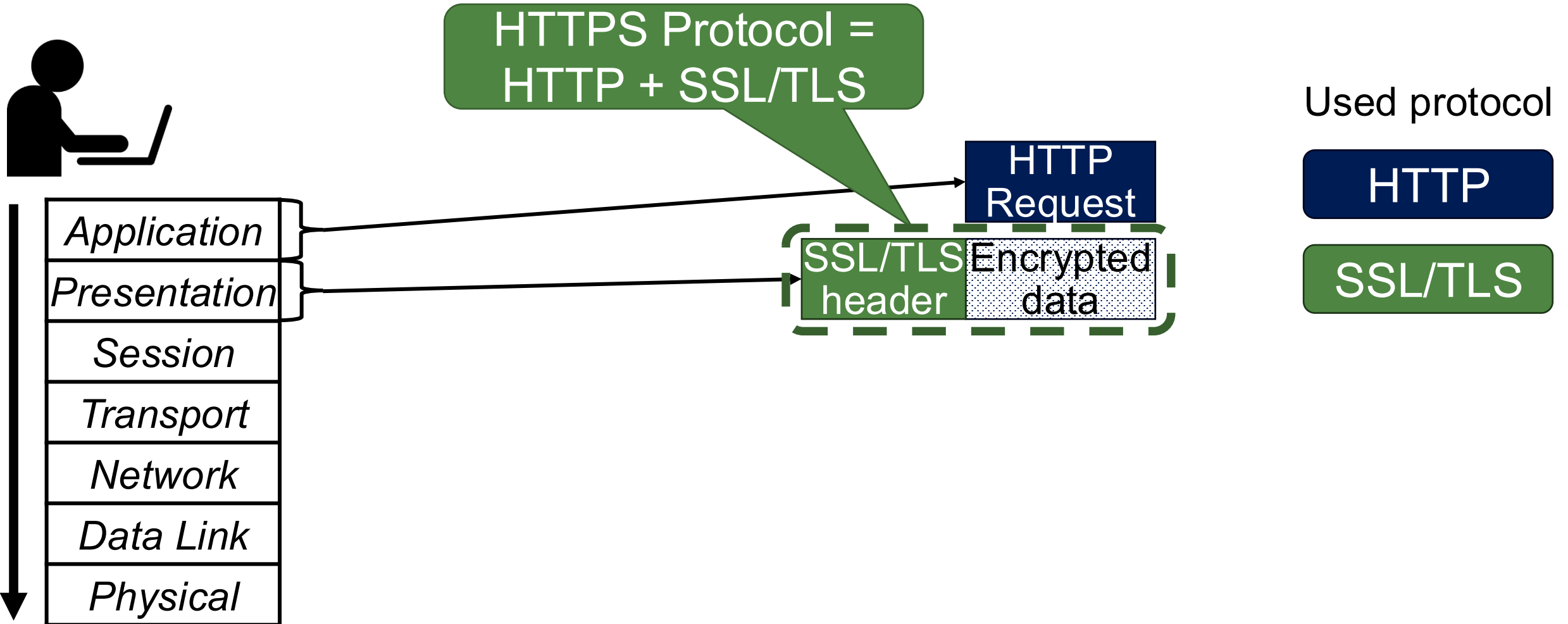


- Even if cryptographic algorithms are secure, threats can still arise in practice
- Many research questions remain:
 - How can we systematically discover vulnerabilities?
 - How can we prevent attacks before they occur?
 - How can we build secure platforms and software?

HTTPS 

HTTPS

- Adding a protocol layer for secure communication!



HTTPS – The Lock Icon



- Goal: the client (Human) can identify secure connection
 - SSL/TLS is being used to protect against active network attacker
- Lock icon should only be show when the page is secure against **network attacker**
 - All elements on the page fetched using HTTPS
 - Contents of the page have not been viewed or modified by an attacker
 - HTTPS certificate is valid – “This webpage is really comes from google.com server!”

HTTPS – The Lock Icon

123



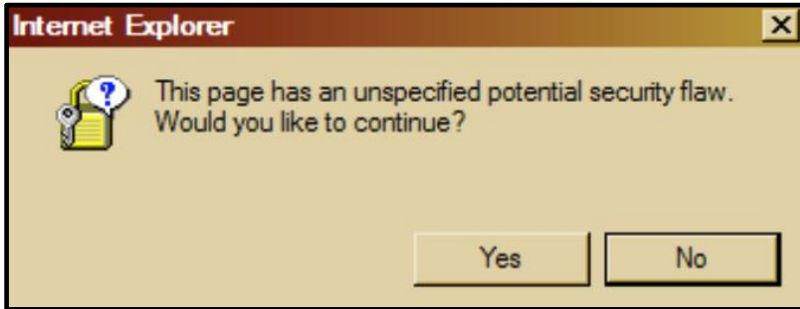
- Goal: the client (Human) can identify s
 - SSL/TLS is being used to protect against
- Lock icon should only be show when the page is secure against **network attacker**
 - All elements on the page fetched using HTTPS
 - Contents of the page have not been viewed or modified by an attacker
 - HTTPS certificate is valid – “This webpage is really comes from google.com server!”

What happens if page served over HTTPS but contains HTTP?

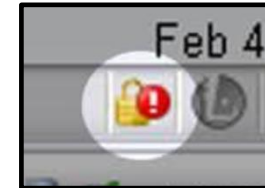
Mixed Content: Combining HTTPS and HTTP

- Page served over HTTPS but contains HTTP

- IE 7: no lock, warning

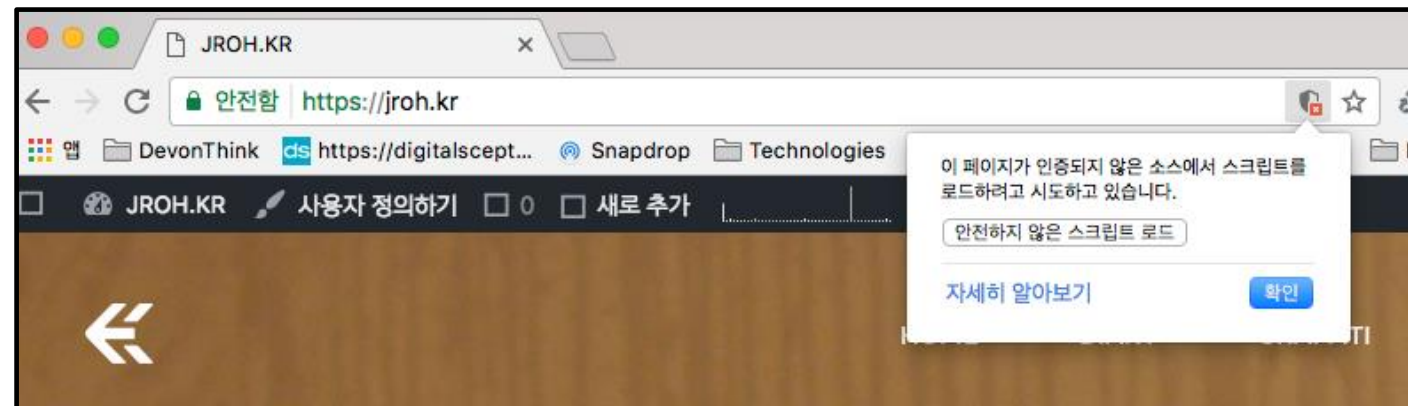


- Firefox: “!” over lock, no warning by default



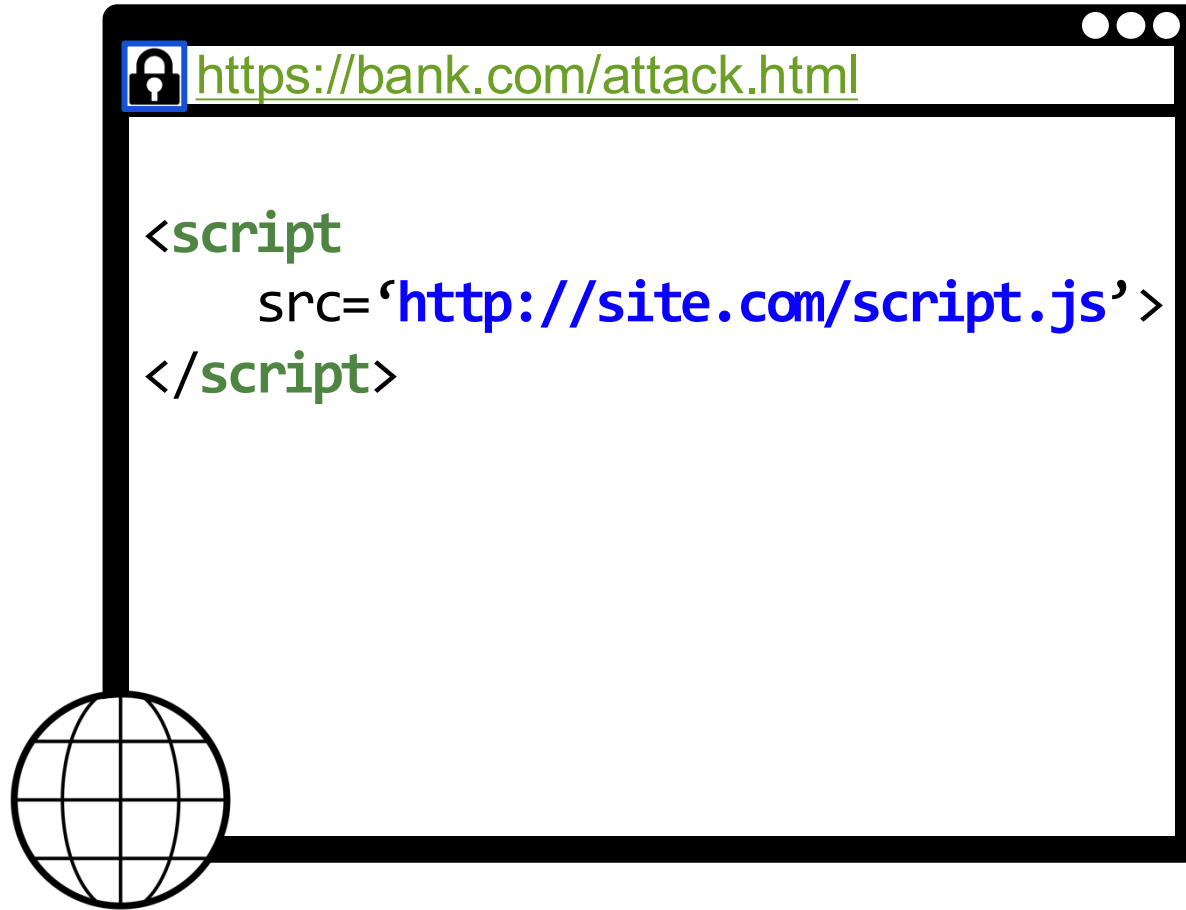
- Safari: does not detect mixed content

- Chrome: lock icon, warning



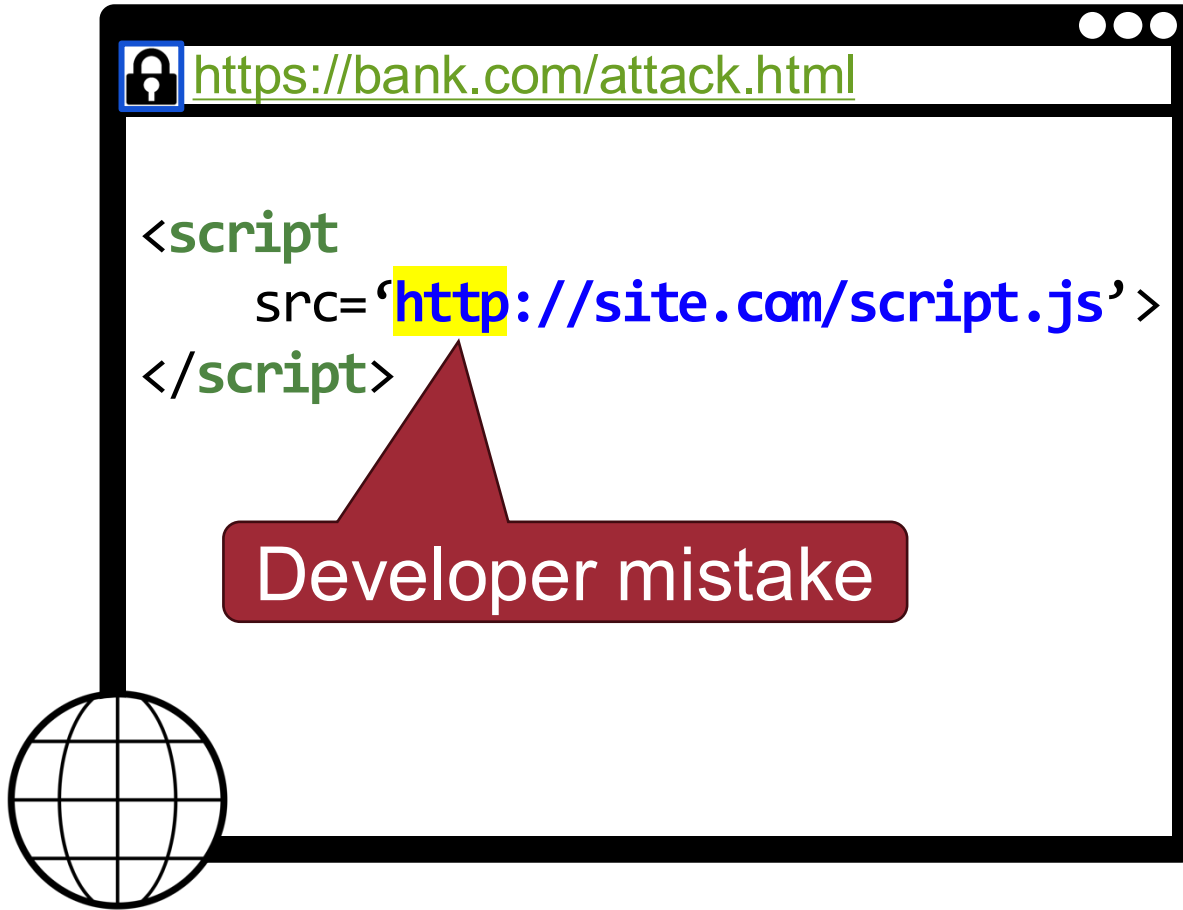
Mixed Content and Network Attacks

125



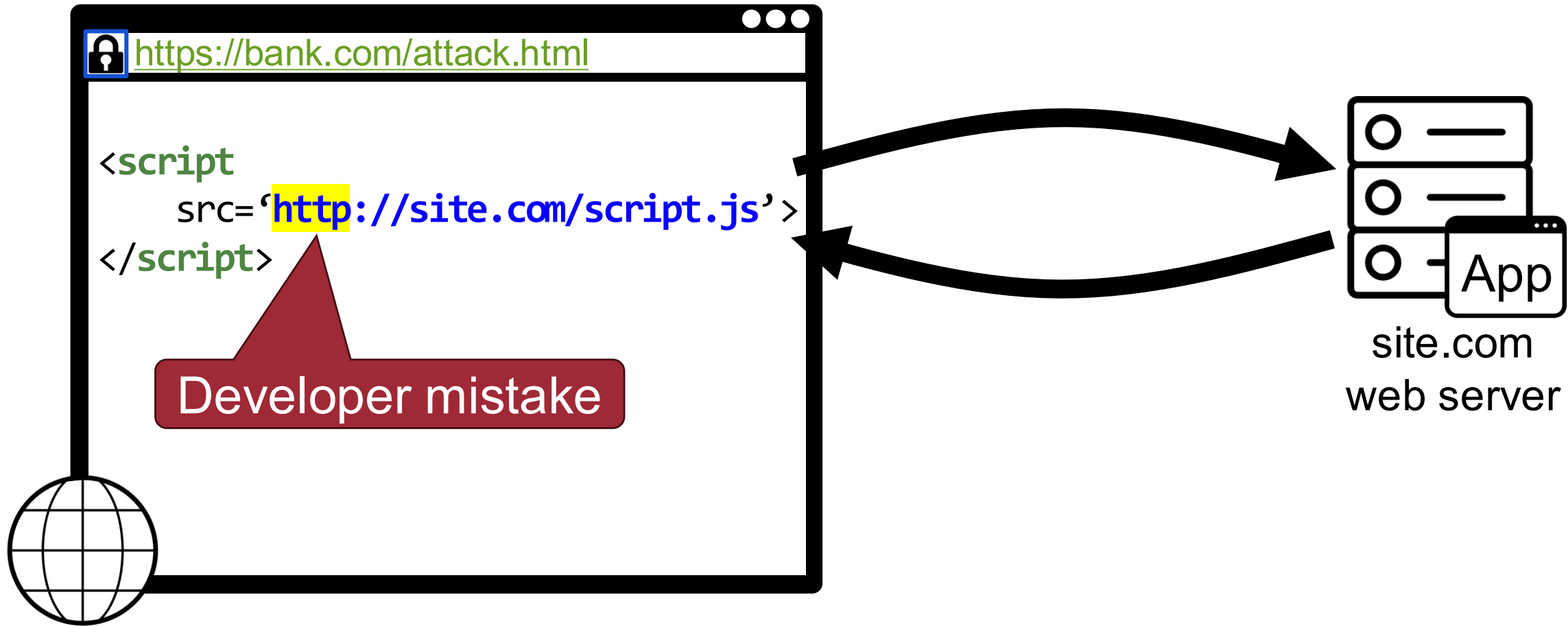
Mixed Content and Network Attacks

126



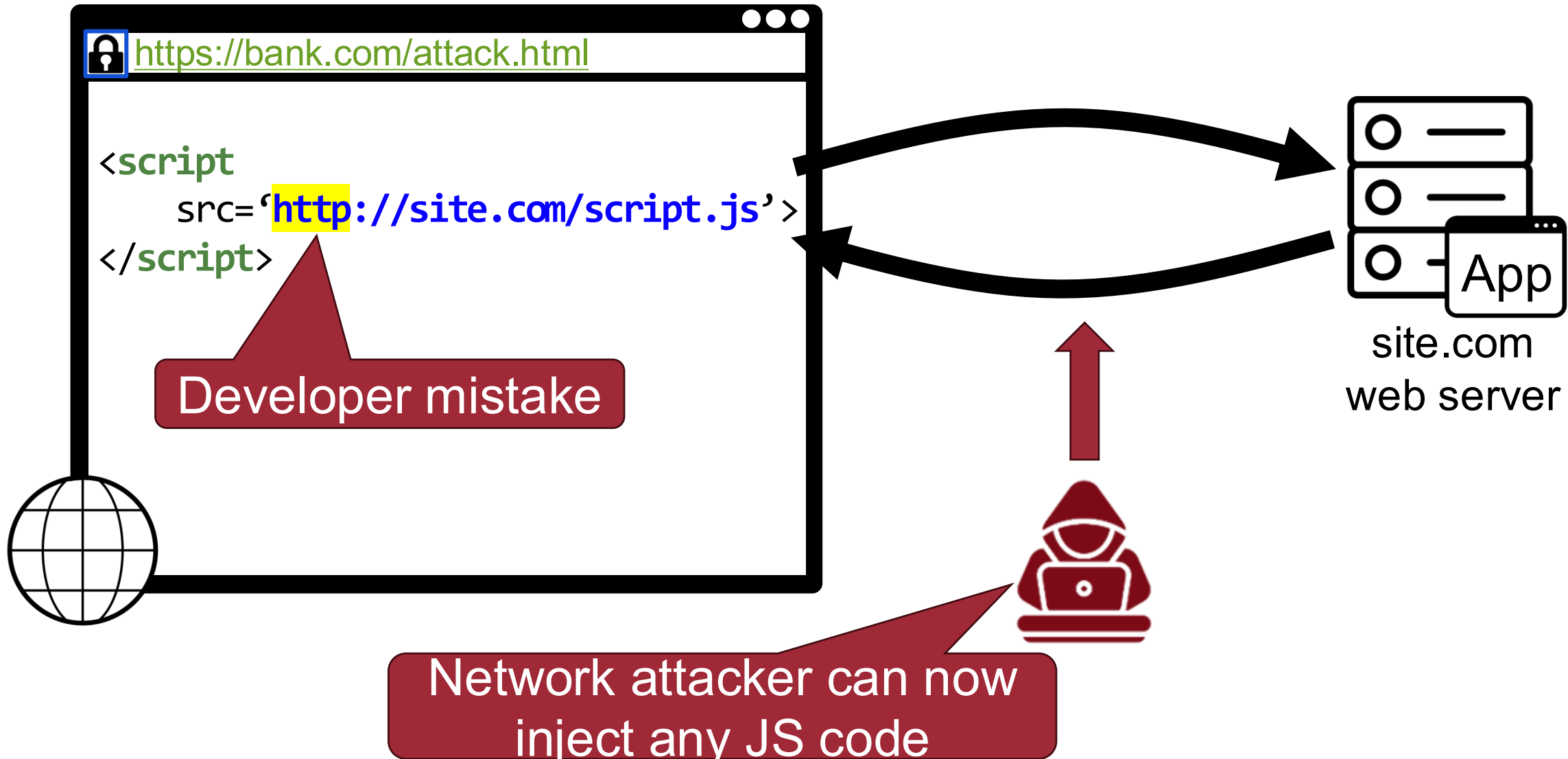
Mixed Content and Network Attacks

127



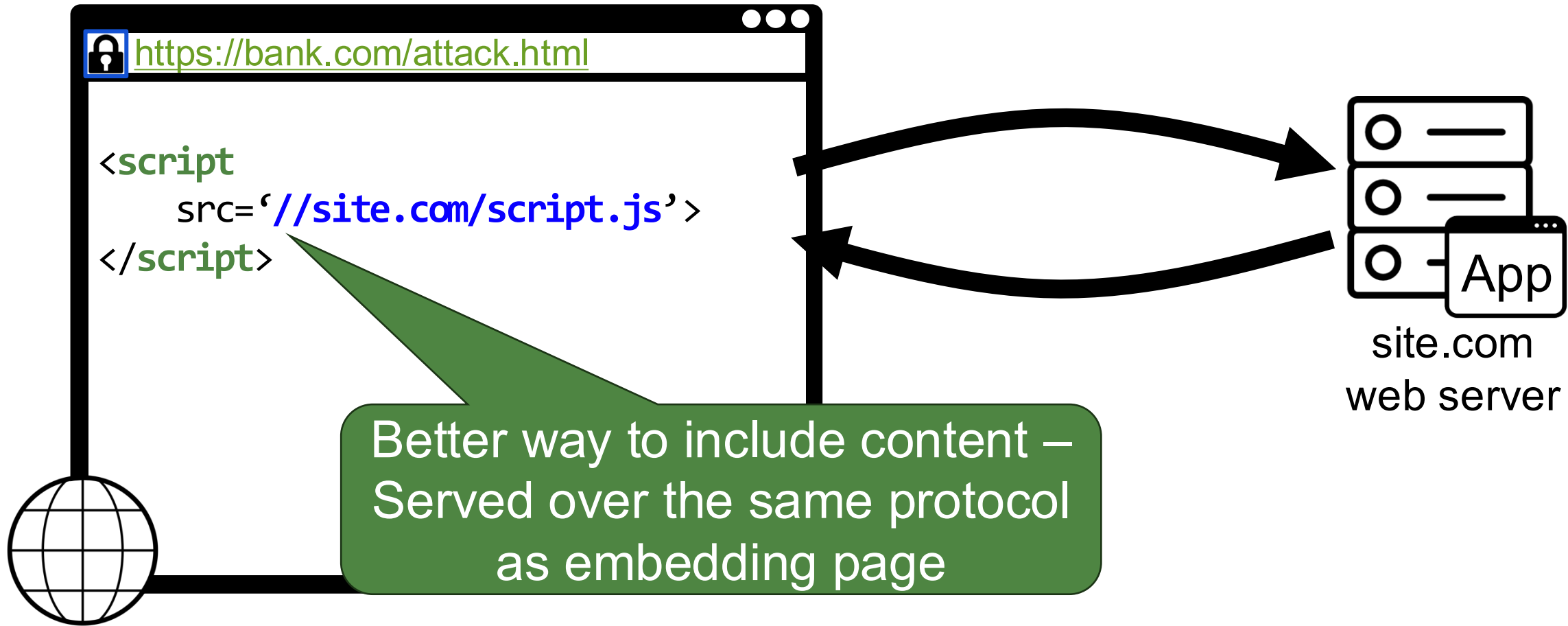
Mixed Content and Network Attacks

128



Mixed Content and Network Attacks

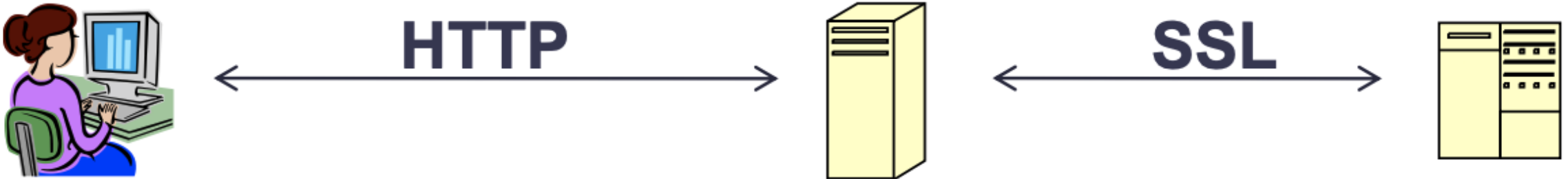
129



HTTPS – Upgrade



- Come to site over HTTP (Port no. 80), redirect to HTTPS (Port no. 443)!



Apache configuration

```
<VirtualHost *:80>  
    ServerName [Domain]  
    Redirect permanent / https://[Domain]/  
</VirtualHost>
```

Forcing HTTPs: HTTP Strict Transport Security

13



- HTTP header (Strict-Transport-Security) send by server
 - Only valid if sent via HTTPS
 - **Strict-Transport-Security: max-age=<expiry in seconds>**
 - includeSubDomains: header is valid for all subdomains
 - preload: allows for inclusion in preload list
 - Ensures that site cannot be loaded via HTTP until expiry is reached
- Enforced by the browser

Summary



- SSL/TLS protocol
 - Satisfy confidentiality
 - Satisfy integrity
 - Satisfy authentication

- HTTPS: HTTP + SSL/TLS protocol

Question?